

10. Programming with accumulators

Some calculations work most easily when they carry and update an intermediate result. However, existing construction manuals for functions that process lists or natural numbers cannot do this. We need a new programming technique for this: *accumulator programming*, and correspondingly adapted construction manuals. We will cover both in this chapter.

10.1 Carry with you interim results

[accumulators/accumulator.rkt](#)

code

We'll start with a seemingly simple function: We need a function that inverts a list, i.e., reverses the order of its elements:

```
; Invert list (: invert ((list-
of %a) -> (list-of %a)))
```

```
(check-expect (invert empty) empty) (check-expect
(invert (list 1 2 3 4)) (list 4 3 2 1))
```

The scaffold and template look like this:

```
(define invert
  (lambda (list)
    (cond
      ((empty? list) ...) ((cons?
list)
... (invert (rest list)) ... ... (first list) ...))))
```

To complete the body of the code, we can refer to the second test case – there, ``list`` is the list with the elements 1 2 3 4. Accordingly, ``(first list)`` is the number 1, and ``(rest list)`` is the list with the elements 2 3 4; that is, the recursive call returns the list with the elements 4 3 2. Therefore, to get the desired result with the elements 4 3 2 1, we must append ``(first list)`` to the end of the result of the recursive call.

We don't yet have a built-in function for appending an element *to the end* of a list – we still need to write it. We could pause our work now to do that.

Just make a note for now that we'll do it later – in the form of a short description and a signature:

```
; Append element to list (: append-
element ((list-of %a) %a -> (list-of %a)))
```

Assuming this function exists, we can easily write `invert` :

```
(define invert
  (lambda (list) (cond
    ((empty? list) empty) ((cons?
    list)
    (append element (invert (rest list))
              (first list))))))
```

The `append-element` function is very similar to the `concatenate` function from section 6.5.

First, test cases:

```
(check-expect (append-element (list 1 2 3) 4) (list 1 2 3 4)) (check-expect (append-
element empty 4) (list 4))
```

Scaffolding and template:

```
(define append-element
  (lambda (list element) (cond
    ((empty? list) ...) ((cons?
    list) ... (first list) ...
    ... (append-element (rest list) element) ...)))
```

In the `cons` case, we can also find the solution using an example: In the test case, `list` has the elements 1 2 3, so the recursive call returns 2 3 4. We just need to append (`first list`), i.e., the 1, to the beginning using `cons`. In the empty case, we append `element` to the end of an empty list – therefore, we need a single-element list containing `element`, which we could do with the built-in `list` :

```
(define append-element
  (lambda (list element)
    (cond
```

```
((empty? list) (list element)) ((cons? list)
  (cons (first list)
        (append element (rest list) element))))))
```

Here we wanted to write "Done!" as usual, but the function doesn't work: DrRacket complains that after the opening parenthesis of `(list element)` there's no function but `#<empty-list>`. Oops! This is because, although we meant the built-in function `list` in `(list element)`, we also use a parameter named `list`, and according to the rules of lexical binding from section 9.6 on page 274, this parameter overrides the built-in function.

We can solve the problem in two ways – we rename the parameter or we Construct the single-element list "by hand". We opted for the latter:

```
(define append-element
  (lambda (list element) (cond
    ((empty? list) (cons element empty)) ((cons? list)
    (cons (first list)
          (append element (rest list) element))))))
```

But back to `inverting`. Inverting lists of length 1000 takes quite a while. You can try this out by using the ``copies`` function from section 7.2 on page 208 and evaluating the output here in the REPL:

```
(invert (copies 1000 42))
```

On Michael Sperber's old computer, this takes a few seconds: that might have been acceptable in the 1970s. Inverting a list of length 400 takes *more than* twice as long as inverting a list of length 200. This is because ``invert`` calls ``append-element`` with every recursive call, and ``append-element`` itself makes as many recursive calls as the list is long. For a list of length 20, this means 19 calls to ``append-element`` for the first call, 18 calls for the second, and so on.

That makes $19 + 18 + 17 + \dots + 1$ recursive calls. Perhaps you remember – that This is an example of the Gaussian summation formula, see section 8.2 on page 221:

$$\sum_{i=0}^n i = \frac{n \times (n + 1)}{2}$$

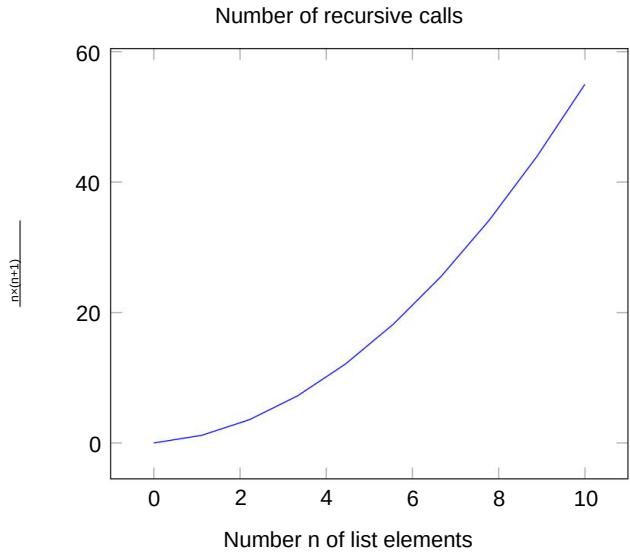


Figure 10.1: Function calls with `invert`

If you calculate this formula for ever-increasing numbers, the result becomes staggering.

It grows rapidly, as shown in Figure 10.1 . What is the reason for this?

If you multiply out the right side, you'll see:

$$\frac{n \times (n + 1)}{2} = \frac{n^2 + n}{2}$$

In the formula, n determines ² The rapid growth, also called *quadratic* .

Fortunately, there's a better way to reverse a list: The `invert` function above constructs the result list by always appending elements *to the end* . This is equivalent to...

not the "natural" construction of lists with `cons`, which appends an element *at the beginning* . The result could be constructed quite simply by appending at the beginning, in the following order.

Intermediate results are calculated, as shown here for the test case (`invert (list 1 2 3 4)`):

```
#<empty-list>
#<list 1>
#<list 2 1>
#<list 3 2 1>
#<list 4 3 2 1>
```

Each intermediate result is created from the previous one by appending an element to the beginning of the list above. This happens in the order in which the elements appeared in the original list. Lists appear: seemingly simple. However, the normal construction instructions for lists do not allow this intermediate result to be included: The result of the recursive call (`invert`) (`rest lis`) is independent of the value of (`first lis`). Therefore, the function from

It is impossible to follow the above sequence of intermediate results using the normal construction instructions, since each intermediate result is appended directly (`first lis`) to the next. will. So we need to approach the problem somewhat differently.

To carry over the intermediate result, we use a separate parameter, a so-called *accumulator*. This collects the inverted list of the elements already "seen." Here is the signature of the new function `invert-helper`:

```
; Helper function for flipping a list
(: invert-helper ((list-of %a) (list-of %a) -> (list-of %a)))
```

The following test case is intended to illustrate how the function works:

```
(check-expect (invert-helper (list 4 5 6) (list 3 2 1))
               (list 6 5 4 3 2 1))
```

The second list – the accumulator – contains the elements that have already been inverted. The first list

The data has not yet been processed; the elements are appended to the accumulator one after the other. To define the function, we first use the familiar template.

for functions that accept a list:

```
(define invert-helper
  (lambda (list inverted)
    (cond
      ((empty? list) ...)
      ((cons? list)
       ... (first list) ...
       ... (invert-helper (rest list) ...) ...))))
```

Quite a few gaps still! Let's fill in the easiest one first, in the empty case: Then, because

The function has already "seen all elements" and `inverted` them into the accumulator – which then contains the inverted input list:

```
(define invert-helper
  (lambda (list inverted)
    (cond
```

```

((empty? list) inverted) ((cons?
list) ... (first list) ...

... (invert-helper (rest list) ...) ...)))

```

In the cons case, we need to add the argument to the second parameter of `invert-helper` :

We are of course tempted to simply write "`inverted`" as with many other recursive-

We have functions before. But we still need to accumulate the elements of `list` into it.

We "ren" and use the previous intermediate result to calculate the next one. For

"accumulating in", we take the first element and append it to the beginning, just as we did in the example calculation:

```

(define invert-helper
  (lambda (list inverted)
    (cond
      ((empty? list) inverted) ((cons?
list)
...
(invert-helper (rest list)
                (cons (first list) inverted))
...))))

```

What else do we need to add? The function is already processing the input list and accumulating its elements into an `inverted` loop. It's already finished; we just need to remove the ellipses.

```

(define invert-helper
  (lambda (list inverted)
    (cond
      ((empty? list) inverted) ((cons?
list)
(invert-helper (rest list)
                (cons (first list) inverted))))))

```

The function is capable, but it's not identical to the original ``invert`` function, which only takes one input.

To "simply reverse a list," we can call ``invert-helper`` with an empty accumulator, as in this test case:

```

(check-expect (invert-helper (list 1 2 3) empty) (list 3 2 1))

```

Task 10.1

Redefine `invert` so that it uses `invert-helper` !

Ÿ

The new version of `invert` does not use a helper function and therefore makes many recursive calls.

The runtime increases *linearly with the number of elements in the input list*.

The `invert` function is so useful that it is built in under the name `reverse` .

10.2 Templates for functions with battery

We are also developing a construction guide for functions with accumulators. Before we do that, however, we want to review the construction process for such functions using another example. We will consider a function that...

which we actually already know, namely from section 6.1 on page 176:

Calculate the sum of the elements in a list of numbers

(: list-sum ((list-of number) -> number))

This time, however, we intend to work with an accumulator. To do this, we need to consider what information the accumulator should actually accumulate. This should be an *intermediate result* , a preliminary version of the desired final result. Since the final result here is the

The sum of all list elements is, if we take the sum of all list elements as an intermediate result, which our function has already "seen" and we call it `sum`. We write the following template:

```
(define list-sum-helper
  (lambda (list sum)
    (cond
      ((empty? list) ... sum ...)
      ((cons? list)
       (list-sum-helper (rest list)
                        (... (first list) ... sum ...))))))
```

Why does it look like this, or rather: What is the difference to the completely normal template for lists as input from construction instructions 18 on page 183? Here it is as a reminder. once again:

```
(define f
  (lambda (... list ...)
```

```
(cond
  ((empty? list) ...) ((cons?
    list)
  ...
  (first list) (f
    (first list))
  ...
  ))))
```

The empty branch contains the accumulator ``sum``, while the template from construction instructions 18 is empty: Here, the list has reached its end, and it's time to calculate the final result that should appear in the empty branch. Because the accumulator is an intermediate result, it must ultimately be at least close to the final result. Usually, the last intermediate result *is* the final result.

In the ``cons`` case, there is a recursive call with ``(rest list)`` as the list argument – as also shown in Construction Guide 18. Furthermore, there is help provided for calculating the accumulator argument, i.e., the next intermediate result. This should include the last intermediate result – here ``sum`` – and the next list element (``first list``). That's why they are included in the template. This part of the template is therefore simply an extension of the original template.

You might also notice that there are no ellipsis (`...`) around the recursive call : you shouldn't write anything around it. This is because the last recursive call to ``list-sum-helper`` produces the result at the end of the list – the function simply has to return it unchanged, and that's why there's nothing around it.

To complete the function, we need to define more clearly than before what exactly the ``sum`` accumulator represents. Above, we somewhat casually stated that it's the sum of all list elements the function has already "seen." However, these are no longer visible to ``list-sum-helper``. We can make them visible by including the ``list-sum`` function, which calls ``list-sum-helper``. Here's the template for that:

```
(define list-sum
  (lambda (list0)
    (list-sum-helper list0 ...)))
```

We deliberately chose the name `list0` to avoid confusion with the `list` from the list-tion `list-sum-helper` function. (Unlike `reverse` – we're trying to improve on this.)

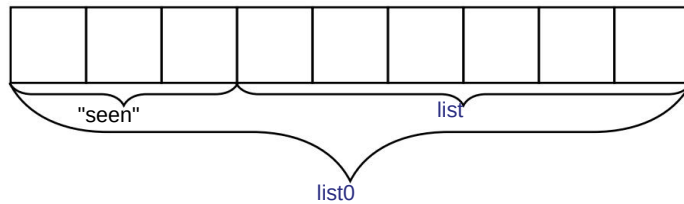


Figure 10.2: Viewed elements of a list in a function with an accumulator

Figure 10.2 shows the relationship between `list0` and `list`: `list0` is the list of *all* elements, `list` marks the position where `list-sum-helper` is currently located, and therefore consists of elements not yet seen.

Before we fill in the template, we should consider the relationship between ``list0``, ``list``, and ``sum``. Here it's still simple, but with some functions using accumulators, we'll see it's more difficult. In this case, it could look like this:

`sum` is the sum of all elements in `list0` before `list`.

We should write this statement as a comment in the function, because everything else follows from it. Because it is so important, it has its own name: *invariant*. ("Vario" means "to change" in Latin; accordingly, an invariant is something that does not change.)

To complete the function, we begin by closing the gap in ``list-sum`` that ``list-sum-helper`` calls with ``list0``. ``list0`` and ``list`` are therefore identical – there are no elements "before `list`". The sum of this empty list is 0, and thus so is the argument in the call to ``list-sum-helper``.

```
(define list-sum
  (lambda (list0)
    (list-sum-helper list0 0)))
```

Next, we'll look at the empty branch. Here, ``sum`` is the sum of all elements before ``list``, and since ``list`` is empty, that sum includes *all* elements of ``list0``. Therefore, ``sum`` is the desired final result. Current status:

```
(define list-sum-helper
  ; sum is the sum of all elements in list0 before list
  (lambda (list sum) (cond

    ((empty? list) sum)
```

```

((cons? list)
 (list-sum-helper (rest list)
                   (... (first list) ... sum ...))))))

```

The recursive call remains. Here, the *new* value of ``sum`` must be calculated, i.e., the sum of all elements before (the rest list). To do this, we must add to the existing sum (the first list) :

```

(define list-sum-helper
  ; sum is the sum of all elements in list0 before list (lambda (list sum) (cond

```

```

  ((empty? list) sum) ((cons?
    list)
    (list-sum-helper (rest list) (+ (first list) sum))))))

```

Complete!

Well, it's a bit annoying to always have to write two functions and always append ``-helper`` . We can make the function clearer by making ``list-sum-helper`` a local definition within ``list-sum`` :

```

(define list-sum
  (lambda (list0)
    (define list-sum-helper
      ; sum is the sum of all elements in list0 before list
      (lambda (list sum)
        (cond
          ((empty? list) sum) ((cons?
            list)
            (list-sum-helper (rest list) (+ (first list) sum))))))
      (list-sum-helper list0 0)))

```

Also, if " helper" bothers you, you can choose a snappier name that you like better. We'll go with "accumulate":

```

(define list-sum
  (lambda (list0) (define
    accumulate
      ; sum is the sum of all elements in list0 before list

```

```

(lambda (list sum)
  (cond
    ((empty? list) sum)
    ((cons? list)
     (accumulate (rest list) (+ (first list) sum))))))
(accumulate list0 0))

```

This example leads to the following construction instructions:

Construction instructions 21 (lists as input, with accumulator: template)

If you are writing a function that accepts a list and uses an accumulator, proceed as follows:

1. Consider what information the accumulator should represent. This is typically an intermediate result – a preliminary value for the final result. In particular, the signature of the accumulator is the same as that of the final result.
2. Construct the template as follows:

```

(define f
  (lambda (... list0 ...) (define
    accumulate (lambda (list
      acc)
        (cond
          ((empty? list) ... acc ...) ((cons? list)
            (accumulate (rest list)
              (... (first list) ... acc ...))))))
    (accumulate list0 ...)))

```

3. Formulate an invariant between list0, list and acc that is as concrete as possible and write it down. Use as a comment on `accumulate`.
4. Fill in the ellipses in the function using the invariant.

The construction guide also shows why it is more difficult to write a function with an accumulator than a "normal" function that accepts lists: You have to find an invariant, and there is little generally applicable help for this.

Task 10.2

Rewrite the `list-min-nonempty` function from section 6.8 on page 196, this time using an accumulator.

Consider what the accumulator should represent and a meaningful invariant!

Just like with `list-min-nonempty`, you need to vary the construction instructions slightly: You can divide the non-empty list into first element and remainder beforehand, and from that

Calculate the correct inputs for `accumulate`. However, `accumulate` is as in the template. $\ddot{y}$

10.3 Accumulating with natural numbers

We can also write functions that make meaningful use of an accumulator with natural numbers. We will demonstrate this using the power function, which we already programmed "normally" in section 7.1 on page 206.

Short description, signature and tests are as usual:

Calculating the power of a number

```
(: power (number natural -> number))
```

```
(check-expect (power 5 0) 1)
```

```
(check-expect (power 5 3) 125)
```

We now use the same naming convention as in construction manual 21 and name the crucial parameter with a 0 at the end.

```
(define power
  (lambda (base exponent0)
    ...))
```

The template is created in a very similar way to the lists: We write an `accumulate` function with parameters `exponent` and `acc`. `Accumulate` does exactly the same thing as in the template.

From construction instructions 19 on page 208, a branching according to the two cases of natural Numbers. The recursive call to the `positive?` branch must pass `(- exponent 1)` exactly as before :

```
(define power
  (lambda (base exponent0)
    (define accumulate
      (lambda (exponent acc)
        (cond
          ((zero? exponent)
           ... acc ...))
```

```

      ((positive? exponent)
       (accumulate (- exponent 1) ... acc ...))) (accumulate
exponent0 ...))

```

Now we need to figure out what `acc` actually is. Since the power is a repeated product of `base`, `acc` should also be a product. While `accumulate` decrements the `exponent` by one each time, we could multiply the accumulator by `base` again each time – similar to how we might do it on a piece of paper. The accumulator is therefore an "intermediate power," and we'll simply call it `power`.

We should formulate another invariant to ensure everything works correctly. For this, we need to clarify which intermediate `power` is actually `power`. Since `exponent` becomes progressively smaller, the difference between `exponent0` and `exponent` is a suitable option – this difference starts at 0 and increases incrementally. The little hat `^` in the comment means "up" here:

```

(define power
  (lambda (base exponent0)
    (define accumulate
      ; power is base^(exponent0 - exponent) (lambda (exponent
      power) (cond ((zero? exponent) ...)

      ((positive? exponent) (accumulate
        (- exponent 1) ...)))
      (accumulate exponent0 ...)))

```

We can fill in the gaps using the invariant: In the first branch, `exponent` is 0. This means that `power` is simply "base raised to the power of exponent 0" —the final result. In the second branch, we need to multiply `base` once by the intermediate result, corresponding to the reduced `exponent`. All that's missing is the value for `power` in the first call to `accumulate`. Since $b = 1$,
 We need to substitute 1:

```

(define power
  (lambda (base exponent0) (define
    accumulate
      ; power is base^(exponent0 - exponent) (lambda (exponent
      power) (cond
        ((zero? exponent) power)

```

```

((positive? exponent)
 (accumulate (- exponent 1) (* power base)))) (accumulate exponent0
1)))

```

Complete!

This also applies to accumulator functions that consume natural numbers.

We formulate a template:

Construction instructions 22 (Natural numbers as input, with accumulator:
Template)

When writing a function that accepts a natural number and uses an accumulator, proceed as follows:

1. Consider what information the accumulator should represent. This is typically an intermediate result – a preliminary value for the final result.
2. Construct the template as follows:

```

(define f
  (lambda (... n0 ...) (define
    accumulate
      (lambda (n acc)
        (cond
          ((zero? n) ... acc ...)
          ((positive? n)
           (accumulate (- n 1)
                       (... acc ...))))))
    (accumulate n0 ...)))

```

3. Formulate and write down as concrete an invariant as possible between n_0 , n and acc . as a comment on `accumulate`.
4. Fill in the ellipses in the function using the invariant.

Task 10.3

Program a function that calculates the *factorial* of a number.

For a number n , its factorial $n!$ is the following product:

$$1 \times 2 \times \dots \times (n - 1) \times n$$

Here's another tip for functions on natural numbers with an accumulator. For the [exponent](#) and [list-sum](#) functions, it doesn't matter whether the function counts up from 0 or from a number.

n counts down: In addition and multiplication, the order doesn't matter. But that's...

Not always so: Often it makes a difference whether the function starts at 0 (or sometimes also

1) starts and counts up or counts down from a number n . (If necessary, refer back to section [7.3](#) on page [211](#).) To decide this, consider whether you will use a pencil and

You would start the paper "at the bottom" or "at the top".

10.4 ANALYZING STOCK PRICES

[accumulators/profit.rkt](#)

code

There are indeed functions that require multiple intermediate results. To demonstrate this, we will write a function that calculates the maximum profit from a series of stock prices.

We represent this series of consecutive courses as a list of numbers. Accordingly, the short description and signature look like this:

[Determine the best possible profit through buying and selling](#)

(: max-profit ((nonempty-list-of real) -> real))

Profit is made here by first buying the stock and then selling it again – and

Not the other way around. The difference between the selling price and the purchase price is then the profit. The following test case illustrates this:

(check-expect (max-profit (list 5 2 7 3 9 5 1)) 7)

The maximum profit could have been achieved by buying at price 2 and selling at price 9.

The scaffolding looks like this:

```
(define max-profit
  (lambda (spots)
    ...))
```

("Spot" is an English word for "course".)

But what happens next?

The simple strategy is to simply calculate the difference between the maximum and minimum of the list as Unfortunately, it's not possible to display the maximum profit: The minimum is 1, but unfortunately it's not high enough. behind the maximum of 9.

This problem is equally unsuitable for a naive, "normal" recursive function. This would be the template:

```
(define max-profit (lambda
  (spots) (cond

    ((empty? spots) ...) ((cons?
  spots) ... (first
    spots) ... ... (max-profit (rest
  spots) ...))))
```

The problem is that the profit of the rest of the list isn't enough to calculate the profit of the entire list using only the profit from the first item. We would also need to know the corresponding selling price.

So let's try some intermediate results. The template looks like this:

```
(define max-profit (lambda
  (spots0) (define
    accumulate

      (lambda (spots acc) (cond

        ((empty? spots) acc) ((cons?
        spots) (accumulate
          (rest spots) ... (first spots) ...
            acc ...)))) (accumulate spots0 ...)))
```

Since the function is intended to calculate the maximum profit overall, it makes sense to include the maximum profit of the previously observed prices between `spots0` and `spots` as an intermediate result:

```
(define max-profit (lambda
  (spots0) (define
    accumulate

      ; max-profit is the maximum profit between ; spots0 and spots
      (lambda (spots max-profit)

        (cond

          ((empty spots) max-profit)
```

```

      ((cons? spots)
       (accumulate (rest spots) ... (first
                                spots) ... max-profit ...))))
(accumulate spots0 0))

```

The problem is that we still don't have enough data to calculate a new value for ``max-profit`` when recursively calling ``accumulate``. Basically, there are only two possibilities: • ``max-profit`` remains as it is. • ``max-profit`` is updated because

it's better to sell at the `first spot` price
 than
 at the best selling price to date.

So we need to find out what the profit would be if we sold at the `first spot` price. To do this, we need to know the best possible *buying price*. This buying price is the lowest price among the past prices: We carry this minimum as a further intermediate result. This minimum is updated using the built-in `min function`.

```

(define max-profit (lambda
  (spots0) (define
    accumulate
      ; min-spot is the minimum of the elements between ; spots0 and spots ;
      max-profit is the maximum
      profit between ; spots0 and spots (lambda (spots min-spot max-profit)

        (cond
          ((empty? spots) max-profit) ((cons?
            spots) (accumulate
              (rest spots) (min (first spots) min-
                spot) ...))))))

(accumulate spots0 ... 0))

```

The correct initial value for ``min-spot`` is still missing when ``accumulate`` is first called. Here, we take advantage of the fact that ``spots0`` is a non-empty list: We can split it into the first element and the rest, as we did with ``list-min`` in section 6.8 on page 196 and in exercise 10.2 on page 299. The first element is then the first minimum.

```

(accumulate (rest spots0) (first spots0) 0))

```

All that's missing is the updated value for `max-profit`. This is the maximum of the previous `max-profit` and the potential profit from selling at the current price (`first spots`). Here's the result:

```
(define max-profit (lambda
  (spots0) (define
    accumulate
      ; min-spot is the minimum of the elements between ; spots0 and spots ; max-
      profit is the maximum profit
      between ; spots0 and spots (lambda (spots min-spot max-profit)

        (cond
          ((empty? spots) max-profit) ((cons? spots)
            (accumulate (rest
              spots) (min (first spots) min-spot) (max
                (- (first spots) min-spot) max-profit))))))

    (accumulate (rest spots0) (first spots0) 0)))
```

Complete!

Task 10.4

The *Fibonacci sequence* is a sequence of natural numbers that begins with 0 and 1. Each subsequent number is the sum of the two numbers before it.

Write a function that calculates the *n*th number from the Fibonacci sequence for a given number *n*.

Write a function with two accumulators to achieve this.

Important for this task: Before you begin, consider whether you would calculate the Fibonacci numbers with paper and pencil, starting at 0 and counting upwards, or counting downwards from *n*. Write your function accordingly. ÿ

10.5 CONTEXT AND FINAL RECURSION

In this section, we'll examine how recursive function calls are evaluated. This will highlight an important difference between functions with an accumulator and the "normal" functions that preceded them.

As an example, let's look at `list-sum` again, first in the accumulator version from section 10.2 on page 295. It's best if you run the evaluation yourself in the Stepper.
from

```
(list-sum (list 1 2 3 4))
```

You're looking at it. Here are the most important steps in the evaluation:

```
(list-sum #<list 1 2 3 4>)
```

```
Ÿ (accumulate #<list 1 2 3 4> 0)
```

```
Ÿ (accumulate (rest #<list 1 2 3 4>) (+ (first #<list 1 2 3 4>) 0)) Ÿ (accumulate #<list 2 3 4> 1)
```

```
Ÿ (accumulate #<list 3 4> 3) Ÿ (accumulate #<list
```

```
4> 6) Ÿ (accumulate #<empty-list> 10)
```

```
Ÿ 10
```

We have always written the value of the sum parameter one below the other, and you can clearly see how the intermediate result gradually moves from 0 towards the final result of 10.

If you run the "old" `list-sum` in section 6.1 on page 6.1 in the Stepper, it looks like this:
It looks completely different:

```
(list-sum #<list 1 2 3 4>)
```

```
Ÿ (+ (first #<list 1 2 3 4>) (list-sum (rest #<list 1 2 3 4>)))
```

```
Ÿ (+ 1 (list-sum #<list 2 3 4>)) Ÿ (+ 1 (+ 2 (list-sum  
#<list 3 4>)))
```

```
Ÿ (+ 1 (+ 2 (+ 3 (list-sum #<list 4>))))
```

```
Ÿ (+ 1 (+ 2 (+ 3 (+ 4 (list-sum #<empty-list>)))) Ÿ (+ 1 (+ 2 (+ 3 (+ 4 0))))
```

```
Ÿ (+ 1 (+ 2 (+ 3 4))) Ÿ (+ 1 (+ 2
```

```
7)) Ÿ (+ 1 9)
```

```
Ÿ 10
```

Here you can see that the recursive calls "accumulate" the individual additions, and the actual work of the addition only takes place after the last recursive call. This "accumulation"

This is because the recursive call in the old version is inside the call to + :

```
(+ (first list) (list-sum (rest list)))
```

In the version with an accumulator, it's exactly the opposite, and the recursive call is centered around the calling `+` around:

```
(accumulate (rest list) (+ (first list) sum))
```

When the computer in the old version evaluates a recursive call, it has to remember that an addition still needs to happen after the recursive call. Accordingly, the chain of `+` calls gets longer with each recursive call. This call to `+` is called the *context* of the recursive call – it's wrapped around it.

In the accumulator version, the recursive call to ``accumulate`` has no context; consequently, nothing accumulates during evaluation or in the stepper. A recursive call without context is called *tail recursive* because nothing further needs to happen after the call; the call is therefore "at the end."

The term "tail recursion" is somewhat unfortunate: whether a function call has a context or not actually has nothing to do with whether it is recursive or not. The better term in English is "*tail call*".
 , which applies to both recursive and non-recursive calls.

The fact that the call in the old ``list-sum`` function is not tail-recursive is already determined by the template, in which the result of the recursive call must still be combined with the first element of the list. This is not the case with the template for functions with an accumulator; consequently, the resulting recursive calls have no context.

The evaluation processes generated by tail-recursive calls proceed in a straight line and are also called *iterative* processes. In other programming languages, these are also referred to as *loops*; many programmers therefore use the name ``loop`` instead of ``accumulate``.

10.6 THE REVERSED LIST PHENOMENON

We now know two variants of ``list-sum``: the original "normal recursive" version and the iterative one. For all the functions we've written on lists so far, it's also possible to write a tail-recursive version. The tail-recursive versions are not better.

On the contrary, they are actually more difficult to write. Therefore, in this chapter, this is just a finger exercise. However, we will show in Chapter 15 on page 465 that in many other programming languages we *have to write list functions iteratively*. That's why it's good that we've already done this.

When writing iterative functions that return lists, there's a small detail to keep in mind. We'll demonstrate this detail using a function that returns a list.

A list of integers extracts the even numbers. This can also be done by simply calling `filter`, but we'll program it again for practice. The short description, signature, and test case are as follows:

Extract even numbers from a list (: evens ((list-of integer) ->
(list-of integer)))

(check-expect (evens (list 1 2 3 4 5 6)) (list 2 4 6))

The template provides the following:

```
(define evens
  (lambda (list0)
    (define accumulate
      (lambda (list acc) (cond
                           ((empty? list) ... acc ...) ((cons? list)
                                                           (accumulate (rest
                                                                    list)
                                                                    ... (first list) ... acc ...))))))
    (accumulate list0 ...)))
```

We need to think of a suitable accumulator again: It makes sense to collect the even numbers we've already seen. So we call the accumulator, just like the function, `evens`, and write a suitable invariant:

```
(define evens
  (lambda (list0)
    (define accumulate
      ; evens contains the even numbers between list0 and list (lambda (list evens)

      (cond
        ((empty? list) ... evens ...) ((cons? list)

        (accumulate (rest list)
                    ... (first list) ... evens ...))))
    (accumulate list0 ...)))
```

The first value for the accumulator must be `empty` according to the invariant. Also according to the invariant, `evens` is the final result. Furthermore, in the recursive call to `accumulate`, we need to calculate a new value for `evens`: We append (`first list`) to the previous intermediate result if it is even:

```
(define evens
  (lambda (list0)
    (define accumulate
      ; evens contains the even numbers between list0 and list
      (lambda (list evens) (cond

        ((empty? list) evens) ((cons?
          list) (accumulate
            (rest list)

              (if (even? (first list))
                (cons (first list) evens) evens))))))

    (accumulate list0 empty)))
```

Looks good, right? There's just one small problem: the test case fails. The function returns `#<list 6 4 2>` instead of `#<list 2 4 6>` – the list is in reverse order!

This is a natural consequence when a list is generated iteratively: In a recursive call where the result list grows, a new element is appended to the *front* using `cons`, so the first element comes last.

This means that the invariant is correct, but imprecise: it says which elements in `The entries` are included, but not in what order. We should expand them:

```
; evens contains the even numbers between list0 and list; in reverse order
```

(You might even want to rename `evens` to `inverted-evens`.)

We still need to correct the function, of course. Fortunately, this is easy: we simply call `'invert'` on the result at the end, and the result will be correct. We don't need to worry about the function becoming quadratic again because we only call `'invert'` at the end, not with every recursive call.

Task 10.5

Write an iterative version of `concatenate` from section 6.6.3 on page 190. Compare it with the `invert-helper` function on page 294: are there any similarities? ỳ

10.7 Fold lists iteratively

You might recall that in section 9.4 on page 265, we wrote the mother of all list functions, `list-fold`, by turning the list function template into a function. All list functions programmed according to this template can be replaced by a simple call to `list-fold`.

For functions with intermediate results, there's a different template, and in this section, we'll do the same for this template as `list-fold` did for the previous template. This time, we'll derive the function directly from the template and give it a name right away: `list-fold-left`.

Why `list-fold-left`? If you look at how `list-fold` works in the Stepper, you'll see that it unrolls the list from right to left. However, a function with an intermediate result works from left to right, hence the `-left`.

Here is the template for the desired function from construction instructions 21 on page 21. te 299:

```
(define list-fold-left
  (lambda (list0 ...)
    (define accumulate
      (lambda (list acc)
        (cond
          ((empty? list) ... acc ...) ((cons? list)

            (accumulate (rest list)
                        ... (first list) ... acc))))))

    (accumulate list0 ...)))
```

Wherever there are ellipses, we need to write names: that's a total of three places. We'll work from back to front, starting with the first call to `accumulate`.

The ellipsis there represents the first value of the accumulator `acc` – we choose the name `start-acc` for it and include it in the parameters of `list-fold-left`.

When calling `accumulate` recursively, `(first list)` and `acc` must be used for the next list.

The values for the accumulator are combined. We'll have a function handle this, similar to the `for cons` loop in a `list-fold` loop. Finally, there's the ellipsis at the end of the `empty?` branch, where the last accumulator becomes the final result. We'll also have a function handle this transition, which we'll call `final-acc->result`.

```
(define list-fold-left
  (lambda (list0 start-acc next-acc final-acc->result)
    (define accumulate
      (lambda (list acc)
        (cond
          ((empty? list) (final-acc->result acc))
          ((cons? list)
           (accumulate (rest list)
                        (next-acc (first list) acc))))))
    (accumulate list0 start-acc)))
```

This allows us to perform the tasks of the previous iterative functions. We'll do this as test cases. Here are `invert` and `list-sum`:

```
(check-expect (list-fold-left (list 1 2 3) empty
                               (lambda (list acc)
                                 (cons
                                  (lambda (inverted) inverted))
                                  (list 3 2 1))))

(check-expect (list-fold-left (list 1 2 3 4 5)
                               0 +
                               (lambda (sum) sum))
              15)
```

Task 10.6

Create `evens` using a call to `list-fold-left` !

ŷ

A signature for `list-fold-left` is still missing . We write down what we know: that `list-fold-left` has four digits and that `list0` is a list.

```
(: list-fold-left ((list-of %a) ... .. ..) -> ...))
```

The first test case shows that the accumulator does not need to have the same signature as the list elements. Therefore, we use a new signature variable `%acc` for the accumulator:

```
(: list-fold-left ((list-of %a) %acc ... ..) -> ...))
```

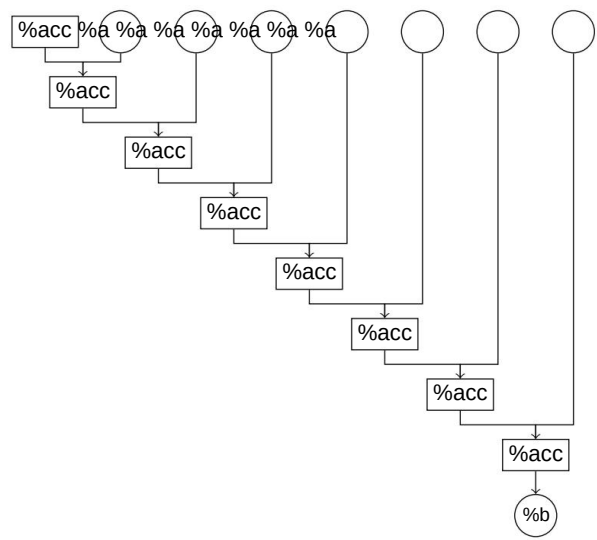


Figure 10.3: List-fold-left operation

We also know that `next-acc` is a two-digit value and `final-acc->result` is a single-digit value. Function is:

```
(: list-fold-left
  ((list-of %a) %acc (... ... -> ...) (... -> ...) -> ...))
```

The arguments of `next-acc` are a list item and the previous accumulator; the output is the next value for the accumulator. We can therefore complete it as follows:

```
(: list-fold-left
  ((list-of %a) %acc (%a %acc -> %acc) (... -> ...) -> ...))
```

The function `final-acc->result` remains , accepting the last accumulator as its argument. The result becomes the final result of the function, for which we choose the signature variable `%b` :

```
(: list-fold-left ((list-of
  %a) %acc (%a %acc -> %acc) (%acc -> %b) -> %b))
```

Figure 10.3 illustrates the relationship between the `list-fold-left` process and the signature. The processing clearly proceeds from left to right.

Figure 10.4, in contrast, shows how `list-fold` works, from right to left. Works on the left. As a reminder, here is the signature again:

```
(: list-fold (%a (%b %a -> %a) (list-of %b) -> %a))
```

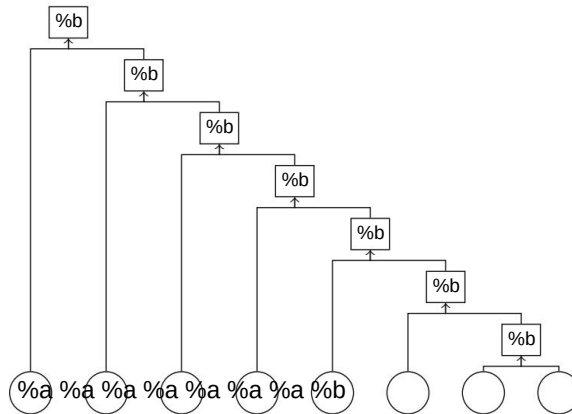


Figure 10.4: List-fold process

NOTES

When a computer evaluates programs, memory is required to manage contexts: For recursive functions without an accumulator, this memory space grows with the size of the arguments. Accordingly, in principle, no memory is needed...

when no context is involved. In teaching languages, no memory is actually used for end-recursive calls. Unfortunately, many other programming languages are not so economical and They waste memory even for tail-recursive calls. More on this in Chapter 15.

TASKS

Task 10.7

Write a function ``list-sum+product`` that accepts a list of numbers and returns a two-element list whose first element is the sum of the list elements and whose second element is their product. Write two versions of the function: a "normal recursive" one without Accumulator and an iterative with two accumulators.

Task 10.8

Write a version of `power2` from section 7.4 on page 214 using an accumulator. Note: You can use the same invariant as with `power`.

Task 10.9

Identify the contexts of the calls to the functions named `p` in the following expressions:

```
(+ (p (- n 1)) 1) (p (-
n 1) acc) (* (p
(rest lis)) b) (+ (* 2 (p (-
n 1))) 1) (p (- n 1) (* acc n))
(f (pn)) (+ (f (pn)) 5) (p (f
(- n 1)) (*
n (hn))) (+ (f (pn))
(hn))
```

Which calls are *tail calls*?

Task 10.10

Write an iterative version of `list-length`.

Task 10.11

Write an iterative version of `list-min-nonempty` in section 6.8 on page 197.

Task 10.12

Write `list-fold-left` using `list-fold` by flipping the input list.

Write the reverse `list-fold` using `list-fold-left`!

ŷ

Task 10.13

Newton's method is used to approximate zeros. For a given starting value, the iteration approaches the target value.

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

getting ever closer to a zero.

Program the Newton-Raphson method!

Note: The solution is complete when the function value is close to 0, i.e., less than a certain tolerance. Since Newton's method does not always yield a solution, program your own. The function is designed to automatically stop after a certain number of steps.

11 ways to program video games

In this chapter, we will program a complete, usable program for the first time: a small video game. For this, we need two add-ons for the teaching language in DrRacket: the Teachpacks `image.rkt` and `universe.rkt`. The former is responsible for displaying the graphics (we already used it in the first chapter), and `universe.rkt` is for moving the graphics and making them interactive.

Both Teachpacks offer much more than can be covered in this chapter. Fortunately, DrRacket's Help Center provides extensive documentation, though currently only in English: For the `image.rkt` Teachpack, search for `2htdp/image`, and for `universe.rkt`, search for `2htdp/universe`. [Here](#) are direct links to the documentation:

<https://docs.racket-lang.org/teachpack/2htdpimage.html> <https://docs.racket-lang.org/teachpack/2htdpuniverse.html>

11.1 IMAGES WITH IMAGE.RKT

For graphics programming with DrRacket, we load a so-called *teachpack*, a small language add-on, in this case with a set of functions for generating images. We already encountered it in section 1.4.2 on page 23 : To load the teachpack, you must select "Add teachpack" in the Language menu (or Language in the English version), and then select the file `image.rkt` in the selection dialog that appears.

11.1.1 SIMPLE PICTURES

In the Teachpack `image.rkt`, various functions generate simple images. For example, the `rectangle` function has the following signature:

`(: rectangle (real real mode color -> image))`

The first two arguments are the width and height of a rectangle in pixels. The argument with the signature `mode` is a string that must be either `"solid"` or `"outline"`.

It determines whether the rectangle is drawn as a solid block or only as an outline.

¹ 2htdp refers to the title of the book *How to Design Programs*, second edition [FFFK14], for which these Teachpacks were originally developed.

The argument with the signature `color` is a string that specifies a color, for example, "red", "blue", "yellow", "black", "white", or "gray". The result of `rectangle` is an image that is displayed by the DrRacket REPL in the same way as other values. Example:

```
(rectangle 100 30 "outline" "brown")
```



The color "transparent" also works, making the image see-through or invisible. (You might ask what an invisible image is good for. For example, if we don't want to place a smaller image perfectly in the center – then we can place it on top of a transparent image.)

There are other functions that draw geometric figures:

```
(: circle (real mode color -> image))
```

The `circle` function returns a circle, where the first argument specifies the radius. The `mode` and `color` arguments are the same as for the `rectangle` function. Example:

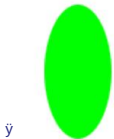
```
(circle 50 "solid" "red")
```



```
(: ellipse (real real mode color -> image))
```

This function returns an ellipse, where the first argument specifies the width and the second the height. Example:

```
(ellipse 50 100 "solid" "green")
```



```
(: triangle (real mode color -> image))
```

This function returns an upward-pointing equilateral triangle, where the first argument specifies the side length. Example:

```
(triangle 50 "solid" "gold")
```

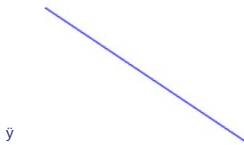


```
(: line (real real color -> image))
```

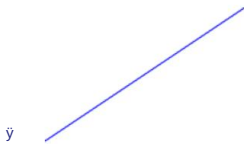
Draws a line. The command `(line whc)` returns an image with width `w` and height `h`, in which the line runs from the upper left corner to the lower right corner. If the line should run along the other diagonal, you can simply reverse the sign of `w` or `h`.

Examples:

```
(line 150 100 "blue")
```



```
(line -150 100 "blue")
```



We can also create an image containing text, using the ``text`` function, which has the following signature:

```
(: text (string real color -> image))
```

The number represents the height of the letters. Example:

```
(text "Write your program!" 20 "red")
```



Sometimes simple rectangles and circles aren't enough, and we want to create more complex shapes. One possibility is the ``polygon`` function, which draws an `n`-gon. It has the following signature:

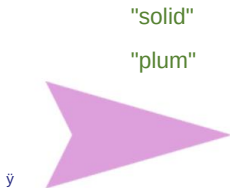
```
(: polygon ((list-of (mixed posn pulled-point)) mode color -> image))
```

This function accepts a list of vertices and the usual `mode` and `color` arguments. A vertex must belong to either the signature ``posn`` or ``pulled-point``. These are built-in record types where ``posn`` might be defined as follows:

```
(define-record posn
  make-posn
  posn?
  (posn-x real)
  (posn-y real))
```

This type defines standard Cartesian coordinates with X and Y components. Here is an example of a polygon with such vertices:

```
(polygon (list (make-posn 0 0) (make-
               posn -20 40) (make-
               posn 120 0) (make-
               posn -20 -40))
```



With `posn` vertices, the edges are all straight lines. [Pulled-point vertices](#) allow you to make the edges curves. [Pulled-point](#) is also a record type.

```
(define-record pulled-point make-
  pulled-point pulled-
  point? (pulled-
  point-lpull real) (pulled-point-langle
  real) (pulled-point-x real) (pulled-
  point-y real) (pulled-point-
  rpull real) (pulled-point-
  rangle real))
```

Like `posn`, `pulled-point` also has an X and a Y coordinate. In addition, there are two "Pull" and two "Angle" components that specify how the edges are bent. Figure 11.1 shows how this works: For each vertex, there is an edge from the previous point – the *incoming* edge – and the edge to the next point – the *outgoing* edge.

The [pulled-point-langle component](#) specifies the angle at which the incoming edge is bent, the [pulled-point-rangle component](#) specifies the angle for the outgoing edge.

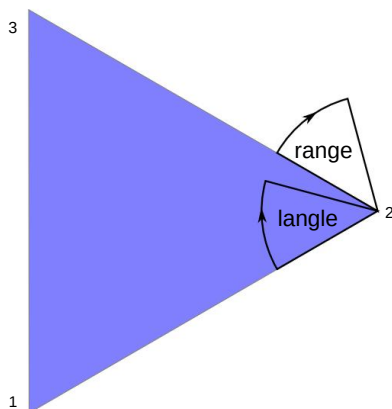


Figure 11.1: How `pulled-point` works

Edge responsible. Figure 11.1 shows in which direction `langle` and `range` the two edges bend from point number 2: The incoming edge comes from point 1, the outgoing edge goes to point 3. The components `pulled-point-lpull` and `pulled-point-rpull` indicate how much the edges are bent on a scale between 0 and 1.

```
(polygon (list (make-pulled-point 1/2 0 0 0 1/2 -20)
               (make-posn -20 40)
               (make-pulled-point 1/2 -20 120 0 1/2 20) (make-posn
               -20 -40))
```

"solid"

"plum"



It is also possible to convert image files into `image.rkt` images. This is done using the "Insert Image" menu item in the special menu. Alternatively, images can be selected and copied from other applications and then pasted into DrRacket. The pasted images then serve as literals for image values. Figure 11.2 shows an example.

Finally, the following functions determine the width and height of an image:

```
(: image-width (image -> natural)) (: image-
height (image -> natural))
```

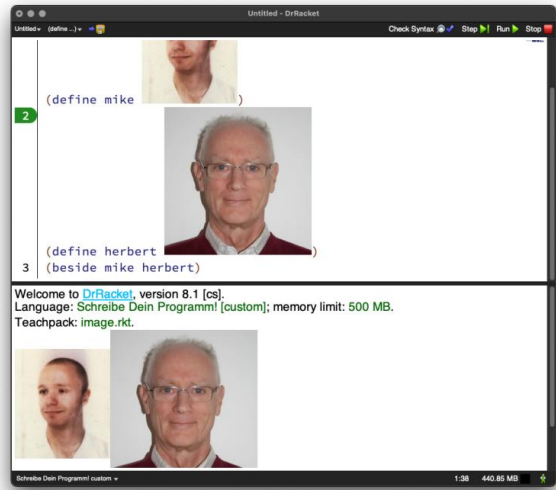


Figure 11.2: Embedding an image file in a program

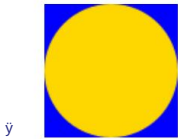
11.1.2 COMBINATION AND MODIFICATION OF IMAGES

Since these geometric shapes are boring on their own, we can combine several images to create a visual representation. to assemble one.

For layering images, there is the `overlay` function, which places one image centered on another image. It's placed on top. The result is large enough to fit both images. Signature and example:

`(: overlay (image image -> image))`

`(overlay
 (circle 50 "solid" "gold") (rectangle
 100 100 "solid" "blue"))`



The ``overlay/xy`` function allows you to shift the lower image relative to the upper one in the X and Y directions. Signature and example:

`(: overlay/xy (image real real image -> image))`

```
(overlay/xy
  (circle 50 "solid" "gold")
  10-20
  (rectangle 100 100 "solid" "blue"))
```



The example shows that `overlay/xy` also makes the result image just large enough to fit the two input images.

Sometimes, even `overlay/xy` doesn't do the right thing, namely when you simply want to place an image into a "scene," for example, an armadillo on a section of road. In this case, just as with `overlay/xy`, we want to control the coordinates at which the superimposed image appears, but the image shouldn't grow larger if, for example, part of the armadillo extends beyond the road section. This task is handled by the `place-image` function, whose signature is the same as that of `overlay/xy`:

```
(: place-image (image real real image -> image))
```

Unlike with `overlay/xy`, the two coordinates are not interpreted relative to the centered position. Instead, the two coordinates specify the position in the second image where the center of the first image is placed, starting from the upper left corner of the first image:

```
(place-image
  (circle 50 "solid" "gold")
  40 70
  (rectangle 100 100 "solid" "blue"))
```



The `place-image` function has a more advanced counterpart, `place-image/align`. While `place-image` sets the reference point in the center of the first image, `place-image/align` also allows for other reference points. Here's its signature:

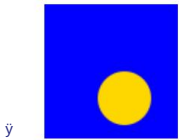
```
(: place-image/align (image real real x-place y-place image -> image))
```

The signatures `x-place` and `y-place` could be defined as follows:

```
(define x-place (signature (enum "left" "right" "center")))
(define y-place (signature
  (enum "top" "bottom" "center")))
```

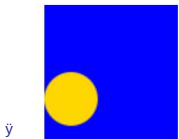
These arguments indicate whether the reference point is horizontally at the left or right edge or in the middle, and vertically at the top, bottom or in the middle:

```
(place-image/align (circle
  20 "solid" "gold")
  40 70 "left" "center"
  (rectangle 100 100 "solid" "blue"))
```



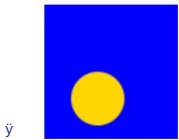
$\bar{y}$

```
(place-image/align (circle
  20 "solid" "gold") 40 70 "right"
  "center" (rectangle 100 100
  "solid" "blue"))
```



$\bar{y}$

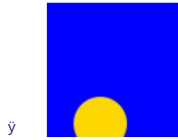
```
(place-image/align (circle 20
  "solid" "gold") 40 70 "center" "center"
  (rectangle 100 100 "solid" "blue"))
```



$\bar{y}$

```
(place-image/align (circle
  20 "solid" "gold") 40 70 "center"
  "top")
```

```
(rectangle 100 100 "solid" "blue"))
```



```
(place-image/align (circle
  20 "solid" "gold")
  40 70 "center" "bottom")
```

```
(rectangle 100 100 "solid" "blue"))
```



You might remember the following functions, `above` and `beside`, from the first chapter on page 25.

They placed two images on top of each other and next to each other, respectively:

```
(: above (image image -> image)) (: beside
(image image -> image))
```

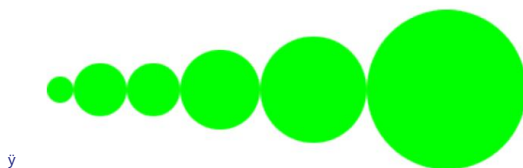
Sometimes an image looks good but is too small or too large. The `scale` function can make it larger or smaller. Signature:

```
(: scale (real image -> image))
```

Example:

```
(define c (circle 20 "solid" "green")) (beside (scale 0.5
c)
```

```
  c
  (scale 1 c)
  (scale 1.5 c)
  (scale 2 c)
  (scale 3 c))
```



11.2 MODELS AND VIEWS

video games/dillo-world.rkt

code

Our video game is about armadillos on the Texas highway. You might remember, we already encountered them in section 3.4 on page 102 :

An armadillo has the following characteristics:

; - weight (in g) ; - alive

or dead (define-record dillo)

```
make-dillo
```

```
dillo?
```

```
(dillo-weight natural) (dillo-
alive? boolean))
```

```
(: make-dillo (natural boolean -> dillo))
```

```
(: dillo-weight (dillo -> natural)) (: dillo-alive? (dillo
-> boolean))
```

```
(define dillo1 (make-dillo 55000 #t)) ; 55 kg, alive (define dillo2 (make-dillo 58000
#f)) ; 58 kg, dead (define dillo3 (make-dillo 60000 #t)) ; 60 kg, alive
(define dillo4 (make-dillo 63000 #f)) ; 63 kg, dead
```

```
; Armadillo run over (: run-over-
dillo (dillo -> dillo))
```

```
(define run-over-dillo
```

```
(lambda (dillo)
```

```
(make-dillo (dillo-weight dillo)
#f)))
```

While the data definition and its associated code capture some properties of armadillos, a graphical representation is lacking for a video game. In software architecture, such a representation with its associated operations is also called *a model* or *domain logic*, and in larger software systems, it's a good idea to separate the model from the graphical representation so that the model remains stable even if the graphical representation changes.

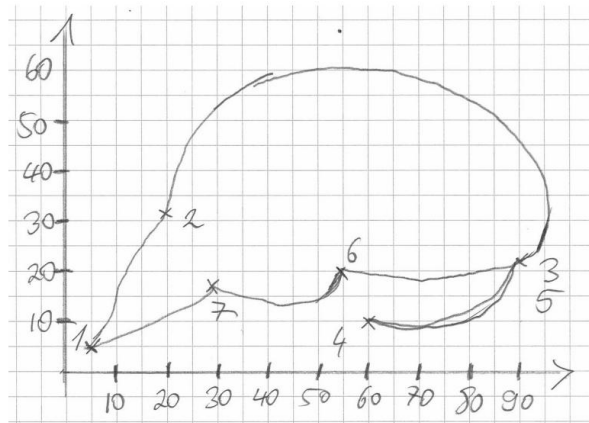


Figure 11.3: Outline of an armadillo

The graphical representation of a model is called its *view*. So, if we are working with armadillos as a model for a video game, we need to calculate a view of it in the form of an image.

We're warning you in advance: The graphic design in this chapter lacks finesse. We hope you can improve them!

The following function turns a dillo value into an image:

```
; Create armadillo image (: dillo-image
(dillo -> image))
```

As a basis for this, we first create an image of the armadillo's body. For this purpose, we initially drew a hand-drawn outline of the armadillo in a coordinate system (see Figure 11.3) and marked the points where the direction of the line abruptly changes – essentially the corners. We numbered these corners and used them to create this polygon:

```
(define dillo-body
  (overlay/xy (polygon (list (make-
    pulled-point 0.3 30
    5 (- 60 5)
    0.2 5)
    (make-pulled-point 0.4 20 20 (- 60 32)
    0.5 -70)
```

```
(make-pulled-point 0.5 120 90 (- 60
                                22) 0.2 -30)
```

```
(make-pulled-point 0.5 90 60 (- 60
                                10) 0.5 90)
```

```
(make-pulled-point 0 0 90 (- 60
                              22) 0.2 -30)
```

```
(make-pulled-point 0 0 55 (- 60
                              20) 0.3 -30)
```

```
(make-pulled-point 0 0 29 (- 60
                              17) 0.5 20))
```

```
"solid"
"brown")
0 -30
(rectangle 100 60 "solid"
            "transparent"))
```

As you can see, we experimented quite a bit until it looked halfway decent, especially with the numbers for the **pulled points**. We did transfer the coordinates directly from the drawing, but we noticed that in mathematics, coordinates run from bottom to top, while in the Teachpack they run from top to bottom. That's why, for example, **(-60 5)** represents the Y-coordinate 5, because the top edge of the drawing is at 60.

Furthermore, the armadillo is curved outwards, especially upwards and to the right. DrRacket cuts off the polygon during display, which is why we overlaid it onto a transparent background using **overlay/xy**. Here's the result:

dillo-body



Now, we don't want to depict every armadillo exactly the same; rather, we want to take its individual characteristics into account when rendering it. For this, we'll write the **`dillo-image` function**, whose short description and signature we've already seen. Here's the framework with the...

Template for composite data as input:

```
(define dillo-image
  (lambda (dillo)
    ...
    (dillo-weight dillo)
    (dillo-alive? dillo)
    ...))
```

We should also consider how much the armadillo weighs and whether it is still alive. We take the weight into account by making the armadillo larger or smaller:

```
(define dillo-image (lambda
  (dillo) (scale (+ 1

                  (/ (- (dillo-weight dillo) 50000) 15000)) dillo-body)

  ...
  (dillo-alive? dillo) ...))
```

We determined the factor for `scale` through trial and error. But `(dillo-alive? dillo)` is still there – we're incorporating that by putting "dead eyes" on a dead armadillo's face:

```
(define dillo-image
  (lambda (dillo)
    (scale (+ 1
              (/ (- (dillo-weight dillo) 50000)
                  15000))
      (if (dillo-alive? dillo)
          dillo body
          (overlay/xy dead eyes
                      -25 -25
                      dillo-body))))))

(define dead-eyes
  (overlay (line 10 10 "green") (line -10 10
                                       "green")))
```

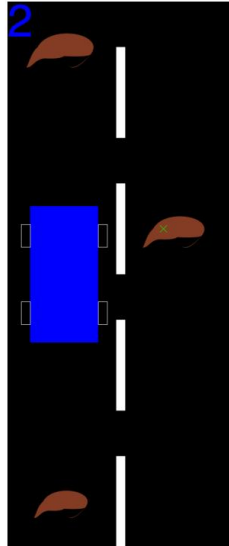


Figure 11.4: Armadillos on the road

11.3 MODELING THE HIGHWAY

One armadillo alone doesn't make a video game: We put several armadillos on the road and – of course – then run them over. It looks like Figure 11.4. The car moves along the road and can switch to either the left or right side, where it then runs over any armadillos it encounters. The score in the upper left corner shows how many armadillos have already been converted from the living to the dead.

In order to represent all of this, we need to write data definitions for everything that is based on the image of the street shows:

- the position of the car
- the positions of the armadillos along with their states
- the score

We are developing data definitions for these aspects of the game. It's already clear that "positions" play an important role, both for the car and the armadillos. The image shows that this includes which side of the road something is on and at which "meter mark" of the road. Here are the data definitions for the side of the road and the position:

```
; street side
```

```
(define side
```

```
  (signature (enum "left" "right")))
```

A position on the road consists of:

; - Road meters (distance from the start of the road in meters)

; - Side

(define-record position make-

position position?

(position-m-

from-start real) (position-side side))

Next, we need to develop the idea of "positions of armadillos along with their conditions" into

Convert a data definition. We'll do this first for a single armadillo and call it...

the concept of "armadillo on the street":

; An armadillo on the road has the following characteristics:

; - Armadillo state

; - Position on the road

(define-record dillo-on-road

make-dillo-on-road

dillo-on-road?

(dillo-on-road-state dillo)

(dillo-on-road position position))

11.4 STREET VIEWS

Now that we've modeled the world, we can bring it to the screen. We'll start with the simple and obvious elements: the road, including the markings on the center line and the car. We already have the armadillos.

To be able to think somewhat realistically about the proportions of everything, we measure everything in meters and convert back and forth between meters and pixels. On un-

A 1m:100 pixel ratio looks good on this screen. These functions convert back and forth:

Convert meters to pixels

(: meters->pixels (real -> real))

```
(define meters->pixels (lambda
  (meters)
  (* meters 100)))
```

Convert pixels to meters

```
(: pixels->meters (real -> real))
```

```
(define pixels->meters (lambda
  (pixels)
  (/ pixels 100)))
```

The road itself is just a black rectangle, that's easy. The road markings are more difficult, meaning alternating white and black stripes. To make this look realistic, we first define the length of the markings and the gaps:

```
(define marking-height 2) ; Height of the stripes
(define gap-height 1) ; Height of the gaps
```

We need to alternate gluing the markings and the blanks together. How many times do you ask? Well, the Texas highway is infinitely long, that would be difficult. But we're only showing a section, so we only make as many stripes as fit into that section. How many there are depends on the height of the section. We could, of course, count them by hand, but that would mean we'd have to decide on the height of the section early on. To avoid that, we first write a function that takes the number of stripes as input. We follow the template for functions on natural numbers:

```
; Paint road markings with a specific number of stripes (: markings (natural -> image))
```

```
(define markings
  (lambda (n)
    (cond
      ((zero? n) empty-image)
      ((positive? n) (above
        (rectangle (meters->pixels .20) (meters->pixels marking-
          height)
          "solid"
          "white"))
```

```

(rectangle (meters->pixels .20) (meters->pixels
          gap-height)
  "solid"
  "black")
(markings (- n 1))))))

```

Here is the actual elevation of the road section:

```

(define road-window-height 12) ; Height of the road cutout

```

The idea is that later you only need to change this one definition if the cutout height doesn't suit you, and then everything else will adjust automatically.

To calculate the number of markings needed, we divide the height of the road cut by the length of one marking plus the gap. To be on the safe side, we add a little extra: the division might not be exact. Also, it's possible that only part of a strip will be visible in the end, so it's good to have one extra in case of doubt.

```

; Number of required markings (define marking-
count (+ 1)

```

```

(quotient road-window-height
  (+ marking height gap height)))

```

From the number we finally create the image of the stripes:

```

; visible markings (define visible-
markings
  (markings marking-count)

```

As a reminder: We introduced the `quotient` function on page 98 ; it divides by integers.

Now we can piece together the image of the street scene. Besides the

Height and width are also added, and first create an empty scene consisting only of black asphalt:

```

; in meters
(define road-width 5)

```

```

(define blank-road-window
  (empty-scene (meters->pixels road-width) (meters->pixels
    road-window-height) "black"))

```

How median strips are placed on the road section depends on the road meter position of the image. Therefore, we write a function that accepts the road meter position and returns a suitable image:

```
; Show street view (: road-window (real
-> image))
```

We use `place-image/align` (see page 323) to align the center lines with the asphalt.
set:

```
(define road-window
  (lambda (meters)
    (place-image/align visible-markings
      ...
      ...
      ... ...
      blank-road-window)))
```

We first place the reference point at the top of the median strip in the middle:

```
(define road-window
  (lambda (meters)
    (place-image/align visible-markings
      ...
      ...
      "center" "top" blank-
      road-window)))
```

In the X direction, the median strip needs to be in the middle; we can simply halve the width of the road for that.

The Y-coordinate is more complicated: it depends on the road length. However, the road length (except at the very beginning) is greater than the height of the section or the length of the median strip image. The median strip image repeats itself every time a stripe plus gap passes. Therefore, we calculate the number of pixels for this:

```
(define marking-segment-pixels
  (meters->pixels (+ marking-height gap-height)))
```

We divide by this number and take the remainder to get the Y-position. That's not quite right – the remainder is always positive, and therefore the center line is too low.

We'll stretch the image upwards by one strip by subtracting the marking segment pixels . This is the result:

```
(define road-window (lambda
  (meters)
    (place-image/align visible-markings
      (/ (image-width blank-road-window) 2) (- (remainder
        (meters->pixels meters) marking-segment-pixels) marking-
        segment-pixels) "center" "top" blank-
        road-window)))
```

Task 11.1

Try out the ``road-window`` property . Remove the subtraction of ``marking-segment-pixels`` and examine the effect!

Ÿ

11.5 Moving Images with Universe.RKT

So far, we only have static, boring images. It's time to bring them to life. For this, we need another Teachpack called `universe.rkt`. To activate it, you need to go back to the Language menu, click on Add Teachpack, select `universe.rkt` in the dialog box that appears, and finally click OK.

The `universe.rkt` Teachpack includes a feature called `animate`. What it does is explained below. The simplest way to illustrate this is with an example that you can type into the REPL:

```
(animate
  (lambda (ticks)
    (place-image/align (circle 5 "solid" "red") 100 ticks "center" "top"
      (square 200 "solid" "black")))))
```

Task 11.2

Try it out!

Ÿ

You see a small red circle moving from top to bottom in a square image.

The new function ``animate`` accepts a function with an input named ``ticks`` that produces an image:

```
(: animate ((natural -> image) -> natural))
```

`Animate` calls the function 28 times per second and displays the resulting images sequentially as an animation. `Animate` passes a natural number as "`ticks`," starting at zero and increasing by one each time: it measures time in so-called *ticks*.

We'll now use ``animate`` to animate the image of the street scene, ensuring the car is centered. First, we'll determine the car's speed. We've chosen a fairly leisurely pace to make sure we capture all the armadillos.

```
; Meters per tick (define
meters-per-tick 0.1)
```

This function is so simple that, exceptionally, separate testing is not worthwhile:

```
Convert ticks to meters
(: ticks->meters (natural -> rational))
```

```
(define ticks->meters
  (lambda (ticks)
    (* meters-per-tick ticks)))
```

This allows us to write a function that converts ticks into road meters and calls `road-window` :

```
; Show road section at time (: road-window-at-ticks (natural ->
image))
```

```
(define road-window-at-ticks
  (lambda (ticks) (road-
    window (ticks->meters ticks))))
```

So try the following:

```
(animate road window at ticks)
```

Task 11.3

Write a different function of your choice with a signature suitable for `animate` and try it out! ỹ

11.6 CARS AND ARMADILLOS ON THE HIGHWAY

The road is finished, next up is the car. We'll start with a wheel:

```
; wheel
(define wheel
  (rectangle (meters->pixels 0.2) (meters->pixels .5) "outline" "white"))
```

Next, we'll combine two wheels to form an image, which will then be placed to the left and right of the Bodywork will be placed:

```
Two wheels on one side of the car
(define wheels-on-one-side
  (above wheel
    (rectangle 0 (meters->pixels 1.2) "solid" "black") wheel))
```

```
; width of the car (define
car-width 1.5)
; Length of the car (define
car-length 3.0)
```

```
; Image of the car
(define car
  (beside
    wheels-on-one-side
    (rectangle (meters->pixels car-width) (meters->pixels car-length)
      "solid" "blue") wheels-
    on-one-side))
```

Finally, we need to place the car and the armadillos on the road. The function for this clearly states that it should produce an image. The inputs are already defined: • the image to be placed on the road • the position of the image.

We also need:

- the time, because it determines which section of the street is displayed • the image of the street

The latter might surprise you. Isn't the image of the road always the same? No, it isn't! First of all, the position of the median strip always varies depending on the time. Furthermore, we don't just want to place one image on the road, but many. To do this, we feed the result of the function back into the function to place another image.

The short description, signature, and framework look like this:

```
; Place image on road (: place-image-on-road
(image natural image position -> image))

(define place-image-on-road
  (lambda (road-image ticks image position)
    ...))
```

Since `position` involves composite data, the template contains calls to the selectors:

```
(define place-image-on-road
  (lambda (road-image ticks image position)
    ...
    (position-m-from-start position) (position-side
    position) ...))
```

We now need to calculate the pixel coordinates of the image relative to the street section. The X-coordinate is relatively simple. We create a variable ``pixels-from-left``, which is derived from ``position-side position``, as already defined in the template. The side is relative to the median strip, for whose X-coordinate we also create an intermediate variable:

```
; X-coordinate of the center of the road, in pixels
(define middle-pixels (/ (image-width road-image) 2))
```

This results in `pixels-from-left` as follows:

```
; X-coordinate of the center of the image (define pixels-from-
left (cond

  ((string=? (position-side position) "left") (* middle-pixels 0.5)) ;
    Center of left lane ((string=? (position-side position) "right") (* middle-
    pixels 1.5)))) ; Center of right lane
```

Next, we need the Y-coordinate, which is a bit more complicated: We need the Y-coordinate of `image`` relative to the bottom edge of the image. (Or the top edge, that works too.)

That would be the following expression:

```
(- (position-m-from-start position)
   (ticks->meters ticks))
```

This coordinate is still in meters, which we need to convert to pixels. We also need to consider that pixel coordinates run from top to bottom, while road meters run from bottom to top – so we subtract this number from the image height. With this, we can

Use `place-image/align`` again (see page 323) to position the image on the street. The complete function looks like this:

```
(define place-image-on-road
  (lambda (road-image ticks image position)
    ; X-coordinate of the middle of the road, in pixels (define middle-pixels
    (/ (image-width road-image) 2))

    ; X-coordinate of the center of the image (define pixels-from-
    left (cond

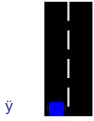
      ((string=? (position-side position) "left") (* middle-pixels 0.5)) ;
      Center of the left lane
      ((string=? (position-side position) "right") (* middle-pixels 1.5)))) ;
      Center of the right lane

    (place-image/align image
      pixels-
      from-left (- (image-height
                    road-image) (meters->pixels (- (position-m-
                                                      from-start position) (ticks->meters ticks))))

      "center" "center"
      road-image)))
```

The function is complex enough that we should test it. Constructing a `check-expect` test beforehand is virtually impossible, but we can at least try out the function in the REPL, most easily at 0 ticks, i.e., at the beginning of the game:

```
(place-image-on-road (road-window 0) 0 car (make-position 0 "left"))
```



There's still a problem! The car is at the bottom and only half visible, but we want it in the middle. So we need to subtract half the height of the road segment, and adjust the `place-image/align` call accordingly :

```
(place-image/align
  image
  pixels-from-left (- (image-
    height road-image)
    (meters->pixels (+ (- (position-m-from-start position) (ticks->meters ticks)) (/ road-
      window-height 2))))
  "center" "center"
  road-image)
```

Now everything is at 0 in the middle!

We can now use this function to first place the car image on the road segment:

```
; put car on the road
(: place-car-on-road (natural position image -> image))

(define place-car-on-road (lambda
  (ticks car-position road-image) (place-image-on-road road-
    image
    ticks
    car car-position)))
```

Task 11.4

Based on `place-car-on-road`, construct a function that you can pass to `animate` and try it out!

ŷ

We'll do something similar for the armadillo picture. The necessary information about its position and condition. We extract the armadillo's information from the `dillo-on-road record`:

```
; Draw an animal on the road (: place-dillo-on-
road (natural dillo-on-road image -> image))
```

```
(define place-dillo-on-road
  (lambda (ticks dillo-on-road road-image) (place-image-on-
    road road-image
      ticks
      (dillo-image
        (dillo-on-road-state dillo-on-road))
        (dillo-on-road-position dillo-on-road))))
```

Now, there isn't just one armadillo on the street, but many, all of which are listed in the world record. For this, we need a function with the following short description and signature:

```
Paint all the animals on the street
(: place-dillos-on-road
  (natural (list-of dillo-on-road) image -> image))
```

We could program this according to the list construction guidelines. However, it's simpler to use the universal `fold` function. (See section 9.4 on page 265.) It looks like this:

```
(define place-dillos-on-road
  (lambda (ticks dillos-on-road road-image) (fold road-image
    (lambda (dillo-on-road image) (place-dillo-
      on-road ticks dillo-on-road image))
    dillos-on-road)))
```

Besides the car and the armadillos, the score in the top left corner is still missing:

```
; Display score (: place-score
(natural image -> image))

(define place-score
  (lambda (score image) (place-
    image/align (text (number-
      >string score) 100 "blue") 0 0 "left" "top" image)))
```

11.7 THE WORLD IS A HIGHWAY

It's time we put everything together to create a game that moves and reacts at the push of a button. The universe.rkt teachpack requires us to create a data definition that contains the entire state of the game. In section 11.3 on page 330, we already listed the following aspects: • the position of the car

- the positions of the armadillos along with their states
- the score

We know that the universe.rkt teachpack will later provide us with the time in ticks: from this, we can calculate the distance in meters to the author's position. We just need to find the page.

represent explicitly. The resulting data definition is therefore slightly modified as follows:

The game world consists of:

- ; - Ticks since the start of the game
- ; - side on which the car drives
- ; - Animals on the street
- ; - Score

```
(define-record world
  make-world
  world?
  (world-ticks natural) (world-car-
    side side)
  (world-dillos-on-road (list-of dillo-on-road)) (world-score natural))
```

Here's an example:

```
; Four armadillos on the road (define dillos-on-road
(list (make-dillo-on-road dillo1
  (make-position 20 "left"))
  (make-dillo-on-road dillo2 (make-position 26 "right")) (make-dillo-on-road dillo3
  (make-position 30 "left")) (make-dillo-on-road dillo4 (make-position 42 "left"))))

; World at the beginning, car is on the left (define
initial-world
  (make-world 0 "left" dillos-on-road 0))
```

For ``place-car-on-road``, we will need the car's position as a `position record`, which is not yet directly available in the world record. The page is already there, though.

We calculate road meters from the ticks. The function for this has a short description, signature, and test cases as follows:

```
; Position of the car (: world-
car-position (world -> position))
```

```
(check-expect (world-car-position (make-world 0 "left" empty 0)) (make-position 0 "left")) (check-
expect (world-car-position (make-
world 100 "right" empty 0))
(make-position (ticks->meters 100) "right"))
```

The template for the function takes into account that both the input and output are composite data:

```
(define world-car-position (lambda
(world) (make-
position ... ...)
...
(world-ticks world) (world-car-
side world)
(world-dillos-on-road world)
(world-score world) ...))
```

Only the first two fields of the `world` are relevant for the car's position – armadillos and Scores are clearly irrelevant. For the position, we need the road distance, which we can determine from... You can calculate the ticks using `ticks->meters` :

```
(define world-car-position (lambda
(world) (make-position
(ticks->meters (world-ticks world)))
(world-car-side world))))
```

Now we finally have all the functions we need to display the entire world. We just need to combine them to display everything in a world record as a street segment:

```
; Show game (: world-
>image (world -> image))
```

```
(define world->image
  (lambda (world)
    (define ticks (world-ticks world))
    (place score
      (world score world)
      (place-car-on-road
        ticks
        (world-car-position world) (place-
          dillos-on-road
          ticks
          (world-dillos-on-road world)
          (road-window-at-ticks ticks))))))
```

You can of course try `(world->image initial-world)` , but that will only show the initial, empty section of the road.

It gets better with ``animate`` . For this, we need a function with a signature `(`natural -> image`)` . Since we already have ``world -> image`` with the signature `(`world -> image`)` , we also need a function with a signature `(`natural -> world`)` that takes a suitable ``world`` value.

generated:

```
; To represent a static world at a specific point in time
(: world-at-ticks (natural -> world))
```

```
(define world-at-ticks
  (lambda (ticks)
    (make-world ticks "left" dillos-on-road 0)))
```

Now we can call `animate` with a combination of the two:

```
(animate (lambda (ticks) (world->image (world-at-ticks ticks))))
```

You see the road passing by, and eventually armadillos appear. But the car is stuck to the left, and the armadillos don't die when the car drives over them.

11.8 RUN OVER SPECIFICALLY

What is still missing so that all aspects of the game can be calculated from the world record, and thus the Texas highway can be simulated:

1. We need to determine if the car touches the position of an armadillo and thus affects it.
runs over.
2. We need to count how many animals were touched by the car at a given time, therefore
be run over so that we can increase the score accordingly.
3. Finally, we have to run over all the animals whose position gets under the car.
We will work through this list in this section.

Armadillo Touch: Next on the list from page 344, we need to determine if the car touches the position of an armadillo. Here is a brief description and signature for this:

Does the car touch a position? (: car-on-position?

(position position -> boolean))

The car touches a position if the position on the same side of the road is close to it. Here are the test cases:

```
(check-expect (car-on-position? (make-position 10 "left")
                                (make-position 10 "right"))
               #f)
(check-expect (car-on-position? (make-position 10 "left") (make-position 10 "left"))
               #t)
(check-expect (car-on-position? (make-position 11 "left") (make-position 10 "left"))
               #t)
(check-expect (car-on-position? (make-position 10 "left") (make-position 11 "left"))
               #t)
```

For the template, we are dealing with composite data in both inputs:

```
(define car-on-position? (lambda
  (car-position position)
  ...
  (position-side car-position) (position-side
    position) (position-m-from-start car-
    position) (position-m-from-start position)
  ...))
```

The two sides of the road must be the same. For the road meters of the two positions, it is relevant that the car's road meters are exactly in the middle of the car – the position must not be more than halfway away from it. To facilitate comparison, regardless of whether the position is in front of or behind the center point, we use the built-in function `abs`. This function calculates *the absolute value* of a number: it converts negative numbers into their corresponding positive values.

(paragraph 5)

5

(abs -5)

5

Here is the finished function:

```
(define car-on-position? (lambda
  (car-position position) (and (string=? (position-
    side car-position) (position-side position))

    (<= (abs (- (position-m-from-start car-position) (position-m-from-start
      position))))
    (/ car-length 2))))
```

How many animals are currently under the car? Next, we'll write a function that counts the animals under the car so we can increase the score accordingly. More precisely, we'll only count *live* armadillos: running over a dead armadillo again doesn't award any points. Here's a short description and signature:

How many animals are touched by the car? (: live-dillos-under-car-count

(position (list-of dillo-on-road) -> natural))

For the test, we should include both live and dead armadillos in the input, as well as those lying under the car and those that are further away:

```
(check-expect
  (live-dillos-under-car-count
    (make-position 10 "left") (list (make-
      dillo-on-road dillo1 (make-position 10 "left"))
      (make-dillo-on-road dillo2 (make-position 10 "right"))
```

```
(make-dillo-on-road dillo2 (make-position 11 "left")) (make-dillo-on-road dillo1
(make-position 9 "left")) (make-dillo-on-road dillo1 (make-position 12 "left"))))
```

2)

The test counts the first and the second-to-last armadillo.

For the definition, we could use the template for functions on lists. Since it

But if the goal is to select elements from a list that meet a specific criterion, we can also use the `filter` function from page 259, the built-in version of `extract-list`. With this, we extract the armadillos to be counted. We just need to...

You can also count using the built-in `length` function. (`Length` is the built-in version of `list-length`, see page 186.)

```
(define live-dillos-under-car-count
```

```
  (lambda (car-position dillos-on-road) (length (filter
    (lambda
      (dillo-on-road) (and (car-on-position?
```

```
        car-position (dillo-
          on-road-position dillo-on-road)) (dillo-alive?
```

```
        (dillo-on-road-state dillo-on-road))))
    dillos-on-road))))
```

Armadillos actually run over. The last task is to change the armadillos hit by the car from a live to a dead state. Like `live-dillos-under-car-count`, the function accepts the position of the car and the armadillos and returns a list of updated armadillo states:

All animals that the car touches were run over.

```
(: run-over-dillos-on-road
```

```
  (position (list-of dillo-on-road) -> (list-of dillo-on-road)))
```

During testing, there are a lot of dead armadillos, so we define the corresponding condition for reuse purposes:

```
(define dead-dillo1 (run-over-dillo dillo1))
```

```
(check-expect (run-
  over-dillos-on-road
  (make-position 10 "left") (list (make-
    dillo-on-road dillo1 (make-position 10 "left"))
    (make-dillo-on-road dillo1 (make-position 10 "right")) (make-dillo-on-road dillo1
      (make-position 11 "left")) (make-dillo-on-road dillo1 (make-position 9 "left")) (make-
        dillo-on-road dillo1 (make-position 12 "left")))) (list (make-dillo-on-road dead-
          dillo1 (make-position 10 "left"))
          (make-dillo-on-road dillo1 (make-position 10 "right")) (make-dillo-on-road dead-
            dillo1 (make-position 11 "left")) (make-dillo-on-road dead-dillo1 (make-position 9 "left"))
            (make-dillo-on-road dillo1 (make-position 12 "left"))))
```

The function therefore applies an operation to each element of the list and returns the results to us in a list: For this we can again use a higher-order function from Chapter 9, namely `map`, the built-in version of `list-map`, see page 265.

```
(define run-over-dillos-on-road
  (lambda (car-position dillos-on-road) (map (lambda (dillo-
    on-road) ...) dillos-on-road)))
```

The argument of the lambda function consists of composite data. Therefore, selector calls are included in the template:

```
(define run-over-dillos-on-road
  (lambda (car-position dillos-on-road) (map (lambda (dillo-
    on-road)
      ...
      (dillo-on-road-position dillo-on-road)
      (dillo-on-road-state dillo-on-road)
      ...)
    dillos-on-road)))
```

The armadillos fall into two classes: those that are just touched by the car, and all the others. We can use the function ``car-on-position?`` to create a binary branch:

```

(define run-over-dillos-on-road
  (lambda (car-position dillos-on-road) (map (lambda (dillo-
    on-road) (if (car-on-position? car-position
      (dillo-on-road-position dillo-on-
        road))

        ...

        ...))

    ...

    (dillo-on-road-position dillo-on-road) (dillo-on-road-state dillo-
      on-road) ...))

    dillos-on-road)))

```

In the first case, we should run over the armadillo, which we do with `run-over-dillo`. In the other case, we return `dillo-on-road` unharmed:

```

(define run-over-dillos-on-road
  (lambda (car-position dillos-on-road) (map (lambda (dillo-
    on-road) (if (car-on-position?

      car position (dillo-
        on-road-position dillo-on-road))
      (make-dillo-on-road
        (run-over-dillo (dillo-on-road-state dillo-on-road))
        (dillo-on-road-position dillo-on-road)) dillo-on-road))

    dillos-on-road)))

```

Task 11.5

What would be the consequence if running over dead armadillos also awarded points? Try to figure this out by thinking it through and then try it out later!

Ÿ

11.9 REACTIVE ANIMATION

At this point, we have described all the basic aspects of the game through functions. We still need to combine them into a game. This should then also be responsive to key presses.

react. For such reactive animations, we still need support from the Teachpack universe.rkt, which includes a new programming language element called `big-bang`. It starts the game and has the following form:

```
(big-bang world clause . . .)
```

The first operand, `world`, is the world at the beginning of the game. Following this are several clauses that describe what the game should do when certain things happen. These can include:

- a key on the keyboard is pressed
- a key on the keyboard is released
- the mouse is moved or a mouse button is pressed.

Furthermore, there is a clause that specifies how an image is generated from the world. It looks like this. out of:

```
(to-draw function)
```

...and the function must meet the signature `(world -> image)`. That's just enough to try out `big-bang` for the first time:

```
(big-bang initial-world
  (to-draw world->image))
```

At least the world is displayed at the beginning, but it doesn't move yet. However, it could react to a keystroke; this can be achieved with a clause of the following form:

```
(on-key function)
```

This function must have the following signature:

```
(world string -> world)
```

It is called up whenever a key is pressed – using the information from before the key press and a string corresponding to the key. For example, `"a"` for the A key, `"+"` for the + key, and so on. For the "special keys," it looks like this:

<code>"left"</code>	Left arrow
<code>"right"</code>	Right arrow
<code>"up"</code>	Up arrow
<code>"down"</code>	Down arrow
<code>"shift"</code>	left shift key
<code>"rshift"</code>	right Shift key

For our game, we only need the left and right arrow keys to move the car to the left or right side of the road. Therefore, we need a function with the following signature:

```
; React to key press (: react-to-key (world
string -> world))
```

We're testing both arrow keys – all other keys should leave the world unchanged:

```
(check-expect (react-
  to-key (make-world 12 "right" dillos-on-road 5) "left")
  (make-world 12 "left" dillos-on-road 5))
```

```
(check-expect
  (react-to-key (make-world 12 "left" dillos-on-road 5) "right") (make-world 12 "right" dillos-on-road
  5))
```

```
(check-expect (react-
  to-key (make-world 12 "left" dillos-on-road 5) "a") (make-world 12 "left" dillos-on-road 5))
```

Here is the framework:

```
(define react-to-key (lambda
  (world key) ...))
```

"Key" is effectively a list: left, right, everything else. So the template looks like this:

```
(define react-to-key (lambda
  (world key) (cond ((string=?
    key
      "left") ...) ((string=? key "right") ...)
    (else ...))))
```

Furthermore, the design instructions provide templates for composite data as both input and output:

```
(define react-to-key (lambda
  (world key) (cond ((string=?
    key
      "left")
    (make-world ... ... ... ...))
```

```

((string=? key "right") (make-
  world ... ..)) (else ...))

...

(world-ticks world) (world-
  car-side world)
(world-dillos-on-road world)
(world-score-road world) ...))

```

The left and right arrow keys only change the side of the car. All other keys do not change anything.

```

(define react-to-key
  (lambda (world key) (cond

    ((string=? key "left")
      (make-world (world-ticks world)
        "left"
        (world-dillos-on-road world)
        (world-score world))))

    ((string=? key "right") (make-world
      (world-ticks world)
      "right"
      (world-dillos-on-road world) (world-score
        world)))

    (else world))))

```

This allows us to extend the `big-bang` call :

```

(big-bang initial-world (to-draw
  world->image) (on-key react-to-
  key))

```

The road is still permanently at the beginning, but we can move the car left and right using the arrow keys.

We've had the street moving before, but we used [an animation](#) then.

`Big Bang` works a little differently; there's a clause like this:

```

(on-tick function)

```

The function `on-tick` is always called when a tick has passed and must have the following signature:

```
(world -> world)
```

This means that the function doesn't receive the number of ticks as a value, as with ``animate``, but has to count them itself. Let's start writing such a function:

How does the world change when a tick of time passes? (: next-world (world -> world))

The framework and template exploit the fact that both input and output are composite data:

```
(define next-world
  (lambda (world)
    (make-world ... ..)
    ...
    (world-ticks world)
    (world-car-side world) (world-
      dillos-on-road world)
    (world-score world)
    ...))
```

First, we'll write a very simple function where only time passes:

```
(define next-world (lambda
  (world)
    (make-world (+ 1 (world-ticks world))
      (world-car-side world) (world-
        dillos-on-road world)
        (world-score world))))
```

We will now incorporate this function into the big-bang expression:

```
(big-bang initial-world (to-draw
  world->image) (on-tick next-world)

  (on-key react-to-key)
```

That looks better: The road is moving, and you can steer the car left and right. But run-over armadillos still don't die, and frustratingly, the score isn't increasing either. That's because we haven't yet enabled the two functions responsible for that –

We haven't yet implemented `run-over-dillos-on-road` and `live-dillos-under-car-count`. We'll add that later.

```
(define next-world (lambda
  (world)
    (define car-position (world-car-position world)) (define dillos-on-road
      (world-dillos-on-road world))
    (make-world (+ 1 (world-ticks world))
      (world-car-side world) (run-over-
        dillos-on-road car-position dillos-on-road)
      (+ (live-dillos-under-car-count
        car position dillos-on-road) (world score
        world))))))
```

There you have it, have fun running over armadillos!

Task 11.6

Change the game so that running over dead armadillos also awards points! Does that make sense? ÿ

Here are some more useful clauses for `big-bang` for more demanding games:

```
(on-mouse function)
```

The function is always called when the mouse is used – that is, when it is moved or when a button is pressed. It must have the following signature:

```
(world integer integer mouse-event -> world)
```

The two integer arguments are the X and Y coordinates of the mouse position. The `mouse-event` signature is defined as follows:

```
(define mouse-event
  (signature
    (enum "button-down" "button-up" "drag" "move" "enter" "leave")))
```

The individual cases have the following meanings:

`"button-down"` The mouse button was pressed. `"button-up"`

`"drag"` The mouse button was released.

The mouse is dragged, meaning it moves while the button is pressed.

"move"	The mouse was moved.
"enter"	The mouse cursor has been moved into the universe.rkt window.
"leave"	The mouse cursor has been moved out of the universe.rkt window.

There's another cool clause called `on-pad`:

(on-pad function)

Once specified, a "game pad" will be displayed, which looks like this:



In the function's signature, the `pad-event values` correspond to the buttons on the pad:

(world pad-event -> world)

(define pad-event

 (signature

 (enum "left" "right" "up" "down" "w" "s" "a" "d" " " " " "shift" "rshift")))

Task 11.7

Extend the `big-bang` command so that the game can also be played with a gamepad! ÿ

TASKS

Task 11.8

Make the armadillo game better!

Task 11.9

Write a video game of your choice.

12 TREES

Trees are a form of data that (like lists) is particularly common in computer science.

Often, tree-like data definitions arise from the problem itself. Abstracting from these data definitions yields a universally usable form of data: the *binary tree*. These binary trees are as versatile as lists and also allow us to organize data into trees in a way that makes it easy to retrieve information quickly.

12.1 SOURCE TREES

`trees/family-tree.rkt`

code

The idea of using a tree as a metaphor for a specific type of data can already be found in the Bible, which uses words like "tree stump" and "shoot" to describe lineage. The first pictorial representations of family trees were derived from these descriptions starting in the 11th century. Family trees typically list a person's name and their relationship to both parents. This leads to the following definition of data:

A person has the following characteristics:

; - Name

; - Parent #1

; - Parent #2

This definition leaves open what kind of data the two parents represent. Of course, they are also people, but if we go far enough up a family tree, they eventually become unknown: every concrete family tree ends somewhere. Therefore, we also need a representation for an "unknown parent" without (known) attributes:

An unknown parent has no properties (define-record unknown-parent)

`make-unknown-parent`

`unknown-parent?)`

This results in a data definition for "parent":

A parent is one of the following: - a person

; - an unknown parent

We can now translate this – and the data definition for "person" – into code:

```
(define-record person
  make person
  person?
  (person-name string) (person-
    parent-1 parent) (person-parent-2
    parent))
```

```
(define parent
  (signature
    (mixed person unknown-parent)))
```

Let's establish a value for the unknown parent right away:

```
(define an-unknown-parent (make-unknown-parent))
```

Here is a small family tree as an example:

```
(define slash
  (make-person "Slash" (make-
    person "Ola Hudson"
      an-unknown-parent an-
      unknown-parent) (make-
    person "Anthony Hudson" an-unknown-
      parent an-unknown-
      parent)))

(define london-hudson
  (make-person "London Hudson"
    slash
    (make-person "Perla Ferrar" an-unknown-
      parent an-unknown-
      parent)))
```

We will now write a function that determines whether someone is the ancestor of a person, so

approximately:

```
Is someone an ancestor of a person? (: ancestor?
(string person -> boolean))
```

```
(check-expect (ancestor? "Slash" london-hudson) #t) (check-expect
(ancestor? "Axl" london-hudson) #f)
```

The template for this function looks like this:

```
(define ancestor?
  (lambda (name person)
    ...
    (person-name person)
    (person-parent-1 person) (person-
      parent-2 person) ...))
```

What can we do with these components? We could compare the person's name with the name we're looking for – if so, it's an ancestor:

```
(define ancestor?
  (lambda (name person)
    (if (string=? name (person-name person))
        #t
        ...)
    (person-parent-1 person) (person-
      parent-2 person)
    ...))
```

The data sets `(person-parent-1 person)` and `(person-parent-2 person)` are mixed data. We could implement the necessary branching directly in ``ancestor?``. Alternatively, we could write a separate function that answers the question of whether a parent is an ancestor. Since there are two parents, such a separate function with a short description and signature, like this, would be worthwhile:

```
Is someone an ancestor of a parent? (: parent-ancestor?
(string parent -> boolean))
```

We'll write this function later. But your signature is enough to continue filling out the `ancestor` template. We check whether parent number 1 or number 2 is the ancestor:

```
(define ancestor?
  (lambda (name person)
    (if (string=? name (person-name person))
        #t
```

```
(if (or (parent-ancestor? name (person-parent-1 person)) (parent-ancestor?
      name (person-parent-2 person)))
    #t
    #f)))
```

This function is correct, but it could be a bit more elegant. The second if statement returns `#t` if the condition is `#t` and `#f` if the condition is `#f` : It always returns the result of the condition. This is a generally applicable rule:

```
(if b #t #f) = b
```

Can we shorten it to:

```
(define ancestor?
  (lambda (name person)
    (if (string=? name (person-name person))
        #t
        (or (parent-ancestor? name (person-parent-1 person))
            (parent-ancestor? name (person-parent-2 person))))))
```

We can also get rid of the remaining if expression because it returns `#t` if the condition returns `#t` or if the or expression returns `#t` . Therefore, we can write the function with a capital `or` :

```
(define ancestor?
  (lambda (name person)
    (or (string=? name (person-name person)) (parent-
      ancestor? name (person-parent-1 person)) (parent-ancestor? name
      (person-parent-2 person)))))
```

This simplification wasn't necessary, but we think the result looks much nicer!

The helper function `parent-ancestor` is still missing . Here are a few tests:

```
(check-expect (parent-ancestor? "Slash" london-hudson) #t) (check-expect (parent-
  ancestor? "Axl" london-hudson) #f) (check-expect (parent-ancestor? "Slash" an-
  unknown-parent) #f)
```

The framework and template result – as always – from the data definition of `parent`:

```
(define parent-ancestor? (lambda
  (name parent)
    (cond
```

```
((person? parent) ...) ((unknown-
parent? parent) ...)))
```

For the first case, we can use ``ancestor?``, in the second case we can use ``#f`` answer-
ten:

```
(define parent-ancestor? (lambda
  (name parent) (cond ((person?
    parent)
    (ancestor? name parent)) ((unknown-parent? parent) #f))))
```

Complete!

Task 12.1

Modify the ``ancestor`` function so that a person is not their own ancestor. Make sure the function still works correctly otherwise! Does the function become simpler? ÿ

12.2 Binary Trees

[trees/binary-tree.rkt](#)

code

Let's take another look at the record definition of `person` :

```
(define-record person make-
  person person?
  (person-
    name string) (person-
    parent-1 parent) (person-parent-2
    parent))
```

Perhaps this reminds you of a record definition from Chapter 5:

```
(define-record confluence
  make-confluence
  confluence?
  (confluence-location string) (confluence-
    main-stem river)
  (confluence-tributary river))
```

The structure is the same for both definitions. In particular, both definitions contain two self-references. In the case of [person](#) , the self-reference is to [parent](#), which is defined as follows:

```
(define parent
  (signature
    (mixed person unknown-parent)))
```

In the case of [Confluence](#) , the self-reference to [River](#) is:

```
(define river
  (signature (mixed creek confluence)))
```

The other cases of [parent](#) and [person](#) differ slightly:

```
(define-record unknown-parent
  make-unknown-parent
  unknown-parent?)
```

```
(define-record creek
  make-creek
  Creek?
  (creek-origin string))
```

Neither of them contains any self-reference. Both data definitions depict tree-like structures: A [person](#) or [confluence record](#) forms a branch that splits into two.

A tree always ends at [unknown-parent](#) or creek records. Because the "inner" branches always branch twice, both are *binary trees*. We can abstract from these two sets of

definitions. Let's start with [person](#) and [confluence](#) . The common name for the branches within a binary tree is *node* or *inner node* . We also need a name for the "name data" that is present in both. *Label* is the usual term. We simply call the signature for self-reference [a tree](#).

```
(define-record node
  make-node node?
  (node-label string)
  (node-left-branch tree)
  (node-right-branch tree))
```

The word "branch" literally translates to "branch", but we use the terms "left subtree" and "right subtree", which are more common in German.

When defining a *tree*, we still need a name for the values at the edges.
of the tree – called *leaves*, in English *leaf*.

```
(define tree
  (signature (mixed leaf node)))
```

The definition of ``leaf`` is still missing. It's not quite so simple here, because ``creek`` already has a name, but ``unknown-parent`` does not. So we have to abstract from both. We already have a name – ``leaf`` – we just need the **lambda**:

```
(define tree-of
  (lambda (leaf)
    (signature (mixed leaf node))))
```

This necessitates a further change, because ``tree`` is used in the definition of ``node``. We must also include the ``leaf`` parameter accordingly :

```
(define-record (node-of leaf)
  make-node node?
  (node-label string) (node-left-
    branch (tree-of leaf)) (node-right-branch (tree-of
    leaf)))
```

We have only seen the notation for abstracting the record signature with the extra brackets around `(node-of leaf)` once before, in the definition of `cons-list-of` in section 6.2 on page 180.

We could be finished here. However, we'll make one more generalization: As we'll see, the markers in trees don't necessarily have to be strings – we'll want to store other types of values there as well. Therefore, we'll also abstract about the signature of the markers. Furthermore, we'll provide the data definitions below:

```
A node consists of
; - marking ; - left
branch
; - right branch
(define-record (node-of leaf label)
  make-node node?
  (node-label label) (node-
    left-branch (tree-of leaf label))
  (node-right-branch (tree-of leaf label)))
```

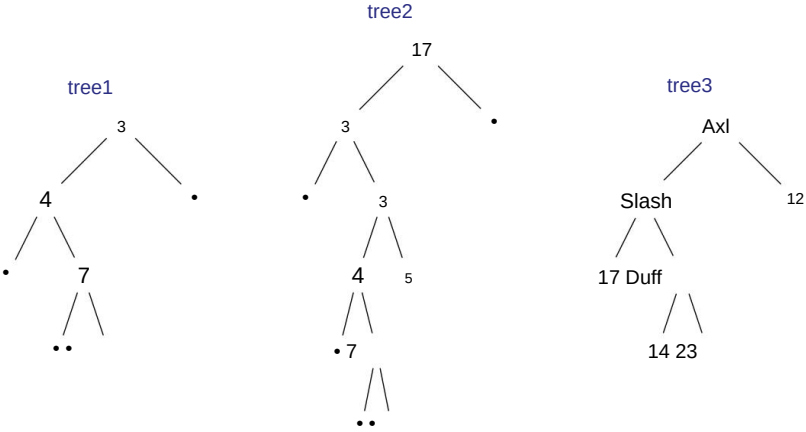


Figure 12.1: Example trees

A binary tree is either a leaf or a node.

```
(define tree-of
  (lambda (leaf label)
    (signature (mixed leaf (node-of leaf label))))))
```

This is the pure definition of a binary tree. Here are two examples where we simply used the value #f as the leaf:

```
(: tree1 (tree-of false number))
(define tree1 (make-node 3 (make-node 4 #f (make-node 7 #f #f)) #f)) (: tree2 (tree-of false number))

(define tree2 (make-node 17 (make-node 3 #f tree1) #f))
```

Here is another example where the leaves are numbers and the markers are strings of characters.

```
(: tree3 (tree-of number string)) (define tree3 (make-
node "Axl"
      (make-node "Slash" 17
        (make-node "Duff" 14 23)
        12))
```

Figure 12.1 graphically represents the three trees tree1, tree2 , and tree3 . Each tree clearly has a "topmost" node, the so-called root – a term that, in this context,

The term is frequently used in relation to trees. It's always necessary to specify what the root refers to. For example, `tree1` is a part of `tree2`. Accordingly, the root of `tree1` (the node with the 3) is also contained within `tree2`, but the root of `tree2` is the node with the 17.

The concept of a *path* is also related to the root: The path of a node or leaf is the list of nodes from the root to that node or leaf. For example, the path of node 14 in `tree3` in Figure 12.1 consists of the nodes Axl, Slash, and Duff.

We can use `tree-of` to compare the definitions of `river` and `confluence`.
simple:

```
(define river (tree-of creek string)) (define river? node?)

(define make-confluence make-node)
(define confluence-location node-label) (define confluence-
main-stem node-left-branch)
(define confluence-tributary node-right-branch)
```

Task 12.2

Define `person` using `tree-of`!

Ÿ

You can calculate all sorts of things on trees. One example is the *depth*, which is the maximum number of nodes on the way to a leaf. (Sometimes this quantity is also called the *height* of a tree.) The signatures of the leaves and markings are irrelevant for the depth of the tree.

Calculate the depth of a tree

```
(: depth ((tree-of %leaf %label) -> natural))
```

Here are two test cases:

```
(check-expect (depth tree1) 3) (check-expect
(depth tree2) 5)
```

In `tree1`, the nodes with the markers 3, 4, and 7 form the longest path to a leaf. In `tree2`, it is 17, 3, 3, 4, 7.

Let's continue with the construction instructions. We need the mixed data template as input. Since the data definition for binary trees has two cases, we need a `cond` with two branches. For the first, we can form conditions using `node?`. The leaves don't have a fixed predicate, but they are simply all the trees that are not nodes – so instead of a condition, we can write `else`:

```
(define depth
  (lambda (tree) (cond
    ((node? tree) ...)
    (else ... 0))))
```

The nodes contain two self-references, so we need two recursive calls:

```
(define depth
  (lambda (tree) (cond
    ((node? tree)
     ...
     (depth (node-left-branch tree)) (depth (node-right-
       branch tree)) ...) (else ...))))
```

For depth calculations, only the path with the maximum number of nodes counts. We also need to include the nodes within [the tree structure](#) . Leaves are not counted at all.

```
(define depth
  (lambda (tree)
    (cond
      ((node? tree)
       (+1
        (max (depth (node-left-branch tree))
              (depth (node-right-branch tree)))))
      (else 0))))
```

Complete!

Task 12.3

Write a function that counts all the nodes in a tree!

Ÿ

Task 12.4

Write a function that returns a list of all the leaves in a given tree!

Ÿ

12.3 TREES FOR SEARCHING

Many programming problems are "search problems": finding a name, a phone number, or an order number from a list. That's what this section is about, and we'll start by taking the word "list" literally and writing a function like this:

```
Is the value an element of a list? (: member?
(%a (list-of %a) -> boolean))
```

We used a signature variable because the list elements can sometimes be numbers, sometimes strings, and sometimes something else. Here are a few test cases:

```
(check-expect (member? 5 empty) #f) (check-expect
(member? 2 (list 1 2 3)) #t) (check-expect (member? "Slash" (list
"Axl" "Slash")) #t) (check-expect (member? "Buckethead" (list "Axl" "Slash")) #f)
```

Here is a framework and template for functions on lists:

```
(define member?
  (lambda (element list)
    (cond
      ((empty? list) ...) ((cons? list)
      ...
      (first list)
      (member? element (rest list))
      ...))))
```

For an empty list, the function can only return `#f`. For a cons list, the template suggests that the function should first check if `(first list)` is the element being sought.

Sounds simple, right? But *how* do we check that? We could write it down here:

```
(= element (first list))
```

...but that would restrict the function to numbers, because `=` only works on numbers. For the two test cases with strings, we would have to use `string=?`. So we need to abstract `member?` via `=` and `string=?` respectively, even before the function is finished. As always, we need another parameter and call it `equals?`:

```
(define member?
  (lambda (equals? element list) (cond
    ((empty? list) ...) ((cons?
      list)
      ...
      (first list)
      (member? equals? element (rest list)) ...))))
```

(Note: The recursive call must – as always – also be extended by the new parameter.)

Now we can perform the comparison using ``equalities?`` and compare it with a Process binary branching:

```
(define member?
  (lambda (equals? element list)
    (cond
      ((empty? list) #f) ((cons?
        list)
        (if (equals? element (first list))
            #t
            (member? equals? element (rest list)))))))
```

Signatures and test cases haven't yet been affected by the new parameter. The ``equals?`` function accepts two list elements and returns a Boolean result. Since the list elements have the signature ``%a``, the signature declaration for ``member?`` looks like this:

```
(: member? ((%a %a -> boolean) %a (list-of %a) -> boolean))
```

For each test case, we need to pass the correct comparison function. That's ``=`` for numbers and ``equals?`` for strings.

```
(check-expect (member? = 5 empty) #f) (check-
expect (member? = 2 (list 1 2 3)) #t) (check-expect (member?
string=? "Slash" (list "Axl" "Slash")) #t) (check-expect (member? string=? "Buckethead" (list
"Axl" "Slash")) #f)
```

Complete!

However, the "member?" feature has a disadvantage: With short lists or when the searched element is missing, it's not ideal. If the list starts with "member?" , it will finish fairly quickly. But imagine the list has... A few million elements and the element you're looking for is at the end. Or not there at all: Then Must be a member? Go through the entire list.

Task 12.5

Using `member`, write a function that returns all elements from two lists that are in the list containing the element in the list. Both lists are listed. How long does this function take in the worst-case scenario? ÿ

Is there any way for a program to find out faster whether a value is an element of a set or Isn't that possible? Indeed, it is, but not with lists: We need a different structure to do that. Search to speed up – trees.

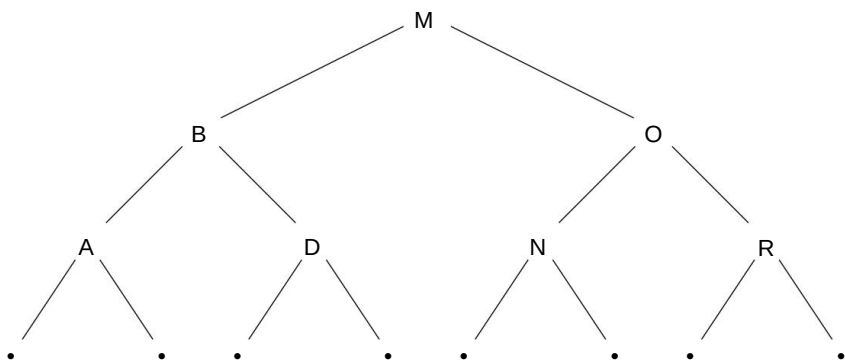


Figure 12.2: Sorted tree by letters

Take a look at Figure 12.2 . In this tree, you don't need to look at all the elements to find a specific element. This is because the letters in the tree are based on certain types are sorted alphabetically:

The root with the label M has two subtrees – the labels of the left subtree are all *before* M in the alphabet, all labels of the right subtree are after M. So if you're looking for a letter – let's say D – then you know, if you see the root with M, that D is in the left subtree of M – marked B, and from there in the The right subtree of B must be located there. You can ignore nodes A, O, N, and R.

The search therefore requires at most as many steps as the tree is deep. That's already a good start. It's easier than searching in the list, where we potentially have to look at all the elements.

So let's program this!

We start with a sorted tree of real numbers. (Real numbers because we can easily compare them using `=` and `<`. We'll generalize this later.) The numbers are attached to the nodes as markers. There's nothing relevant on the leaves, so we always use `#f`. Accordingly, the short description and signature look like this:

Is a number present in a sorted tree?

```
(: tree-member? (real (tree-of false real) -> boolean))
```

The signature ``false`` is new: it only describes the value ``#f``. Accordingly, there is also a signature ``true`` for ``#t``.

Here is an example tree and some tests that use it:

```
(define tree4
  (make-node 5
    (make-node 3 #f #f)
    (make-node 17
      (make-node 10 #f (make-node 12 #f #f)) #f)))
```

```
(check-expect (tree-member? 5 tree4) #t) (check-expect
(tree-member? 17 tree4) #t) (check-expect (tree-member?
3 tree4) #t) (check-expect (tree-member? 10 tree4) #t)
(check-expect (tree-member? 2 tree4) #f)
```

Here is the framework:

```
(define tree-member?
  (lambda (value tree) ...))
```

We'll add the second case to the tree template: If the function reaches a leaf, then the `value` is definitely not in the tree; the result is then `#f`:

```
(define tree-member?
  (lambda (value tree)
    (cond
      ((node? tree)
       ...
       (node-label tree))
```

```

    (tree-member? value (node-left-branch tree))
    (tree-member? value (node-right-branch tree))
    ...
    (else #f))))

```

For nodes, we can distinguish three cases: when the marker is the **value** we are looking for , when the **value** is smaller than the marker (and therefore must be in the left subtree), and when it is larger. This leads to the following further development:

```

(define tree-member?
  (lambda (value tree)
    (cond
      ((node? tree)
       ...
       (tree-member? value (node-left-branch tree)))
       (tree-member? value (node-right-branch tree))
       ...
      (cond
        ((= value (node-label tree)) #t) ((< value
        (node-label tree)) ...
        (else ...)))
      (else #f))))

```

Since `(node-label tree)` appears twice, we define it and set the
Assemble the components of the template as follows:

```

(define tree-member?
  (lambda (value tree)
    (cond
      ((node? tree)
       (define label (node-label tree))
       (cond
         ((= value label) #t)
         ((< value label)
          (tree-member? value (node-left-branch tree))) (else
          (tree-member? value (node-right-branch tree))))))
      (else #f))))

```

12.4 PRODUCE SORTED TREES

In the test cases for `tree-member?`, we always used the tree `tree4`, which we constructed directly with `make-node`. We had to ensure ourselves that it also...

is sorted. In this section, we automate this construction. We write a

Function that inserts a new element into an existing sorted tree:

```
; Insert number into sorted tree (: tree-insert (real (tree-
of false real) -> (tree-of false real)))
```

Next, we need test cases. We could do it the way we always do: We write a

The call to `tree-member?` and the expected result value are used. However, in this case, the exact result value isn't crucial. What matters is that the inserted element is actually present in the resulting tree. Furthermore, it's tedious to always write out the entire tree.

That's why we use `tree-member` to test `tree-insert`.

```
(check-expect (tree-member? 5 (tree-insert 5 tree4)) #t) (check-expect (tree-member? 11
(tree-insert 11 tree4)) #f)
```

Later we will find that `tree-insert` can deliver different sorted trees, all of which are correct.

Task 12.6

Both tests expect `#t` to return `tree-member?`. Would it be useful to also write tests using `#f ?`

If you have a bad feeling about the two meager tests: you're right! In Chapter 13 on page 401, we will show how to better test functions like `tree-insert`.

Are the framework and template of `tree-insert` exactly the same as those of `tree-member`?

```
(define tree-insert
  (lambda (value tree)
    (cond
      ((node? tree)
       ...
       (tree-insert value (node-left-branch tree))
       (tree-insert value (node-right-branch tree)) ...)
      (else ...))))
```

The case distinction for nodes is also the same as in `tree-member?`, therefore we can also adopt the branching from the template there:

```
(define tree-insert
  (lambda (value tree)
    (cond
      ((node? tree)
       ...
       (tree-insert value (node-left-branch tree))
       (tree-insert value (node-right-branch tree))
       ...
       (cond
         ((= value (node-label tree)) ...) ((< value (node-
           label tree)) ...)
         (else ...)))
      (else ...))))
```

Here too, the first case is simple: if `value` is the marker of a node, then the tree already contains the value, the function doesn't need to insert anything and can simply return `tree` . Even if `tree` is a leaf (the last `else`), it's quite simple: We construct a new, single-element tree:

```
(define tree-insert
  (lambda (value tree)
    (cond
      ((node? tree)
       ...
       (tree-insert value (node-left-branch tree)) (tree-insert value
         (node-right-branch tree))
       ...
       (cond
         ((= value (node-label tree)) tree)
         ((< value (node-label tree)) ...)
         (else ...)))
      (else (make-node value #f #f)))))
```

There remain two cases where the value to be inserted lies to the left or right of the node marker. It must be inserted accordingly in the left or right subtree.

That's exactly what the two recursive calls from the template do. The other subtree remains unchanged.

```
(define tree-insert
  (lambda (value tree)
    (cond
      ((node? tree) (cond
        ((= value (node-label tree)) tree)
        ((< value (node-label tree))
         (make-node (node-label tree)
                    (tree-insert value (node-left-branch tree))
                    (node-right-branch tree)))
        (else
         (make-node (node-label tree)
                    (node-left-branch tree) (tree-insert
                                              value (node-right-branch tree))))))
      (else
       (make-node value #f #f)))))
```

Complete!

12.5 SEARCH TREES

Our functions `tree-member?` and `tree-insert` only work on numbers. The building-

However, the values shown in Figures 12.2 and 12.4 both contain letters. If we want to allow values other than numbers, we must once again abstract away from everything related to numbers.

has to do.

Take another look at the definition of `tree-member`: There are two places that are "number-specific", namely `=` and `<`. If we wanted to include strings in a sorted tree, we would have to write `string=?` and `string<?`.

Task 12.7

Abstract `tree-member?` and `tree-insert` using `=` and `<`.

Try passing the function `>` instead of `<`. Do `tree-member` and `tree-insert` still work correctly then? How do the trees that come out of `tree-insert` with `<` differ from those that do not? those with `>`?

The abstracted versions of `tree-member?` and `tree-insert` have a disadvantage:

Each time we call these functions, we must write down the two arguments for `=` and `<`.

This is annoying and always an opportunity for error because we have to do it completely consistently. So we want to try to do without these additional parameters. To do this, we use a trick and combine the functions for `=` and `<` with ``tree``. The result is called a *search tree*. The resulting data definition is as follows:

A search tree consists of

; - Function for `=`

; - Function for `<`

; - Binary tree

To implement the definition in code, we use a record definition. This time, we abstract the signature of the elements using a signature parameter called ``element``:

```
(define-record (search-tree-of element)
  make-search-tree search-tree?
  (search-tree-label=?-function (element element -> boolean)) (search-tree-label-<?-function
    (element element -> boolean))
  (search-tree-tree (tree-of false element)))
```

Compare the signature of ``search-tree-tree`` with the signature of the trees in ``tree-member?`` and ``tree-insert!``

Here is the search tree from Figure 12.2 on page 369:

```
(define search-tree1 (make-
  search-tree
  string=? string<? (make
    node "M"
      (make-node "B"
        (make-node "A" #f #f)
        (make-node "D" #f #f))
      (make-node "O"
        (make-node "N" #f #f)
        (make-node "R" #f #f)))))
```

We are now writing a variant of `tree-member?` that accepts search trees:

```
; determine if the element is present in the search tree
(: search-tree-member? (%a (search-tree-of %a) -> boolean))
```

```
(check-expect (search-tree-member? "M" search-tree1) #t) (check-expect
(search-tree-member? "D" search-tree1) #t) (check-expect (search-tree-member?
"N" search-tree1) #t) (check-expect (search-tree-member? "R" search-tree1) #t)
(check-expect (search-tree-member? "Z" search-tree1) #f)
```

Here is the framework for the function definition, along with the template for `search-tree`:

```
(define search-tree-member?
  (lambda (value search-tree)
    ...
    (search-tree-label=?-function search-tree) (search-tree-label-
    <?-function search-tree)
    (search-tree-tree search-tree)
    ...))
```

We use `tree-member` again and copy it into the trunk for this purpose:

```
(define search-tree-member?
  (lambda (value search-tree)
    ...
    (search-tree-label=?-function search-tree)
    (search-tree-label-<?-function search-tree)
    (search-tree-tree search-tree)
    ...
    (define tree-member?
      (lambda (value tree) (cond
        ((node? tree)
         (define label (node-label tree)) (cond
           ((= value label) #t)
           ((< value label)
            (tree-member? value (node-left-branch tree)))
           (else
            (tree-member? value (node-right-branch tree))))))
      (else #f))))
    ...))
```

However, we still need to abstract away from `=` and `<`. To do this, we'll first just invent new names: `label=?` for `=` and `label<?` for `<`.

```
(define search-tree-member?
  (lambda (value search-tree)
    ...
    (search-tree-label=?-function search-tree) (search-tree-label-
    <?-function search-tree)
    (search-tree-tree search-tree)
    ...
    (define tree-member?
      (lambda (value tree)
        (cond
          ((node? tree)
            (define label (node-label tree))
            (cond
              ((label=? value label) #t)
              ((label<? value label)
                (tree-member? value (node-left-branch tree))) (else
                (tree-member? value (node-right-branch tree))))))
          (else #f))))
    ...))
```

Unlike usual, we don't define any parameters for `label=?` and `label<?`: The functions for these are already in the template; we just need to give them the correct names using local definitions.

```
(define search-tree-member?
  (lambda (value search-tree)
    (define label=? (search-tree-label=?-function search-tree))
    (define label<? (search-tree-label-<?-function search-tree))
    ...
    (search-tree-tree search-tree)
    ...
    (define tree-member? ...)
    ...)))
```

Finally, we need a call to `tree-member?` to get started. For this, we use the last building block from the template:

```
(define search-tree-member?
  (lambda (value search-tree)
    (define label=? (search-tree-label=?-function search-tree))
    (define label<? (search-tree-label-<?-function search-tree)) (define tree-member?

      (lambda (value tree)
        (cond
          ((node? tree)
            (define label (node-label tree))
            (cond
              ((label=? value label) #t)
              ((label<? value label)
                (tree-member? value (node-left-branch tree))) (else
                (tree-member? value (node-right-branch tree))))))
          (else #f))))
    (tree-member? value (search-tree-tree search-tree))))
```

Task 12.8

Write a function called `search-tree-insert` based on `tree-insert`, corresponding to `search-tree-member`.

ÿ

12.6 SORTED TREES ARE MORE EFFICIENT THAN LISTS

Sorted trees are more efficient for searching. To understand why, consider the tree in terms of "levels"—the first level is the root, the second level its subtrees, the third level its subtrees, and so on. The better the tree is sorted, the fewer levels there are, and the fewer steps are needed for searching.

Each level can hold twice as many nodes as the level above it. In a tree with $2^1 = 1 + 2 = 3$ nodes, then...
 Depth 1 can hold 1 = 20 nodes, depth 2 can hold 2 = 7, then 15, and so ⁰
 on. You might notice that the numbers 1, 3, 7, and 15 are each the predecessor of a power of two. We can therefore try to generalize this into a formula.

The depth of the tree is called t . Then we assume (or at least hope) that for all t :

$$0^2 + \dots + 2t \cdot 1 = \sum_{i=0}^{t-1} i^2 = 2t \cdot 1$$

Since t is a natural number, we can use mathematical induction following the instructions in section 8.3 on page 225. We need to prove that for all $t \in \mathbb{N}$:

$$\sum_{i=0}^{t-1} i^2 = 2t \cdot 1$$

For $t = 0$, the sum ranges from 0 to -1 and is therefore empty. The result is the additive identity.

$$\begin{aligned} \sum_{i=0}^{-1} i^2 &= 0 \\ &= 1 \cdot 1 \\ &= 2 \cdot 0 \cdot 1 \end{aligned}$$

Induction prerequisite:

$$\sum_{i=0}^{t-1} i^2 = 2t \cdot 1$$

Inductive circuit (to be shown):

$$\sum_{i=0}^{(t+1)-1} i^2 = 2(t+1) \cdot 1$$

Proof:

$$\begin{aligned} \sum_{i=0}^{(t+1)-1} i^2 &= \sum_{i=0}^t i^2 \\ &= \sum_{i=0}^{t-1} i^2 + t^2 \\ &= 2t \cdot 1 + t^2 \text{ Induction hypothesis} = 2t \cdot 1 + t^2 \\ &= 2(t+1) \cdot 1 \end{aligned}$$

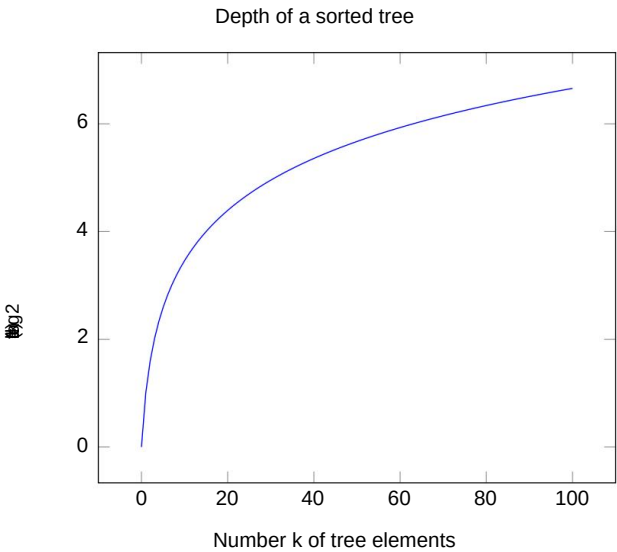


Figure 12.3: Relationship between the number of nodes and the depth of a binary tree

You might be wondering what this formula is good for. Well, we can flip the right side. (The left side is more difficult.) If the number of nodes is k , then:

$$\begin{aligned} k &= 2^{t-1} + 1 \\ t &= \log_2(k + 1) \end{aligned}$$

The $\log_2$ function is the so-called *logarithm* to base 2, also known as the *base-2 logarithm*. It is the inverse function of the exponential function with base 2.

Look at Figure 12.3 : You can see that the depth – and thus the logarithm – grows much more slowly than the number of nodes, and the curve becomes increasingly flat. This curve explains why using a sorted tree is a good idea: The depth of the tree is the number of elements you have to traverse to find the desired element (or rather, to determine that it's not in the search tree). However, this only applies if the search tree is "fully populated." So, at some point, we need to consider how to ensure that search trees are always as fully populated as possible. We'll get to that later.

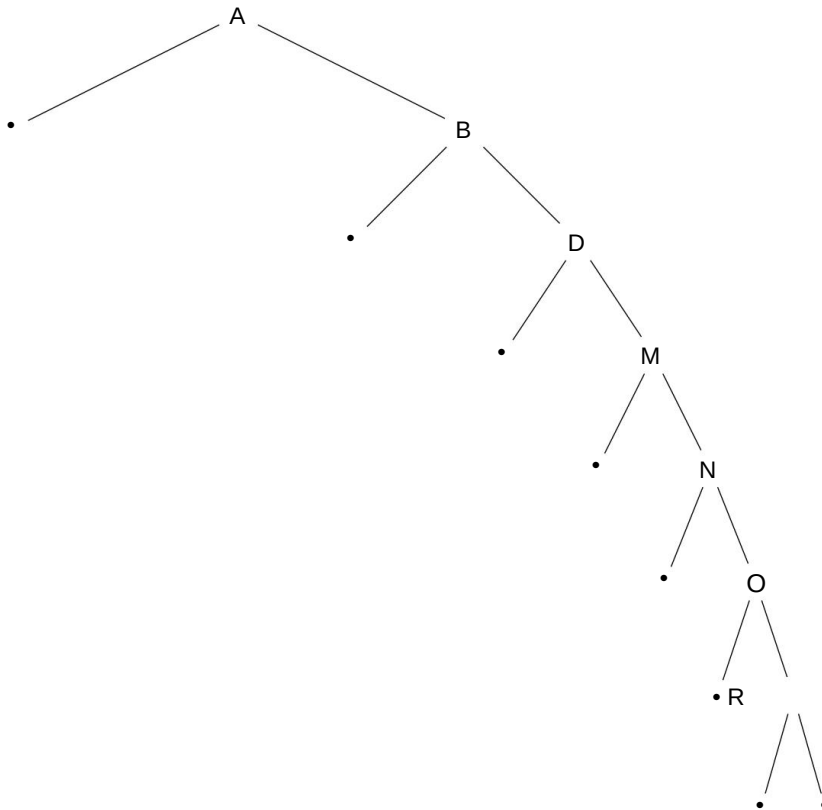


Figure 12.4: Degenerate search tree

12.7 BALANCE SEARCH TREES

Figure 12.4 shows a search tree that is not fully populated – a so-called *degenerate search tree*. Search tree. With this search tree, the search takes exactly the same amount of time as with a list.

Task 12.9

Write an expression of calls to `search-tree-insert` to create the search tree in Figure 12.4 !

ÿ

In this section, we want to write a variant of `search-tree-insert` where never-
Sometimes such a degenerate search tree can emerge, and the tree is always *balanced*.

At this point, we are entering an area of programming that is mostly unfamiliar in computer science. *Algorithms and Data Structures* is [BG20]. (Sometimes also just *algorithms* or just *data structures*.) An algorithm is generally a function that can be used in many programs. Accordingly, a data structure is a data definition on which an algorithm is based: the two often belong together.¹

Many algorithms are based on clever ideas, the creators of which can't even fully explain how they came up with them. Accordingly, we don't expect you to... (And neither did we) come up with the material in this section ourselves. Fortunately, in many areas of programming, there are already ready-made algorithms and data structures that we don't have to program ourselves. In this section, we show, using examples, how and why such a data structure works. You might find it interesting – If not, it is sufficient to skim or skip this section.

So how can we balance a search tree? In doing so, we should not only ensure that... that the search function works efficiently. Balancing shouldn't be too much work either – Otherwise, the advantages gained through searching will be negated by the effort required for balancing. We must find a compromise between the savings in searching and the Effort required for balancing. Such compromises are common in algorithms, and are often... Measurements were used to select and gradually improve them. That's the case here as well.

Here we show you a balancing function described in a scientific paper by Stephen Adams [Ada93] from 1993. This paper is based on a programming competition in which Adams came in second place in terms of efficiency. At the same time, his However, the solution was significantly more elegant than the first-placed one, which is why the essay is still frequently used today. is used to program efficient search trees.

Adams' idea is based on a variant of [search-tree-insert](#) that, when constructing a new node, ensures that the tree is not too unbalanced. If this threatens to happen, the tree is *rotated* and thus balanced.

12.7.1 Size-Annotated Trees

To determine if the tree is becoming unbalanced, the algorithm compares the number The node in the two subtrees of a node. Now it would be silly if it had to do that every time. You'd have to count them to find out how many there are. It's much more efficient to determine the size of a tree within a tree.

¹ The term "algorithm" comes from the name of the Persian mathematician *Al-Khwarizmi*. The German School computer science still has detailed definitions of the term, but these are not suitable for practical programming. are largely irrelevant.

We'll carry it ourselves as part of marking a node. We'll define that first, before we start balancing.

Trees where the size is displayed at each node are called *size-annotated*. In English, the term is less cumbersome: *sized*. Besides the size, we also need the "actual" marker at each node. Together, this results in the following data and record definition:

The marker on a size-annotated tree consists of:

```
; - Number of nodes
; - "actual" marking (define-record (sized-
label-of label)
  make-sized label
  sized-label
  (sized-label-size natural)
  (sized-label-label label))
```

Here is the tree from Figure 12.2 on page 369 again, but this time with size annotations:

```
(define sized-tree1
  (make-node (make-sized-label 7 "M")
    (make-node (make-sized-label 3 "B")
      (make-node (make-sized-label 1 "A") #f #f)
      (make-node (make-sized-label 1 "D") #f #f))
    (make-node (make-sized-label 3 "O")
      (make-node (make-sized-label 1 "N") #f #f)
      (make-node (make-sized-label 1 "R") #f #f))))
```

It's quite cumbersome to write down the correct size manually every time. We can easily let the computer do this by writing a helper function that calculates the size when constructing a node. Here is a short description and signature:

Construct size-annotated nodes

```
(: make-sized-node (%label (tree-of %leaf (sized-label-of %label)))
  (tree-of %leaf (sized-label-of %label)))
-> (node-of %leaf (sized-label-of %label)))
```

This signature matches that of `make-node` – here it is for comparison:

```
(: make-node (%label (tree-of %leaf %label) (tree-of %leaf %label))
  -> (node-of %leaf %label)))
```

For testing purposes, we take the expression from `sized-tree1` and replace the calls to `make-node`.

through which of `make-sized node`. The result should be the same:

```
(check-expect (make-sized-node
  "M"
  (make-sized node "B"
    (make-sized-node "A" #f #f) (make-sized-
      node "D" #f #f))
  (make-sized node "O"
    (make-sized-node "N" #f #f) (make-sized-
      node "R" #f #f)))
sized-tree1)
```

Here is the framework with a template for the function. The signature indicates that the function returns a `node` record; accordingly, the template consists of a call to the constructor:

```
(define make-sized node
  (lambda (label left-branch right-branch) (make-node ... .... ...)))
```

We can pass the two arguments for `left-branch` and `right-branch` directly to the constructor. We only need to calculate their size: this is the sum of the

both subtrees, plus 1, because the `make-node` creates another new node:

```
(define make-sized node
  (lambda (label left-branch right-branch) (make-node

    (make-sized label (+ 1
      (sized-tree-size left-branch) (sized-tree-size
        right-branch))
    label)
    left-branch right-branch)))
```

Look at the subexpression that calculates the size: It uses a function ``sized-tree-size`` which calculates the size of a tree. But that function doesn't exist yet – it's just wishful thinking! We still need to add that. Here's a short description, signature, and two tests:

; Provide the size of a size-annotated tree (: sized-tree-size

```
((tree-of %leaf (sized-label-of %label)) -> natural))
```

```
(check-expect (sized-tree-size "A") 0) (check-expect
(sized-tree-size sized-tree1) 7)
```

Here is the framework and template. The function processes mixed data, so we need a branch:

```
(define sized-tree-size
  (lambda (tree) (cond
    ((node? tree) ...)
    (else ...))))
```

In the node case, we already know where the size is located, namely in the sized-label tag. In the other case, it is a sheet that has a size of 0:

```
(define sized-tree-size
  (lambda (tree)
    (cond
      ((node? tree)
        (sized-label-size (node-label tree))) (else 0))))
```

If you look at the signature declarations of `make-sized-node` and `sized-tree-size`, you will see that the following signatures occur multiple times:

```
(node-of %leaf (sized-label-of %label))
(tree-of %leaf (sized-label-of %label))
```

This will happen more often, so we abstract:

```
; Signature for size-annotated nodes (define sized-node-
of
  (lambda (leaf label)
    (node-of leaf (sized-label-of label))))
```

```
; Signature for size-annotated trees (define sized-tree-
of
  (lambda (leaf label)
    (tree-of leaf (sized-label-of label))))
```

Task 12.10

Write better signature declarations for ``make-sized-node`` and ``sized-tree-size`` using these functions! y

We can also foresee that we will frequently need to mark a size-annotated node. Therefore, we define a helper function:

```
; extract the marker from a size-annotated node
(: sized-node-label ((sized-node-of %leaf %label) -> %label))
```

```
(define sized-node-label
  (lambda (node)
    (sized-label-label (node-label node))))
```

12.7.2 BALANCED SEARCH TREES

Unfortunately, there is no overarching strategy for algorithm development analogous to the...

Construction instructions. Often, it starts with an idea, viewed from a particular perspective.

The problem arises, linked to variations and experiments.

The starting point of Stephen Adams' idea is the `search-tree-insert` function .

It places the new element at the first suitable position in the search tree: This is the minimum possible overhead. Adams' idea is to essentially retain ``search-tree-insert`` , but

with one addition: If inserting a new element throws the tree out of balance

When something new is introduced, it gets a bit rebalanced. It's like whenever something new is introduced...

Buy the new item, put it on the shelf in the room, and then tidy up a little. The trick is to tidy up just enough

so that the room doesn't look cluttered in the long run.

Chaos subsides. Annotating with size helps with this.

So first we need a variant of the data definition for search trees with size-annotated trees:

```
A size-annotated search tree consists of
; - Function for =
; - Function for <
; - size-annotated binary tree
(define-record (sized-search-tree-of element)
  make-sized-search-tree sized-search-tree?)
```

```
(sized-search-tree-label=?-function (element element -> boolean))
(sized-search-tree-label<?-function (element element -> boolean)) (sized-search-tree-tree (sized-tree-of
false element)))
```

Here is an example:

```
(define sized-search-tree1
  (make-sized-search-tree
    string=? string<?
    (make-sized node "M"
      (make-sized node "B"
        (make-sized-node "A" #f #f)
        (make-sized-node "D" #f #f))
      (make-sized node "O"
        (make-sized-node "N" #f #f)
        (make-sized-node "R" #f #f)))))
```

Task 12.11

Why do we need to define a new record type? Could n't we define `sized-search-tree-of` like this?

```
(define sized-search-tree-of
  (lambda (element)
    (search-tree-of (sized-label-of element))))
```

Ÿ

We're changing `search-tree-member?` to `sized-search-tree-member?`. To do this, we copy it, rename it, and modify it so that it uses `sized-search-tree`.

```
; determine if the element is present in the search tree
(: sized-search-tree-member?
  (%a (sized-search-tree-of %a) -> boolean))
```

```
(check-expect (sized-search-tree-member? "M" sized-search-tree1) #t) (check-expect (sized-search-tree-
member? "D" sized-search-tree1) #t) (check-expect (sized-search-tree-member? "N" sized-search-tree1)
#t) (check-expect (sized-search-tree-member? "R" sized-search-tree1) #t) (check-expect (sized-search-tree-
member? "Z" sized-search-tree1) #f)
```

```

(define sized-search-tree-member? (lambda (value
  search-tree)
  (define label=? (sized-search-tree-label=?-function search-tree))
  (define label<? (sized-search-tree-label-<?-function search-tree)) (define tree-member?

  (lambda (value tree)
    (cond
      ((node? tree)
       (define label (sized-node-label tree))
       (cond
         ((label=? value label) #t)
         ((label<? value label)
          (tree-member? value (node-left-branch tree))) (else
          (tree-member? value (node-right-branch tree))))))
      (else #f)))) (tree-
member? value (sized-search-tree-tree search-tree))))

```

Compare it to [search-tree-member?](#) There are only two differences:

- The function uses size-annotated search trees and trees instead of the "normal" ones.
- The function calls [sized-node-label](#) instead of [node-label](#).

Now we will turn to the construction of balanced search trees. The function for this is-

The following is created from [search-tree-insert](#) by switching to [sized-search-tree](#):

[; insert new element into size-annotated search tree](#) (: [balanced-search-tree-insert](#)

```

(%a (sized-search-tree-of %a) -> (sized-search-tree-of %a))

```

```

(define balanced-search-tree-insert
  (lambda (value search-tree)
    (define label=? (sized-search-tree-label=?-function search-tree)) (define label<? (sized-search-tree-
label-<?-function search-tree))
    (define tree-insert
      (lambda (value tree) (cond
        ((node? tree)

```

```

(cond
  ((label=? value (sized-node-label tree))
   tree)
  ((label<? value (sized-node-label tree))
   (make-balanced-node (sized-
     node-label tree)
     (tree-insert value (node-left-branch tree))
     (node-right-branch tree))) (else
    (make-balanced-node
      (sized-node-label tree) (node-left-
        branch tree)
      (tree-insert value (node-right-branch tree))))))
(else
  (make-sized-node value #f #f))))
(make-sized-search-tree
  label=? label<?
  (tree-insert value (sized-search-tree-tree search-tree))))

```

For the construction of a new node, the function does not use `make-node` (or `make-sized-node`), but a function yet to be written called `make-balanced-node`.

She then has to take care of the "little bit of tidying up" described above by balancing-

mern. It has the same signature as `make-sized node`:

```

; create a new knot, thereby rebalancing
(: make-balanced-node (%label (sized-tree-of false %label)
                           (sized-tree-of false %label)
                           -> (sized-node-of false %label)))

```

Here is the framework:

```

(define make-balanced-node
  (lambda (label left-branch right-branch)
    ...))

```

We could take the easy way out and write:

```

(define make-balanced-node
  (lambda (label left-branch right-branch) (make-sized-node
    label left-branch right-branch)))

```

This, however, corresponds to the strategy of a certain 12-year-old child of one of the authors: simply throw the new element onto the bed, close the door, and that's it. But the idea is: if the disorder is too great, we want to tidy up a bit. Here, disorder means "imbalance between the left branch and the right branch." And we define this imbalance (more precisely, Stephen Adams does) by the size of the subtrees. Naively, we could do it like this:

```
(define make-balanced-node
  (lambda (label left-branch right-branch)
    (cond
      ((= (sized-tree-size left-branch)
          (sized-tree-size right-branch))
       (make-sized-node label left-branch right-branch)) (else ...)))) ; balance
```

...but that would be too harsh: It would mean that the function would have to rebalance *every time* even the slightest imbalance arose. Besides, that's impossible: A tree of size 3, for example, cannot be perfectly balanced.

So we need a "softer" criterion. Stephen Adams had the following idea: If

If one subtree is more than n times as large as the other, then the balance is re-established.

How did he arrive at this, and what should n be? How he arrived at it is not recorded, but we assume he tried different criteria, and this one resulted in a function that is both simple and, on average, fast. In doing so, he also determined what a sensible size for n should be, namely 5. The following template is created:

```
(define ratio 5)

(define make-balanced-node
  (lambda (label left-branch right-branch)
    (define left-size (sized-tree-size left-branch)) (define right-size (sized-tree-
      size right-branch))
    (cond
      ((> right-size ; right side is overweight (* ratio left-size)) ...)
      ((> left-size ; left side is overweight (*
        ratio right-size)) ...) (else

      (make-sized-node label left-branch right-branch))))))
```

As you can see, we've moved the 5 into a separate definition because it's used twice and can therefore be easily changed. Adams improved the function even further by adding another criterion that doesn't require balancing:

```
(define make-balanced-node
  (lambda (label left-branch right-branch)
    (define left-size (sized-tree-size left-branch))
    (define right-size (sized-tree-size right-branch)) (cond

      ((< (+ left-size right-size) 2)
        (make-sized-node label left-branch right-branch))
      ((> right-size ; right side is overweight)
        (* ratio left-size) ...)
      ((> left-size ; links are overweight (* ratio right-size)) ...)

      (else
        (make-sized-node label left-branch right-branch))))))
```

The first branch always comes into play when there is only one node in the two sub-trees together: then balancing makes no sense.

12.8 BALANCE THROUGH ROTATION

To balance the tree, let's look at what such an imbalance might look like. In Figure 12.5, the tree in the upper left is labeled a and c, with triangular subtrees X, Y, and Z.

Now imagine that the tree has significantly more nodes on the right than on the left. It is then out of balance and should be rebalanced. This is accomplished by a *rotation*, which transforms this tree into the tree shown in the upper right: Subtree Y is shifted from right to left, thus alleviating the imbalance, at least somewhat.

In the lower left, you can also see a tree that has become unbalanced: Here, too, a subtree is shifted from right to left, but this one is located one level deeper. This rotation is somewhat more complicated, which is why it is called a "double" rotation; the first one is accordingly called a "single" rotation.

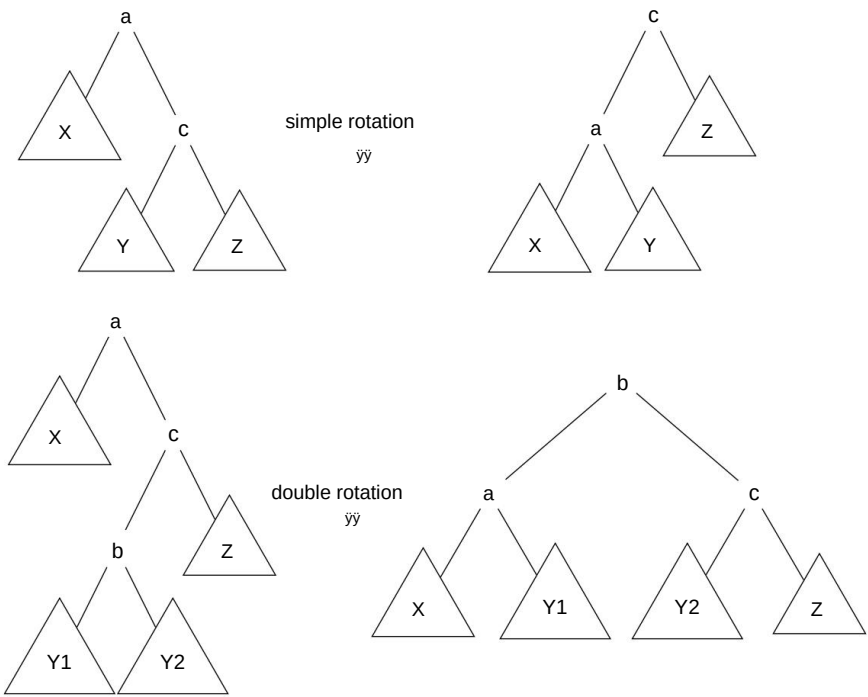


Figure 12.5: Balancing trees by rotation

Task 12.12

Why do the rotations in Figure 12.5 not cause the trees on the right side to be no longer sorted?

When should the algorithm rotate once and when twice? Take another look at this.

Figure 12.5 shows the upper left tree. Of the subtrees X and Y, one is larger than the other one. Let's assume that's X. Then a simple rotation reduces the imbalance between left and right. But if Y is greater than Z, then the simple rotation The imbalance is greater: The already larger block Y then joins forces with X, while the smaller block Z is left all alone on the right. In this case, the double Rotation displayed, showing only a subtree of Y, namely Y1, shifted to the left.

Task 12.13

Take a piece of paper and draw the two rotations from 12.5 so that the left side is predominant in each case.

We now transfer these rotations into the body of `make-balanced-node`. To avoid errors, we use the same names in the code as shown in Figure 12.5:

```
(define make-balanced-node
  (lambda (label left-branch right-branch)
    (define left-size (sized-tree-size left-branch)) (define right-size (sized-tree-size
      right-branch)) (cond

      ((< (+ left-size right-size) 2)
        (make-sized-node label left-branch right-branch)) ((> right-size (* ratio left-size))
        (define a label) (define X left-branch)

        (define c (sized-node-label right-branch)) (define Y (node-left-branch
          right-branch)) (define Z (node-right-branch right-branch)) ... ...)))
```

Task 12.14

The definitions of `c`, `Y`, and `Y` don't actually check whether `right-branch` is really a node. Couldn't it also be a leaf? `y`

Next comes the rotation: The function still needs to decide whether to perform a single or a double rotation. This depends on whether `Y` or `Z` is larger:

```
(if (< (sized-tree-size Y) (sized-tree-size Z))
    ... ; single ... ; double
```

For simple rotation, we need to translate the right side of Figure 12.5 into code:

```
(if (< (sized-tree-size Y) (sized-tree-size Z))
    (make-sized node c
                    (make-sized-node a XY
                                      Z)
    ...)
```

In the diagram, the double rotation shows further names for the subtrees of Y.

It makes sense to bind them as well. Unfortunately, ``if`` doesn't allow binding local variables; we have to use a ``cond`` instead.

```
(cond
  ((< (sized-tree-size Y) (sized-tree-size Z))
    (make-sized node c
      (make-sized-node a XY)
      Z))
  (else
    (define b (sized-node-label Y))
    (define Y1 (node-left-branch Y)) (define Y2 (node-
      right-branch Y))

    (make-sized node b
      (make-sized-node a X Y1)
      (make-sized-node c Y2 Z))))
```

Task 12.15

In the double rotation, the function assumes that Y is a node. Couldn't it also be a leaf? $\dot{y}$

We now need the function to handle the reverse case, where the preponderance is on the left and nothing is on the right. The code for this is "mirrored" compared to the case just discussed.

The last case is still missing: This occurs when the tree does not exhibit excessive imbalance. In that case, we could simply use `make-sized node`.

Here's the full function:

```
(define make-balanced-node
  (lambda (label left-branch right-branch)
    (define left-size (sized-tree-size left-branch)) (define right-size (sized-tree-size
      right-branch)) (cond

      ((< (+ left-size right-size) 2) (make-sized-node
        label left-branch right-branch)) (> right-size (* ratio left-size))
```

```
(define a label)
```

```
(define
```

```
((< (sized-tree-size Y) (sized-tree-size Z))
```

```
  (make-sized node c
```

```
    (make-sized-node a XY) Z))
```

```
(else
```

```
  (define b (sized-node-label Y)) (define Y1
```

```
    (node-left-branch Y))
```

```
  (define Y2 (node-right-branch Y)) (make-sized-  
    node b
```

```
    (make-sized-node a X Y1)
```

```
    (make-sized-node c Y2 Z))))
```

```
((> left-size (* ratio right-size)) (define c label)
```

```
(define a (sized-node-label left-branch))
```

```
(define
```

```
(make-sized node a
```

```
  X (make-sized-node c YZ)))
```

```
(else
```

```
  (define b (sized-node-label Y)) (define Y1
```

```
    (node-left-branch Y))
```

```
  (define Y2 (node-right-branch Y)) (make-sized-  
    node b
```

```
    (make-sized-node a X Y1)
```

```
    (make-sized-node c Y2 Z))))
```

```
(else
```

```
  (make-sized-node label left-branch right-branch))))
```

There are currently no options for [balanced-search-tree-insert](#) and [make-balanced-node](#). Tests. This is concerning, especially because the functions are quite complex and many Cases must be covered. It is worth recalling Mantra 7 on page 46 once again :

Mantra 7

If your function seems complicated to you, it probably contains errors: either because it should actually be simpler, or because the task is complicated and therefore its solution is too. is prone to errors.

We will be really testing [balanced-search-tree-insert](#) and [make-balanced-node](#). Only in the next chapter. Here's a preparation exercise:

Task 12.16

Start thinking about *what* actually needs to be tested. Is it enough, To test the function in the same way as [search-tree-insert](#), i.e., only to test whether inserted elements Are they actually in the search tree afterwards? ÿ

TASKS

Task 12.17

Write a function that accepts a binary tree and returns a list of all the markers in the tree. tree returns.

Task 12.18

The ``search-tree-member?`` function only checks *whether* an element exists in a search tree. This is not helpful, for example, when searching for phone numbers based on given names. Extend the functionality so that it can also be used to search for phone numbers, for example. Demonstrate a phone number search as an example!

Use *entries* as markers in the search tree; these entries consist of a *key* (for example, the name) and a *value* . Write `data`, `record`, and `signature` definitions. for entries. Modify the `search-tree-insert` function so that it uses keys and The element is accepted. Write a `search-tree-find` function that finds the value corresponding to a given key.

Task 12.19

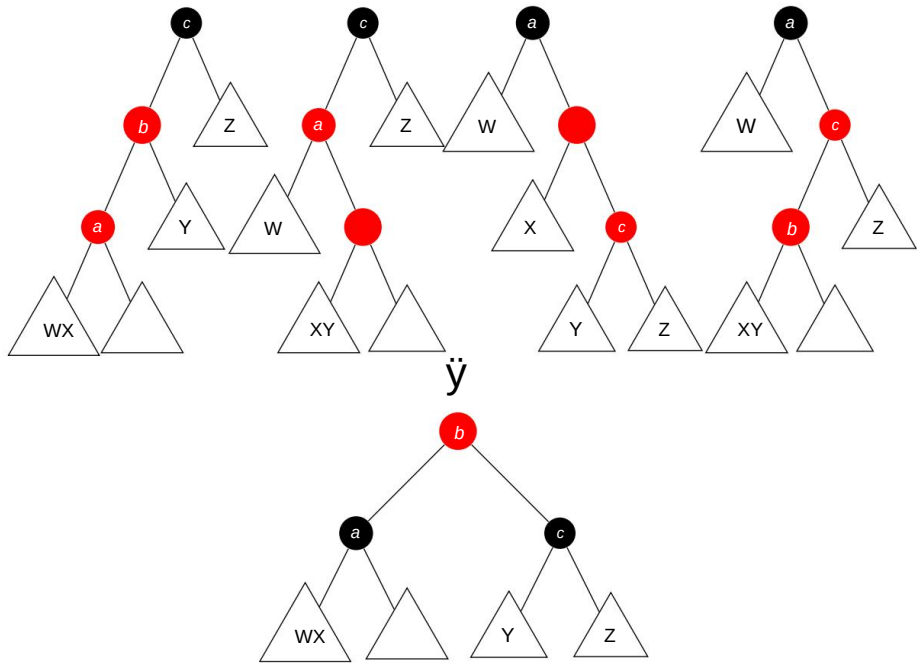


Figure 12.6: Red-Black-Trees

Figure 12.6 illustrates an alternative strategy for balancing search trees. This strategy does not use size-annotated trees, but rather trees where each node has a *color* as part of its label. Only the colors red and black are used; accordingly, these trees are called *red-black trees*. (If you are viewing the figure in black and white: the gray labels are red, the black ones are black.) Leaves are always black.

The following conditions apply to a red-black tree:

- No red node has a red subtree.

- On every path leading from the root to the leaves, the number of black

The nodes are the same.

These conditions guarantee that the longest possible path, i.e., a path where red and black nodes alternate, is less than twice as long as the shortest possible path. (The shortest possible path consists only of black nodes.)

If a red-black tree is to be extended by one node, this must be done after inserting it.

The condition still applies, so it needs to be balanced.

Program red-black trees using the following instructions!

- First, write data and record definitions for red-black trees analogous to size annotations for animals, trees, and search trees.
- Write functions `red-tree?` and `black-tree?` that check if a (sub)tree is red or is black.
- Write a function `red-black-tree-member?` analogous to `sized-tree-member?`. • Write a function `red-black-tree-insert` analogous to `balanced-search-tree-insert`.

Task 12.20

Write a function `search-tree-delete` that removes an element from a search tree.

Task 12.21

Develop an alternative representation for finite sequences based on the following data definition:

An `AList` is one of the following: - the empty `AList`

; - a single-element `A-list`; - a concatenation of two `A-lists`

Program some of the functions from Chapter 6 using this new representation, for example, `list-sum`, `concat`, `list-map`, and `list-filter`. What are the advantages and disadvantages of this representation compared to lists?

Task 12.22

Trees don't always have to be binary: It's also possible to define trees so that a node, instead of always two, carries an arbitrarily long list of subtrees. The resulting trees are called "rose trees."

1. Write a data definition for rose trees and translate it into code!
2. Do you need a separate definition for leaves?
3. One can use rose trees to program a special form of search tree.

ren, where the entries are lists.

The search tree has a path for each entry, where the nodes along the path are the elements of the list. The marker at the root is ignored.

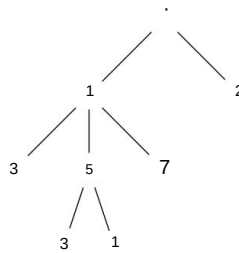


Figure 12.7: Rose search tree

Figure 12.7 shows an example of a rose search tree containing the lists `#list<1 3>`, `#list<1 5 3>`, `#list<1 5 1>`, `#list<1 7>` and `#list<2>`.

Write functions analogous to `search-tree-insert` and `search-tree-member` for rose search trees!

13 PROPERTIES OF FUNCTIONS

How do we actually know that the function we've written works correctly? If we work with construction instructions, the chances aren't bad. Systematic construction helps to write the function correctly from the outset. But verification is better: Tests help to find any errors. Unfortunately, this doesn't always work because each test only checks a single example. At least, that's been the case so far. In this chapter, we'll change that by formulating general *properties* of functions instead of individual examples. From these properties, we can automatically create tests that are more effective than the previous `check-expect`-based tests. For absolute certainty, we can occasionally also prove properties mathematically. This chapter shows how.

13.1 ACCURACY AND TESTS

`properties/heat-water.rkt`

code

Do you remember the `heat-water` function from section 2.5 on page 59? It had some rather complicated branching. A few errors occurred along the way. In the end, we split `heat-water` into two functions: `heat->temperature` and `temperature->heat`.

Here is the first one, where we have simplified the conditions in the branching point somewhat:

```
; Calculate temperature from heat (: heat-
>temperature (real -> real))
```

```
(define heat->temperature (lambda
  (heat) (cond
```

```
    ((<= heat 0) heat) ((<=
  heat 80) 0)
    ((<= heat 180) (- heat 80))
    (else 100))))
```

There were also quite a few test cases:

```
(check-expect (heat->temperature -50) -50) (check-expect
(heat->temperature 0) 0) (check-expect (heat-
>temperature 20) 0)
```

```
(check-expect (heat->temperature 80) 0) (check-expect
(heat->temperature 81) 1) (check-expect (heat-
>temperature 180) 100) (check-expect (heat->temperature
200) 100)
```

The test cases actually have two tasks:

1. You should document the function using examples .
2. They should also ensure that the function is programmed *correctly* .

However, this feature has problems in both aspects. There are so many test cases, and they are arbitrarily and seemingly randomly selected. This makes it difficult for readers to understand the underlying principle. As a result, we (and perhaps you too) can't shake the feeling that errors might still slip through the test cases.

Task 13.1

Intentionally modify the function so that it contains a bug, but in such a way that all test cases still run successfully.

A [check-expect test](#) is unfortunately always just a single example, which limits its explanatory power.¹ Often, a general statement is better documentation.

In this case, we could, for example, state that it's about water, so the temperature can never exceed 100°C . This can be expressed not only as text, but also as a piece of code:

```
(<= (heat->temperature heat) 100)
```

On its own, this expression makes no sense: `heat` is not bound. We need to add that the statement applies to *all* values of `heat` . Strictly speaking, not even to absolutely *all* values, only to all values of the signature `real` . We can actually write this down using a new form called `for-all` :

```
(for-all ((heat real)) (<= (heat-
>temperature heat) 100))
```

The `"for-all"` part, as the name suggests, means "for everyone". It says:

For all numbers called heat, the result of `(heat->temperature heat)` must be less than or equal to 100.

¹ In professional development, such a test is called a test *unit test*.

The `for-all` function allows the formulation of *properties*. A for-all expression has the following general form:

```
(for-all ((var 1 sig1) . . . (var n sign)) b)
```

The ``var i`` must be variables, the ``_i`` signatures, and ``b`` (the body) an expression that returns either a Boolean value or a property. The ``for-all`` expression takes as its value a property stating that ``b`` holds for *all* values of ``var i`` that satisfy the signatures ``sigi``.

Figure 13.1: `for-all`

"Why are there double brackets around `'heat real'`?" you might wonder. This is because the outer pair of brackets can contain multiple variables, each with its signature in an inner pair of brackets. Examples of this will follow. Figure 13.1 describes in more detail how for-all expressions are generally structured.

The result of the for-all expression is displayed somewhat unclearly in the REPL:

```
(for-all ((heat real))
  (<= (heat->temperature heat) 100))
ȳ #<:property>
```

In German, «property» means «Eigenschaft», because it is a property of the function `heat-temperature`.

This feature does not (always) replace unit tests, but it is a valuable addition to the documentation.

It can also help ensure accuracy. To do this, we wrap it up.

The property is also a `check property`:

```
(check-property
  (for-all ((heat real)) (<= (heat-
    >temperature heat) 100)))
```

`Check-property`, like `check-expect` or `check-within`, creates a test case.

Since ``heat->temperature`` is programmed correctly, ``check-property`` only reports a passed test. The true capabilities of ``check-property`` only become apparent when the function encounters an error. For example, if we change 180 to 280, the following message appears:

Property falsifiable with heat = 206



Important: If you try this out yourself, the specific number may be different.

`Check-property` tests a property similarly to `Check-Expect`. A `Check-property` form looks like this:

```
(check-property e)
```

`e` is an expression that must provide a property – usually a for-all expression.

The form then tests this property. (More on this in the next section.)

The `Check-property` function only works for properties where values can be meaningfully generated from the signatures. This is the case for most signatures, but not for `any` and for signature variables that have been defined.

Figure 13.2: `check-property`

The word "falsifiable" is quite a strong one: It means that Racket has found a *counterexample* to the property. We can verify this:

```
(heat->temperature 206)
```

```
ȳ 126
```

...and 126 is greater than 100. The counterexample found by the `check-property` test could, if we hadn't made the mistake intentionally, help us find and eliminate it.

Figure 13.2 describes the structure of `check-property` in detail.

Task 13.2

Intentionally introduce a few more errors in `heat->temperature`. Which of these will be detected by the `check-property` test and which will not? ȳ

Since `check-property` tests a property, this technique is also called *property-based testing*. We will apply these to other functions later in this chapter.

13.2 HOW CHECK-PROPERTY WORKS

But first, let's take a brief detour: What happens during a `check-property` test?

It would be great, of course, if he could say with certainty what the property means: that, for example, the temperature is truly at most 100 for *all* inputs. That is generally

Unfortunately, this is impossible.² In some cases, however, it is possible to prove properties of functions formally. (More on this in section 13.9 on page 432.)

`Check-property` cannot prove that a property is true. Instead, it performs sampling: it randomly selects values for the signatures and repeats this process to create many individual tests from a single test case – typically 100.

The use of the term "random" in this context is common in computer science, but a better word would be "chaotic." In fact, `check-property` produces the same tests every time the program is run.

The technique of *property-based testing*, i.e., first formulating general properties and then generating test cases for them, was originally published under the name *QuickCheck* [CH00] and was a major breakthrough in the effectiveness of automated tests.

13.3 MORE FEATURES AND INEXACT NUMBERS

Let's continue with `heat-water`: `Heat->temperature` is just a helper function for it, along with `temperature->heat`. Here too, we could write down a property that tells us something about the range of numbers that comes out of the function.

However, there is an even more valuable property: `temperature->heat` should be the reverse of `heat->temperature`. From this, we can derive the following property or test:

```
(check-property (for-
  all ((temp real))
    (= (heat->temperature (temperature->heat temp))
      temp)))
```

This states that if a temperature is converted into heat and then back into a temperature, the result should be the same. Unfortunately, the test fails.

Property falsifiable with `temp = 0`



We can substitute 0 for `temp` in the REPL :

```
(heat->temperature (temperature->heat 0)) ÿ cond: all
conditions resulted in #f
```

² That this is impossible has been mathematically proven and recorded as *Rice's theorem*. It is a topic in theoretical computer science.

The form is defined in one property.

`(==> pe)`

for the property `e` to only hold if the expression `p` yields the value `#t`.

For a `==>` form, `check-property` only generates tests where `p` returns `#t`.

Figure 13.3: Requirements for properties

There was something about that, wasn't there? Perhaps you remember: A temperature of 0 °C cannot be uniquely assigned to a heat value, which can range from 0 to 80. That's why `heat->temperature` refuses to produce a result for the input 0. So we need to adjust our test so that it says "for all real numbers *except* 0".

Here's how it works:

```
(check-property (for-
  all ((temp real))
    (==> (not (= temp 0)) (=
      (heat->temperature (temperature->heat temp))
      temp))))
```

The arrow `==>` is new and only works in the context of a property: it means "holds under the condition". Figure 13.3 explains in detail how the form works. Here, it states that the equation only holds under the condition that `temp` is not 0.

Note: You could also write the property above using `if` instead of `==>`:

```
(check-property (for-
  all ((temp real)) (if (= temp
    0)
    #t
    (= (heat->temperature (temperature->heat temp))
      temp))))
```

The property would mean the same thing. However, `check-property` handles this notation differently, namely worse. If `check-property` generates 100 tests, then all those where `temp` is 0 are counted as successes, even though nothing is actually being tested there. If, hypothetically, `temp` is 0 for three of the tests, then only 97 correct tests are performed.

In the variant with `==>`, `check-property` ensures that 100 tests are actually performed where `temp` is not 0. This results in more tests being conducted.

However, when we run the test case, there is a strange surprise:

Property falsifiable with temp = #i24.571428571428573

As already mentioned, the specific number may look different, but it's about the strange `#i`. This stands for "inexact" because it is a so-called *inexact number*. Such numbers are used by DrRacket (and virtually all other programming languages) to represent the results of calculations that cannot be performed with absolute precision (at least not without reasonable effort).

Up until now, everything in this book has been perfectly precise because our programs have internally used exact fraction arithmetic. To calculate such an inexact number, you can try, for example, this in the REPL to calculate the square root of 2:

```
(sqrt 2)
```

```
ȳ #i1.4142135623730951
```

The square root of 2 has infinitely many decimal places, which is why Racket only calculates and rounds a few of them. And so that we and you know that rounding has occurred, the `#i` is displayed before it.

The rounding process is even more complicated than it seems: it involves *binary* calculations. The exact procedure is quite intricate and would fill another book. More information on this topic can be found, for example, in David Goldberg's standard work [Gol91].

But what exactly happened to our property? We can determine the `#i` number of Insert hand into the property and evaluate in the REPL:

```
(heat->temperature (temperature->heat #i24.571428571428573))
```

```
ȳ #i24.57142857142857
```

You can see that rounding obviously occurred during the calculation, specifically at the last decimal place. This might seem strange to you because `temperature->heat` only involves addition and subtraction – there's no sign of "rounding" there. If we remove the `#i`, the calculation is exact:³

```
(heat->temperature (temperature->heat 24.571428571428573))
```

```
ȳ 24.571428571428573
```

³ If you ever encounter one of the other languages included in the Racket system: In most of them, unlike here, every number with a decimal point is treated as inexact.

This is because "inexactness" is contagious: if even one input is inexact when calling a calculation function such as `+` or `*`, rounding will occur.

Incidentally, because this rounding is sometimes surprising, `check-expect` refuses to compare inexact numbers.

Task 13.3

Try it out: `(check-expect #i1 #i1)`

Ÿ

In our property, we specified the signature `real` for `temp`: This signature also contains inexact numbers, which is where the problem begins. We can approach it in two ways: • We replace the signature `real`

in the property with `rational`. `rational` contains no inexact numbers. However, this has the disadvantage that it only tests for exact numbers, even though the functions also work for inexact numbers. • We take the effect of rounding into account by changing the condition in the property.
soften it a little.

We do the latter and only require that the difference between the actual and expected result does not exceed a certain amount:

```
(check-property (for-
  all ((temp real))
    (=> (not (= temp 0)) (<= (abs
      (- (heat->temperature (temperature->heat temp))
        temp))
        0.0000001))))
```

As a reminder: The built-in function `abs` calculates the absolute value, i.e., it reverses the sign for negative numbers, see section 11.8 on page 346.

Unfortunately, the test still fails, for example with the following output:

Property falsifiable with temp = 105

We can try this out in the REPL:

```
(heat->temperature (temperature->heat 105)) cond: all
conditions resulted in #f
```

This is because `temperature->heat` only works for temperatures up to 100°C: water can't get any hotter. So we need to extend our precondition:

```
(check-property
  (for-all ((temp real))
    (==> (and (not (= temp 0))
              (< temp 100))
          (<= (abs
                (- (heat->temperature (temperature->heat temp))
                  temp))
              0.0000001))))
```

This text illustrates quite well how `heat->temperature` and `temperature->heat` are related: one reverses the other, at least approximately. In mathematics, this is said to mean that... a function is the *inverse* of another function.

Task 13.4

Try to trick the final `check-property test`. Instead of introducing small errors, Try using completely different functions that have nothing to do with water temperatures. but still possess the above property. `ȳ`

This task demonstrates that properties are no guarantee of correctness: Just like with unit tests. It often takes several of these or additional `check-expect tests` to ensure sufficient safety.

to take care of it.

The technique from the task is helpful here: Consider how you can test a test case using false You can trick functions. Then add test cases that detect these errors.

13.4 RELATIONAL PROBLEMS

`properties/sort.rkt`

code

We could have replaced the properties for `heat-water` with a (long) series of `check-expect` tests. The properties are helpful, but not essential. But there are Functions for which unit tests are fundamentally unsuitable, namely those that solve so-called *relational problems*. These are the topics of this section.

We will first write such a function to explain the concept: It is designed to sort the members of a band by age.

Here are the data and record definitions for a band member:

```
(define-record band-member
  make-band-member
  (band-member-name string)
  (band-member-born natural))
```

And here is the specific band (as of 2021):

```
(define axl (make-band-member "Axl Rose" 1962))
(define duff (make-band-member "Duff McKagan" 1964)) (define slash (make-
band-member "Slash" 1965))
(define dizzy (make-band-member "Dizzy Reed" 1963)) (define richard (make-
band-member "Richard Fortus" 1966)) (define frank (make-band-member "Frank Ferrer"
1966))
(define melissa (make-band-member "Melissa Reese" 1990))
```

```
(define guns-n-roses (list axl
  duff slash dizzy richard frank melissa))
```

We'll do this with a simple, albeit inefficient, method *called insertion sort*: The sorting function uses a sorted list as an accumulator. This list is initially empty, and the function adds one element at a time from the input list by inserting it at the correct position. We'll first write a helper function to insert an element. We'll postpone testing until we're ready to test the sorting function. Short description, signature, framework, and template:

; Insert band member into a sorted list (: insert

```
(band-member (list-of band-member) -> (list-of band-member)))
```

```
(define insert
  (lambda (band-member list)
    (cond
      ((empty? list) ...) ((cons? list)
      ...
      (first list) (insert
        band member (rest list))
      ...))))
```

In the first case, the function inserts an empty list: The result should then be the list with ``element`` as its only element. In the second case, it is still unclear where ``band-member`` is inserted, before or after ``first list``. The function must compare the two birth years.

```
(define insert
  (lambda (band-member list)
    (cond
      ((empty? list) (cons band-member empty)) ((cons? list)

      (if (<= (band-member-born band-member)
            (band-member-born (first list)))
          (cons band-member list)
          (cons (first list)
                (insert band member (rest list))))))))
```

Using the ``insert`` function, we will now build the ``sort-band`` function. Short description, signature, and unit test:

```
; Sort volume by age
(: sort-band ((list-of band-member) -> (list-of band-member)))
```

```
(check-expect (sort-band guns-n-roses)
               (list axl dizzy duff slash richard frank melissa))
```

Here is the template for the function – with accumulator:

```
(define sort-band (lambda
  (list0)
    ; Invariant: ...
    (define accumulate
      (lambda (list acc)
        (cond
          ((empty? list) ...)
          ((cons? list)
           (accumulate (rest list)
                       ... (first list) ... acc ...))))))
    (accumulate list0 ...)))
```

When accumulating, `list` contains the elements already seen from `list0`, in sorted order. From this we can formulate an invariant:

```

; Invariant: list contains the band members
; between list0 and list, sorted.

```

This allows us to fill in the gaps: ``acc`` is empty on the first call. If ``list`` is empty, then ``acc`` is the desired final result. The function calculates the new intermediate result using... from `insert`:

```

(define sort-band
  (lambda (list0)
    ; Invariant: list contains the band members; between list0 and list,
    ; sorted. (define accumulate

    (lambda (list acc)
      (cond
        ((empty? list) acc) ((cons?
                               list)
                              (accumulate (rest list)
                                           (insert (first list) acc)))))) (accumulate list0
empty)))

```

Done! Wait, there's still one small problem:

The actual value

```
#<list
```

```

#<record:band member "Axl Rose" 1962>
#<record:band member "Dizzy Reed" 1963>
#<record:band member "Duff McKagan" 1964>
#<record:band member "Slash" 1965>
#<record:band member "Frank Ferrer" 1966>
#<record:band member "Richard Fortus" 1966>
#<record:band member "Melissa Reese" 1990>>

```

is not the expected value

```
#<list
```

```

#<record:band member "Axl Rose" 1962>
#<record:band member "Dizzy Reed" 1963>
#<record:band member "Duff McKagan" 1964>
#<record:band member "Slash" 1965>
#<record:band member "Richard Fortus" 1966>

```

```
#<record:band member "Frank Ferrer" 1966>
#<record:band member "Melissa Reese" 1990>>.
```

Why is that? Richard Fortus and Frank Ferrer were born in the same year. The unit test assumes that Fortus is sorted before Ferrer, but `sort-band` does the exact opposite. Therefore, `sort-band` is n't wrong: there are simply multiple correct answers.

The unit test is therefore inefficient, even if we choose the order so that it doesn't fail. If we change the search function at some point, the order can change and the test will fail again, even if the function is correct.

Since `sort-band` can have more than one correct result for a given input, we speak of a "relational problem": The precise result is not fixed, only the relationship ("relation") between input and output. And to describe this, a property is the right tool. What might a meaningful property of a function that sorts look like? Well, that the output is sorted. To determine this, we write a function:

```
; Band sorted?
(: band-sorted? ((list-of band-member) -> boolean))
```

The tests allow for different sequences, as long as they are sorted:

```
(check-expect
  (band-sorted? (list axl dizzy duff slash frank richard melissa)) #t) (check-expect

  (band-sorted? (list axl dizzy duff slash richard frank melissa)) #t) (check-expect

  (band-sorted? (list dizzy axl duff slash richard frank melissa)) #f) (check-expect

  (band-sorted? (list axl dizzy duff richard slash frank melissa)) #f)
```

Here are the scaffolding and template:

```
(define band-sorted?
  (lambda (list) (cond

    ((empty? list) ...)
```

```
((cons? list)
  ... (first list) ... ... (band-
    sorted? (rest list) ...))))
```

The empty case is simple: an empty list is sorted. The cons case is a bit more complicated. To check the order, we need to compare two elements of the list, but there's only [the first list](#). So we need the second element. This only exists for lists with more than one element, which is why we need a second branch:

```
(define band-sorted?
  (lambda (list) (cond

    ((empty? list) #t) ((cons?
    list) (cond

      ((empty? (rest list)) ...) ((cons? (rest
    list))

      ...

      (first list) (first
    (rest list)))) (band-sorted?
    (rest list))

    ...))))
```

The inner empty case is the list with one element: This is always sorted. Finally, in the cons case, we can compare the first two elements:

```
(define band-sorted?
  (lambda (list) (cond

    ((empty? list) #t) ((cons?
    list) (cond

      ((empty? (rest list)) #t) ((cons? (rest
    list))

      (and (<= (band-member-born (first list))
    (band-member-born (first (rest list))))
    (band-sorted? (rest list))))))))
```

We can now use this function to write that `sort-band` always sorts lists produced:

```
(check-property (for-
  all ((list (list-of band-member)))
    (band-sorted? (sort-band list))))
```

This feature perfectly encapsulates what `sort-band` is all about: producing sorted lists. It's already a good piece of documentation.

Is it also a good test case? You might have a nagging feeling that we 've used another custom-written function to test `sort-band`, one that's almost as complicated as `sort-band` itself. What if we made a mistake in `band-sorted?`, one that prevents the property test case from detecting a bug in `sort-band`? While theoretically possible, this is unlikely and becomes even less so as we add more test cases with meaningful properties.

These properties represent a form of redundancy, analogous to making the load-bearing walls of a building slightly stronger than absolutely necessary. Whether this redundancy justifies the effort of writing a function like `band-sorted?`, which is only useful for testing, depends on the specific case: the more important the correctness of the function and the more complex it is, the greater the value of such test cases.

Nevertheless, it is possible to circumvent the above property, quite easily in fact:

```
(define sort-band
  (lambda (list0)
    (empty)))
```

That's a bit unfair, of course. But the function that tricks the test case is simpler than the correct function. So it's easier to do it wrong than right. Therefore, we should look for other properties that find such simple but incorrect solutions. For example, we could require that the output list be the same length as the input:

```
(check-property
  (for-all ((list (list-of band-member)))
    (= (length (sort-band list)) (length list))))
```

Task 13.5

The following version of `sort-band` passes the previous test cases:

```

(define sort-band
  (lambda (list) (cond

    ((empty? list) empty) ((cons? list)

      (map (lambda (band-member)
              (first list)
              list))))))

```

Can you write a test case that finds the error in this function?

Ÿ

13.5 DESIGN INSTRUCTIONS FOR TEST CASES?

Hopefully, the previous examples of test cases with properties have convinced you that writing such test cases is a good idea. But how do you actually do this with the next function you want to test? Construction guides analogous to those for functions would be great, showing how we can derive properties from the signature of the function under test. This wouldn't have worked for the functions in the previous sections of this chapter.

Nevertheless, there are a few things you can try if you're looking for qualities for a
If you want to formulate a new function:

1. Use your knowledge about the size of your problem.

Examples:

- When it comes to the temperature of water, you know that the temperature cannot be greater than 100°C.

- You know that the output of a sort function should be sorted.

2. Often, a problem involves several functions that need to be combined in a specific way.

Work together. Write down what this collaboration looks like.

Example: [Temperature->heat](#) reverses [heat->temperature](#) .

3. Try to circumvent the existing properties by using simple but faulty variations of your function. Then look for properties that prevent this circumvention.

Example: The empty list as a result of [sort-band](#) is always sorted, but still incorrect.

Section [13.7](#) on page [427](#) will further supplement this list with properties that are actually related to the signatures of the functions to be tested.

13.6 TESTING SEARCH TREES

`trees/binary-tree.rkt`

code

In section 12.7.2 on page 386, we encountered the function `balanced-search-tree-insert`, which was quite complicated. Perhaps you felt the same way we did – we had a slightly uneasy feeling about whether the function was actually correct. It has quite a few branches, not only within the function itself but also in the helper function `make-balanced-node`. The test cases

It is quite unlikely that every possible form of search tree could be tested.

Another difficulty in testing `balanced-search-tree-insert` is that the function solves a relational problem: there is more than one correct result. In a test case using `check-expect`, which expects a specific result from `balanced-search-tree-insert`, the function might return a different, but still correct, result. The test case then fails, and finding the cause is tedious.

But with a few features, we can (hopefully) eliminate that uneasy feeling and increase our confidence in the function. The previous section offered some suggestions on how we can proceed. The first suggestion was:

Use your knowledge about the size of your problem.

The size of our problem here is the balanced search tree. The term itself already contains a considerable amount of knowledge:

1. The result of `balanced-search-tree-insert` should be a sorted tree where all tags in the left subtree of a node should be smaller than the tag of the node.
2. The trees are size-annotated: The size annotation should also be correct, i.e., for every tree.
The nodes reflect the actual size of the tree below.
3. Finally, the balanced search tree should of course also be *balanced*.

We'll start with the second point and then leave the first one for you as an exercise. To check if the size annotation is correct, we need to examine the annotation of each node in the tree. For this, we need a helper function. It has the following short description and signature:

`; Is the size annotation on the search tree correct? (: proper-sized-search-tree? ((sized-search-tree-of %a) -> boolean))`

Here are two simple test cases for this:

`(check-expect (proper-sized-search-tree? sized-search-tree1) #t)`

```
(check-expect (proper-sized-search-tree?
  (make-sized-search-tree
    = <
    (make-sized node
      5
      (make-node (make-sized-label 5 3) #f #f)
      (make-node (make-sized-label 2 7) #f #f))))
  #f)
```

"Just two measly test cases," you might object, "more would certainly be better." However, we'll be calling them in `check-property tests` along with `balanced-search-tree-insert`, and we hope that this will provide the necessary redundancy and increase our confidence in the correctness of `balanced-search-tree-insert`. But is this hope justified?

We can imagine two scenarios in which this is not the case, i.e., the test case does not find any errors, even though some exist:

- If a `proper-sized search tree` *always* returns `#t`, then every test case using it is useless. However, we ruled that out using the one `check-expect test case`.
- The ``proper-sized-search-tree`` function happens to be so flawed that it accepts incorrect results from ``balanced-search-tree-insert`` as correct. However, ``proper-sized-search-tree`` is much simpler than ``balanced-search-tree-insert``, and errors are less likely there. By backing up one function with the other, we have increased the redundancy in the overall system.

Here is the framework for the function:

```
(define proper-sized-search-tree?
  (lambda (search-tree) ...))
```

We proceed with the trunk in the same way as with the function `sized-search-tree-member?` on page 387. The actual tree is contained in the `search-tree record`. We extract this and let an internal helper function do the actual work.

```
(define proper-sized-search-tree?
  (lambda (search-tree)
    (define proper?
      (lambda (tree) ...))

    (proper? (sized-search-tree-tree search-tree))))
```

Here is the template:

```
(define proper-sized-search-tree?
  (lambda (search-tree)
    (define proper?
      (lambda (tree) (cond

        ((node? tree)
         ...

         (proper? (node-left-branch tree)) (proper? (node-
           right-branch tree)) (node-label tree) ...))

        (else ...))))

    (proper? (sized-search-tree-tree search-tree))))
```

We can already fill in a few gaps: For a leaf, the size annotation is always correct because it's not present. For a node, a size annotation is correct if it's the sum of the sizes of the subtrees plus 1 (for the node itself). The function also needs to check the size annotations of the subtrees; this is handled by the recursive calls from the template. The finished result looks like this:

```
(define proper-sized-search-tree?
  (lambda (search-tree)
    (define proper?
      (lambda (tree)
        (cond
          ((node? tree)
           (define left (node-left-branch tree))
           (define right (node-right-branch tree)) (and (proper? left)
             (proper? right) (= (sized-
               label-size (node-label
                 tree))
                 (+1)
                 (sized-tree-size left)
                 (sized-tree-size right))))) (else #t))))

    (proper? (sized-search-tree-tree search-tree))))
```

But we had written the function to test ``unbalanced-search-tree-insert``. If it's correct, then it only produces trees where ``proper-sized-search-tree?`` returns `#t` as a result. We might be tempted to write something like this:

```
(check-property
  (for-all ((search-tree (sized-search-tree-of natural))
            (element natural)
            (proper-sized search tree?)
            (balanced-search-tree-insert element search-tree))))
```

This doesn't work, however, because the signature `(sized-search-tree-of natural)` is not suitable for generating search trees. Specifically, the signature only states that the size annotation is a natural number, but not which one: for most search trees generated this way, it would be incorrect. And if the input for `balanced-search-tree-insert` is already incorrect...

Given this, we cannot expect the output to be correct: Garbage in, Garbage out.

Task 13.6

Other aspects of the signature `(sized-search-tree-of natural)` make it unsuitable for generating correct search trees. Find one! ÿ

Instead, we repeatedly call ``balanced-search-tree-insert`` to generate correctly size-notated search trees.

We retrieve the elements for this from a list and write...

Accordingly, a function that turns a list into a search tree:

```
Create a search tree from all elements of a list
(: list->balanced-search-tree ((%a %a -> boolean) (%a %a -> boolean)
                               (list-of %a)
                               -> (sized-search-tree-of %a)))
```

This is a typical task for `fold`: Start with an empty search tree (for whose construction we need the functions for comparing the markers) and then for each Call up the list item `balanced-search-tree-insert`:

```
(define list->balanced-search-tree
  (lambda (label=? label<? elements)
    (fold (make-empty-sized-search-tree label=? label<?)
          balanced-search-tree-insert
          elements)))
```

And with the help of this function, we can finally write the test that ensures that the `balanced-search-tree-insert` function correctly handles size annotation:

```
(check-property
  (for-all ((elements (list-of natural)))
    (proper-sized search tree?)
    (list->balanced-search-tree = < elements))))
```

Task 13.7

Write a function analogous to `property-sized-search-tree` that determines whether a search tree is actually sorted, and make this a property and a `check-property` test case for `balanced-search-tree-insert`!

From the list at the beginning of this section, we have checked off two items. One is still missing, namely to test whether `balanced-search-tree-insert` actually delivers trees that are balanced, as the name suggests.

Let's assume that `balanced-search-tree-insert` does *n't* balance correctly.

That would be quite tricky, because the tests only test the search tree property, not its balance.

Problems only arise when programs have to deal with larger amounts of data.

because the search time in the tree is longer than expected.

What exactly does "balanced" mean? The `balanced-search-tree-insert` function uses a "soft" criterion to avoid having to clean everything up every time.

How often the function rebalances is controlled by the definition of `ratio` on page 390. Stephen

Adams, the author of the underlying paper, claims that the search trees

are always balanced in such a way that for a node whose left subtree has the size and whose

For the right subtree where size r applies, the following formula applies:

$$\frac{l}{ratio} \leq r \leq l \times ratio$$

From this formula we can create a function that checks a search tree to see if it contains a valid search tree.

it is maintained throughout the entire tree. This follows the same pattern as

at `proper-sized-search-tree?`:

Is a search tree balanced?

```
(: balanced-search-tree? ((sized-search-tree-of %a) -> boolean))
```

```

(define balanced-search-tree?
  (lambda (search-tree) (define
    balanced?
      (lambda (tree)
        (cond
          ((node? tree)
            (define left (node-left-branch tree))
            (define right (node-right-branch tree)) (define left-size
              (sized-tree-size left))
            (define right-size (sized-tree-size right)) (and (balanced? left)
              (balanced? right)
              (<= (/ left-size ratio)
                right-size (*
                  left-size ratio))))
          (else #t))))
    (balanced? (sized-search-tree search-tree))))

```

Following the same pattern as with `proper-sized-search-tree`, we can formulate a property and a `check-property` test case :

```

(check-property (for-
  all ((elements (list-of natural)))
    (balanced-search-tree?
      (list->balanced-search-tree = < elements))))

```

(So many tests are now being conducted that it will take a while for them all to go through.)

But oops, the test case failed. In our case, it produced the following counterexample:

Property falsifiable with elements = `#<list 1 1 3>`



Let's try out in the REPL what kind of search tree results from this list:

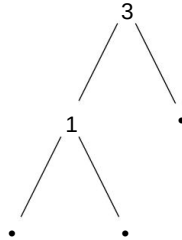
```

(list->balanced-search-tree = < (list 1 1 3))
ȳ #<record:sized-search-tree-of #<function:=> #<function:<>
  #<record:node-of #<record:sized-label-of 2 3>
    #<record:node-of #<record:sized-label-of 1 1>
      #f
    #f>
  #f>>

```

(The second 1 could have been omitted by [check-property](#) . Apparently, it's not programmed quite optimally.)

What happened? Graphically represented, the tree from the REPL looks like this:

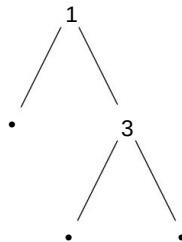


And indeed: This tree violates the equation

$$\frac{l}{\text{ratio}} \approx r \approx l \times \text{ratio}.$$

In this case, $l = 1$ and $r = 0$. However, it's not actually that badly unbalanced.

Furthermore, it is impossible to satisfy the equation with a search tree containing elements 1 and 3. The only other possibility is this one:



Here, $l = 0$ and $r = 1$, and the equation is also not valid. The reason the equation is unsatisfiable is that the tree is very small: only two elements.

This fact is taken into account in the programming of [balanced-search-tree-insert](#). Seen. Take another look at [make-balanced-node](#) on page 394. As a reminder:

```

(define make-balanced-node
  (lambda (label left-branch right-branch)
    (define left-size (sized-tree-size left-branch))

```

```
(define right-size (sized-tree-size right-branch)) (cond ((< (+ left-size right-
size) 2)

(make-sized-node label left-branch right-branch)) ...)))
```

The first branch of the `cond` function only applies when the tree is very small: In that case, no rotation or other balancing occurs. This is precisely the situation we're in. The equation holds true for all larger trees. Therefore, the wording in the paper on which the function is based is somewhat sloppy, and we need to account for it in our test case by...

```
(<= (/ left-size ratio)
right-size (* left-
size ratio))))))
```

replace by:

```
(or (< (+ left-size right-size) 2)
(<= (/ left-size ratio)
right-size (* left-
size ratio))))))
```

And lo and behold: Now the test case runs. It happens from time to time that a `check-property` test case finds counterexamples that aren't actually counterexamples because we haven't quite formulated the test condition correctly. Finding the actual error is often annoying, but sometimes we also learn something in the process that we didn't know before. (In this case: That small trees need to be handled separately.)

Task 13.8

What happens if you remove the small tree branch from `make-balanced-node`? Does any of the test cases find this out? What exactly is the cause of the displayed error?

We have now completed the list from the beginning of the section. Have the tests conducted so far sufficiently ensured that `balanced-search-tree-insert` is working correctly?

Well, we've verified that the resulting search trees have the correct form. However, we haven't yet checked that the content is correct. For this, we use the second hint from section 13.5 on page 416:

Often, a problem involves several functions that work together in a specific way. Write down what this collaboration looks like.

The function `balanced-search-tree-insert` is only useful insofar as the inserted elements can also be found using `sized-search-tree-member`. This is precisely the desired interplay of several functions. For this purpose, we again use `list->balanced-search-tree` to create a search tree from a list and verify that `sized-search-tree-member` returns `#t` for every element in the list.

```
(check-property (for-all
  ((elements (list-of natural)))
  (define search-tree (list->balanced-search-tree = < elements))
  (every? (lambda (element)
    (sized-search-tree-member? element search-tree)
    elements))))
```

We programmed the function `"every?"` specifically for this test case:

```
; Returns a function for all elements of a list #t? (: every? ((%a -> boolean) (list-of %a) ->
boolean))
```

```
(check-expect (every? even? (list 2 4 6)) #t) (check-expect (every?
positive? (list 1 2 3)) #t) (check-expect (every? positive? (list 1 0 3)) #f)
```

```
(define every?
  (lambda (p? list)
    (cond
      ((empty? list) #t) ((cons?
list)
      (and (p? (first list)) (every? p?
(rest list)))))))
```

That's good, but do you remember the third clue in section 13.5 on page 416?

Try to trick the existing properties by using simple but faulty variations of your function.

Task 13.9

Find a simple but faulty variant of the `balanced-search-tree-insert` function that tricks all previous test cases!

We definitely need a function that checks that values *not* in a search tree are indeed *not* found. We'll start as before:

```
(check-property (for-all
  ((elements (list-of natural)))
  ...
  (list->balanced-search-tree = < elements)
  ...))
```

We now need some number that we can check is not in the search tree. The test case also retrieves this using a `for-all` loop:

```
(check-property
  (for-all ((elements (list-of natural))
    (number natural))
    ...
    (not (sized-search-tree-member?
      number
      (list->balanced-search-tree = < elements)))
    ...))
```

To ensure that `number` is not accidentally included in the search tree, we use `=>`:

```
(check-property
  (for-all ((elements (list-of natural))
    (number natural))
    (=> (not (member? = number elements)) (not (sized-
      search-tree-member?
        number
        (list->balanced-search-tree = < elements))))))
```

In colloquial terms, it says:

For all lists of elements and each number, the following applies: If `number` is not a member of `elements`, then `sized-search-tree-member?` returns the result `#f` for `number` and the search tree of the numbers in `elements`. With this property, we have sufficiently described the interaction between `balanced-search-tree-insert` and `sized-search-tree-member?` and can rest easy knowing that the functions are working correctly.

13.7 ALGEBRAIC PROPERTIES

Have you seen this before?

$$(a + b) + c = a + (b + c)$$

Or this?

$$(a \times b) \times c = a \times (b \times c)$$

These equations are known as *associativity*, sometimes also as *the associative law*. The word appeared briefly on page 232, where the following equation is given:

$$\text{cat}(u, \text{cat}(v, w)) = \text{cat}(\text{cat}(u, v), w).$$

Associativity is an extremely useful equation because it states that in a series of + or × operations, the parentheses are completely irrelevant: And if they are irrelevant, they can be omitted.

Associativity is not simply an equation (in this case, there are already three), but a property of the functions +, ×, and cat. Accordingly, one can say "+ is associative" or "cat is associative." All these functions share the structure of their signatures.

Here are these signatures, as if they were in a program:

```
(: + (number number -> number))
(: * (number number -> number)) (:
concatenate ((list-of %a) (list-of %a) -> (list-of %a)))
```

(Instead of the mathematical function cat, we have listed the function `concatenate` from section 6.6.3 on page 190.)

Task 13.10

Are ``and`` and ``or`` associative? How do their signatures match those of the functions above? ÿ

We have already seen this type of signature when discussing combinators, in Chapter 5 on page 141. An associative function is therefore a special type of combinator.

There are so many associative operations that it's worth searching for them on suspicion when creating a new data definition. And if you find an associative combinator, it's worth writing down this finding. In conversation with others, you can then simply say "cat is associative," and ideally, everyone will know what you mean.

`Expect` provides a property analogous to the functionality of `check-expect`. An `expect` expression has the following form:

```
(expect e1 e2)
```

`e1` and `e2` are expressions. The resulting property is satisfied if `e1` and `e2` return the same value – the comparison is performed as with `check-expect`.

Since `expect` compares like `check-expect`, it does not work on inexact numbers.

Figure 13.4: `expect`

We can also write associativity as a property using the `check-property` property. We'll start with `concatenate`, `+` and `*` come afterwards.

```
(check-property (for-
  all ((a (list-of integer)) (b (list-of integer)) (c
    (list-of integer))) (expect
      (concatenate (concatenate ab)
        c)
      (concatenate a (concatenate bc)))))
```

You might be wondering why `(list-of integer)` is used as the signature and not `(list-of %a)`, as stated in the signature declaration of `concatenate`. This is because signatures within a `for-all` loop cannot contain signature variables so that DrRacket can generate reasonably random values. (See also Figure 13.2 on page 404.)

And then there's something else, namely the `expect` function: We need this to compare the two lists that result from `concatenate`. So far, we only know functions that compare numbers (`=`), strings (`string=?`), and Boolean values (`boolean=?`). There isn't a general comparison function for lists. `expect` returns a property that compares arbitrary values – see Figure 13.4.

Task 13.11

Using the Teachpack `image.rkt`, write down the properties for the associativity of the combinators from section 5.2 on page 149 – `overlay`, `beside`, and `above`.

An associative function combines two "things" into one "thing". `+` combines two numbers into one, `concatenate` two lists into one, and `and` combine two Boolean values into one.

We can abstract from the signature declarations of `+`, `*`, `concatenate`:

```
(: op (ss -> s))
```

Associativity therefore requires not only the operation op but also the signature s such that op has the signature $(ss \rightarrow s)$. (And of course the associativity equation itself.)

Such “triads” consisting of a “basic signature”, an operation (or several operations), and equations are studied and classified in mathematical algebra. (There, sets are used instead of signatures.) The particularly useful, interesting, or frequently occurring of these triads—algebraic structures—are given their own names.

A combination of signature, comminator with signature $(ss \rightarrow s)$ and associative law

This is called, for example, a *semigroup*.⁴

Many semigroups fulfill another property: they possess a signature value that “does nothing,” the so-called “neutral element.” This already played a role in section 6.1 on page 6.1. Now we can finally understand it properly.

Let's call the neutral element n . Then the following properties apply:

```
(for-all ((as))
  (expect (op an a)) (for-all
    ((as))
      (expect (op na a))))
```

If you're wondering why we don't simply use ``and`` to combine both properties, it's because ``and`` only works on Boolean values, while ``expect`` returns a single property.

The neutral element can vary from semigroup to semigroup, even if the signature s is always the same. For example, 0 is the neutral element of $+$ and 1 is the neutral element of $\times$, even though both are numbers.

Task 13.12

What is the neutral element of “or” and “and”?

Ÿ

In the `concatenate` method, the neutral element is the empty list. We can capture this using ``check-property``:

```
(check-property (for-
  all ((a (list-of integer)))
    (expect (concatenate an empty) a)))
```

4. If “semigroups” exist, you might ask, do “groups” also exist? Yes, they do, and in algebra there is a large subfield called “group theory.” However, groups are much rarer than semigroups in programming, so we will focus on the latter.

```
(check-property (for-all
  ((a (list-of integer)))
  (expect (concatenate empty a) a)))
```

There is also a neutral element for `overlay`, `beside`, and `above`. It is called `empty-image`.

Task 13.13

Formulate `check-property` test cases that show that `empty-image` is a neutral element with respect to `overlay`, `beside`, and `above`.

As you can see, all the semigroups we have considered so far have a neutral element.

In fact, that's usually the case. So often, in fact, that if your data analysis reveals a semigroup, you should always also look for a neutral element. A semigroup with a neutral element is accordingly called a *monoid*.

If you were already familiar with associativity, then probably in this context with numbers. We can of course also express these using `check-property`:

```
(check-property (for-all
  ((a number)
    ((b number) (c
      number)
      (= (+ a (+ bc))
        (+ (+ ab) c)))))
```

But there is a surprise:

Property falsifiable with

a = #i9.75 b = -9 c = #i-7.7

(Your exact figures might be different.)

And indeed, when we try this in the REPL, both sides of the Equality reveals something different:

```
(+ #i9.75 (+ -9 #i-7.7))
→ #i-6.949999999999999
(+ (+ #i9.75 -9) #i-7.7)
→ #i-6.95
```

You may recall section 13.3 on page 13.3: The `#i` stands for "inexact", which means that the addition of these numbers rounds them off. Unfortunately, rounding also breaks the associativity. It gets better if we replace "number" with "rational", because when calculating with rational numbers numbers are not rounded.

Therefore, be careful when using real numbers in properties, i.e., the signatures of `real numbers` and `number`. When in doubt, use `rational methods` !

13.8 PROPERTIES OF FUNCTIONS ON LISTS

Lists form a semigroup with the `concatenate` operation, which gives `concatenate` a special significance as a list operation. We can leverage this when formulating tests on list functions by relating them to `concatenate`.

As a reminder, the list-sum function has the following signature:

```
(: list-sum ((list-of number) -> number))
```

Since the function takes a list as input, we can investigate what happens when the list

The following results from `concatenate` :

```
(check-property
  (for-all ((a (list-of rational))
            (b (list-of rational))))
  (= (list-sum (concatenate ab))
     (+ (list-sum a) (list-sum b))))
```

Remember: When using `for-all`, use `rational` instead of `number` to avoid problems with the Avoid the curve.

What's happening here? The ``list-sum`` call is being pushed "inwards" from the left to the right side of the equation. Additionally, the ``concatenate`` function is being applied to lists on the left side. The equation involves adding two numbers on the right side.

Now, lists with `concatenate` form a semigroup, just like numbers with `+`. The `list-sum` function not only maps lists to numbers, but also converts `concatenate` into numbers.

The sum of the numbers in the lists: The structure of one semigroup is calculated using `list-sum` in The structure of the other semigroup is mapped. In algebra, such a function is called a *homomorphism*. When using this term, one should always specify which algebraic structure it refers to. In this case, it is a semigroup *homomorphism*.

Finding a homomorphism in your code is almost always a good sign – you You're probably on the trail of a useful insight.

Task 13.14

Show that `list-sum` is even a monoid homomorphism, i.e., consisting of the neutral element. The neutral element in one monoid becomes the neutral element in the other monoid. Write a [formula/example].
[check-property test case!](#) Ÿ

Task 13.15

Formulate a property of `invert` using `concatenate` and write a
[check-property test case!](#) Ÿ

13.9 EXCURSUS: PROVING PROPERTIES

[Check-property](#) is quite good at finding counterexamples for properties that are not valid. That applies. But would we trust [check-property](#) – let's say – to ensure the correctness of an aircraft's control software? After all, it's possible that during the 100 tests, the
[The check-property](#) is randomly generated, so that a counterexample is simply not included.

Therefore, if a feature is truly indispensable for the use of a software, it should We can further increase the certainty that critical properties are valid. This can be achieved, for example, by... by mathematically *proving* that a property holds true. This section provides an example of how this can be done.

As we will see, this is quite tedious: We already have properties of Functions were proven in Chapter 8, for example the associativity of `cat` in Section 8.5.2 on Page 231. Here we will revisit associativity, but this time not on the not the mathematical `cat`, but the programmed function `concatenate`.

We therefore want to prove the following: For all lists `u`, `v` and `w`, the following holds:

```
(expect (concatenate u (concatenate vw))
        (concatenate (concatenate uv) w))
```

Basically, we conduct the proof using `In-` as with the mathematical function `cat`.

Induction. Unlike there, however, with `concatenate` we cannot simply combine arbitrarily many equations. instead, we must ensure that `DrRacket` is used for evaluation for both sides.

The expectation function calculates the same result. However, we must take into account that... `DrRacket` performs the evaluation steps for a call to `concatenate` in a specific order. Therefore, we must proceed with the proof in the same way as `Stepper` would.

Actually, we would even need to formalize this evaluation strategy in order to truly achieve a

to provide proof. We will do this later in Chapter 14 on page 439. Here, we content ourselves with the insight that when a function is called, the arguments are evaluated before the evaluation of the function body continues.

We use an induction template as described in section 8.5.2 on page 231.

1. We opt for induction via `u`, just as before. This ensures the template fits.

Let's use the name `f` for this, thus reformulating the property as follows:

```
(expect (concatenate f (concatenate vw)) (concatenate
      (concatenate fv) w))
```

2. Next, we consider the case where `f` is the empty list `#<empty-list>`. The equation it looks like this:

```
(expect (concatenate #<empty-list> (concatenate vw)) (concatenate
      (concatenate #<empty-list> v) w))
```

3. We will now prove this equation, starting with the "left-hand side" of the equation. DrRacket would first evaluate the first `concatenate`, replacing it with its definition and evaluating `empty` to `#<empty-list>`:

```
(concatenate #<empty-list> (concatenate vw)) ÿ ((ÿ (list1
list2) ...) #<empty-list> (concatenate vw))
```

(We wrote `ÿ` instead of `lambda` to save space.)

Next, `(concatenate vw)` is evaluated. Since we don't know what `v` and `w` are, we don't know what the result will be, nor how many steps it will take. But something will come out – we'll call that `vw`:

```
ÿ . . . ÿ ((ÿ (list1 list2) ...) #<empty-list> vw)
```

Now we can finally continue with the body of the function. We'll use `#<empty-list>` for `list1` and `vw` for `list2`:

```
ÿ (cond ((empty? #<empty-list>) vw) ((cons? #<empty-list>) ...))
```

The first branch of the **conditional** statement takes effect immediately:

```
ÿ (cond (#t vw) ((cons? #<empty-list>) ...))
ÿ vw
```

We can't go any further here. So let's look at the other side of the equation:

```
(concatenate (concatenate fv) w) =
(concatenate (concatenate #<empty-list> v) w)
```

Here too, DrRacket will use the definition for each of the two `concatenates` :

```
Ÿ ((Ÿ (list1 list2) ...) (concatenate #<empty-list> v) w) Ÿ ((Ÿ (list1 list2) ...)
```

```
(Ÿ (list1 list2) ...) #<empty-list> v) w)
```

Here, the "inner" call from `concatenate` is evaluated first:

```
Ÿ ((Ÿ (list1 list2) ...)
  (cond ((empty? #<empty-list>) v)
        ((cons? #<empty-list>) ...) w)
Ÿ ((Ÿ (list1 list2) ...) vw)
Ÿ ... Ÿ vw
```

The result is therefore the same as for the left side of the equation. Thus, the property is proven for the empty list.

4. Next, we need to formulate the induction hypothesis. It looks like this:

```
(expect (concatenate f (concatenate vw)) (concatenate
  (concatenate fv) w))
```

5. The inductive step follows. For this, we need to consider a list that is "one longer" than `f`, i.e., consisting of a first element `f1` and a remainder `f`. We call this list `f1f`:

```
(expect (concatenate f1f (concatenate vw)) (concatenate
  (concatenate f1f v) w))
```

6. To prove the inductive step, we first evaluate the left-hand side. We again use the name `vw` for the result of `(concatenate vw)`:

```
(concatenate f1f (concatenate vw)) Ÿ ((Ÿ (list1
list2) ...) f1f (concatenate vw)) Ÿ . . . Ÿ ((Ÿ (list1 list2) ...) f1f vw) Ÿ (cond
((empty? f1f) vw) ((cons? f1f)
```

```
(cons (first f1f) (concatenate (rest f1f) vw))))
```

This time it's the `cons?` branch's turn, because `f1f` is not empty:

```
Ÿ (cond (#f vw) ((cons?
  f1f)
  (cons (first f1f) (concatenate (rest f1f) vw))))
```

```

ŷ (cond ((cons? f1f) (cons
      (first f1f) (concatenate (rest f1f) vw)))) ŷ (cond (#t (cons (first f1f)
      (concatenate (rest f1f) vw)))) ŷ (cons (first f1f) (concatenate (rest f1f) vw)) ŷ (cons f1
      (concatenate (rest f1f) vw)) ŷ (cons f1 (concatenate (rest f1f) vw)) ŷ
      (cons f1 ((ŷ (list1 list2) ...) (rest f1f) vw)) ŷ (cons f1 ((ŷ
      (list1 list2) ...) f vw))

```

We can't go any further. So let's move on to the right side:

```

(concatenate (concatenate f1f v) w) ŷ ((ŷ (list1
list2) ...) (concatenate f1f v) w) ŷ ((ŷ (list1 list2) ...) ((ŷ (list1 list2) ...)
f1f v) w) ŷ ((ŷ (list1 list2) ...)

      (cond ((empty? f1f) v) ((cons?
      f1f) (cons (first f1f)
      (concatenate (rest f1f) v)))) w)
ŷ ((ŷ (list1 list2) ...)
      (cond (#fv)
      ((cons? f1f) (cons (first f1f)
      (concatenate (rest f1f) v)))) w)
ŷ ((ŷ (list1 list2) ...)
      (cond ((cons? f1f) (cons (first f1f)
      (concatenate (rest f1f) v)))) w)
ŷ ((ŷ (list1 list2) ...)
      (cond (#t (cons (first f1f) (concatenate (rest f1f) v)))) w)
ŷ ((ŷ (list1 list2) ...)
      (cons (first f1f) (concatenate (rest f1f) v)) w) ŷ ((ŷ (list1 list2) ...) (cons
      f1 (concatenate (rest f1f) v)) w)
ŷ ((ŷ (list1 list2) ...)
      (cons f1 ((ŷ (list1 list2) ...) (rest f1f) v)) w) ŷ ((ŷ (list1 list2) ...) (cons f1
      ((ŷ (list1 list2) ...) fv)) w)

```

We don't know what f , v , and w are. However, we assume that evaluating $((\hat{Y} \text{ (list1 list2) } \dots) \text{ fv})$ will produce *some result*, and we call that fv :

```

ŷ . . . ŷ ((ŷ (list1 list2) ...) (cons f1 fv) w)

```

But that's quite a lot of work, and also prone to errors. In practice, this only works for small functions. Anyone who wants to prove the properties of larger programs usually uses special tools that can check the proofs and, in many cases, derive them automatically.

TASKS

Task 13.16

What interesting properties does division have? Write them down as properties of $/$.

Task 13.17

Write down as complete a list as possible of interesting properties of the arithmetic and logical operations you know. Include comparison operations such as $<$, $\dot{y}$, etc.

Also, for each operation, find an interesting property that *is not* true and check if DrRacket finds a counterexample in each case.

Task 13.18

The distributive law applies to multiplication and addition :

$$a (b + c) = a b + a c$$

This law is not limited to numbers. It also applies to **AND** and **OR** . Formulate this as properties and have DrRacket check them.

Task 13.19

Formulate properties of **filter** and **map** in the context of **concatenate** and test them.

Task 13.20

The following function squares natural numbers:

```
(: square (natural -> natural)) (define square
(lambda (n) (cond
```

```
((zero? n) 0)
((positive? n) (+
  (square (- n 1))
  (- (+ nn 1))))))
```

Formulate this as a property and check it with `check-property`.

Bonus: Prove the property using induction.

14 EXCURSUS: THE λ -CALCULATE

This chapter deals with an important foundation for programming, which includes, among other things... It explains how the evaluation of a program works. Up to now, we've only done this in colloquial terms, neglecting or imprecisely addressing some subtle aspects of the evaluation.

This was explained. We want to correct that in this chapter.

For this we use the so-called λ -calculus, written out: *lambda calculus*, which formally

This summarizes what the Stepper in DrRacket visualizes. The "Lambda" in the name of the calculus is the template. for the **lambda** in our teaching languages.

14.1 WHAT DOES AN OLD "CALCULATION" HAVE TO DO WITH PROGRAMMING?

Perhaps you've already wondered why **lambda** is called that and not **function**.

Or something like that. The word "Lambda" holds the historical key to many aspects of programming. today. Before we delve into the technical details, this section briefly summarizes the history. which explains why the λ -calculus is so important for programming.

When the first programming languages were developed in the 1940s, their inventors looked for inspiration in existing "languages." There were two obvious starting points:

- written natural languages
- Mathematics

Both approaches have led to different traditions in the development of programming languages, which have influenced each other.

Mathematics is therefore an important source of inspiration because humanity used it centuries before the invention of computers to express facts unambiguously and To describe clearly: We need something like this for programming as well.

Our teaching languages are rooted in the tradition of mathematics. The now-famous computer scientist John McCarthy worked in the late 1950s on a system intended to represent "common sense".

As part of this project, he developed the programming language LISP [McC60]. As a trained mathematician, McCarthy sought inspiration in mathematics for this. Mathematical notation has the problem that the same notation can be used for different purposes in different contexts.

It is used for various purposes. Making general mathematical notation understandable to a computer is therefore extremely complex and was unthinkable for McCarthy, who believed that computers could be used for mathematical purposes. had to do that, which only have a tiny fraction of today's computing and storage capacity.

McCarthy therefore needed a particularly simple mathematical formalism that was nonetheless powerful enough to express mathematical functions. However, mathematical notation for defining functions was (and is) particularly diverse. (Some would say "sloppy.") McCarthy discovered that a mathematical calculus from the 1930s provided him with precisely the notation he needed for defining functions and their applications, and he therefore developed LISP based on this calculus.

This calculus was the λ -calculus of the mathematician Alonzo Church [Chu41], who had developed it as a contribution to the foundations of logic. (The λ itself had arisen by accident, because Church's typesetter mistakenly thought he saw a λ instead of the "roof" when writing $x^\wedge$.) The computers of the time could only display Roman letters, so McCarthy wrote out λ as the word **lambda**.

LISP already had a notation very similar to that of language learning: everything was enclosed in parentheses, and the operator always came first. (McCarthy had also developed another notation, but it proved unpopular.) The concept of lists as we know them today was also present back then. ("LISP" stands for "list processing.")

The λ -calculus also caught the attention of others as a possible basis for the development of programming languages. Particularly influential was Peter Landin's paper with the grandiose title "*The next 700 programming languages*" [Lan66], in which he proposed using the λ -calculus as the basis for an entire family of programming languages.

LISP and the language ISWIM ("If You See What I Mean") proposed in Landin's paper served as models for almost all modern programming languages. While many alternative approaches to programming languages with different foundations (for example, object-oriented programming) have existed since the 1960s, and these initially dispensed with the λ -calculus, their developers later realized that the elements of the λ -calculus, or of the programming languages built upon it, were an enrichment and added them.

So, if you understand λ -calculus, you know the foundation for modern programming. Getting there requires some patience. In particular, we still need to introduce some more mathematical notation, which will help you if you ever want to understand one of the relevant papers or books on this topic.

14.2 THE LANGUAGE OF λ -CALCULUS

The λ -calculus reduces the elements of programming to only the absolutely necessary: the fewer elements there are, the less we have to explain and define.

So what is absolutely essential in our teaching languages? In Chapter 1, we started with a number. Do we need numbers, then? We have written many programs without numbers.

We will see that we can replicate numbers using other elements of the language.

The next element is function applications. These are found in virtually every program. Function applications, however, only make sense if we can also create functions: we need abstractions. And since every abstraction includes one or more variables, we also need variables. So, three fundamental ideas:

- Application •
- Abstraction
- Variable

Variables are names, and all three ideas of the λ -calculus are fundamentally related to variables:

An abstraction *binds* a variable; an application assigns something to a variable.

In terms of application and abstraction, we even limit ourselves to a single parameter. We can replicate all other elements of our programming languages by translating them into these three constructs. They form the backbone of the λ -calculus.

To begin, we first need a *language* for the calculus. We define this using a grammar. (This is the notation for inductive definitions that we introduced in section 8.6 on page 233.)

The term "countable set" appears in the definition. While technically necessary, you don't need it for understanding and intuitively grasping λ -calculus, so you can ignore it. However, if you're interested: the adjective "countable" applied to a set means that the set is infinitely large and can be numbered or "counted." Here, we're referring to the countable set of variables, which correspond to variables in programs. We could also number these variables by assigning numbers to the letters.

Definition 14.1 (language of the λ -calculus L_λ)

Let V be a countable set of variables. The language of the λ -calculus, the set of λ -terms, L_λ , is defined by the following grammar:

$$\begin{aligned} \lambda L_\lambda &::= \lambda \dot{\lambda} V \dot{\lambda} \mid \\ &(\dot{\lambda} L_\lambda \dot{\lambda} \dot{\lambda} L_\lambda) \mid \\ &(\dot{\lambda} \dot{\lambda} V \dot{\lambda} \dot{\lambda} L_\lambda) \end{aligned}$$

A λ -term of the form $(e_0 e_1)$ is called an *application*. A term of the form $(\dot{\lambda} v. e)$ is called an *abstraction*, where v is called a *parameter of the abstraction* and e is the *body*.

Here are a few examples of λ -terms. In these specific examples, we have used x , y , and f as variables. You shouldn't attach any importance to the choice of these letters. We could just as easily have used a , b , and c .

x Variable
 (fx) Application
 $((fx) (\lambda x.x))$ Application $(\lambda x.$
 $(\lambda y.(xy))$ Abstraction $(\lambda x.(fx))$
 Abstraction $(\lambda x.(f (\lambda x x)))$
 Abstraction $((\lambda x.x) (\lambda x.x))$
 Application

We have indicated the type of term in each case. This corresponds to the grammar clause for λ -terms. With x , it should be clear that it is a variable. Both applications and abstractions begin with an opening parenthesis. However, in abstractions, the parenthesis is directly followed by a λ : Accordingly, the next two terms are applications, and the three after that are abstractions. The last term is an application – after the opening parenthesis comes another set of parentheses, and the λ only after that.

Task 14.1

Classify whether each is a variable, an application, or an abstraction:

y
 (xy)
 $(\lambda x.y)$
 $((\lambda x.y) f) f$

 $(f (\lambda x.y))$ λ

Sometimes, however, we also talk about λ -terms of a certain form in general; for example, the definition above states " λ -term of the form $(e_0 e_1)$ ". This refers to all λ -terms that originate from the second rule of the grammar, meaning that e_0 and e_1 can be any λ -terms.

Here are some examples of λ -terms of this form:

$(fx).(f (\lambda x.x)), ((\lambda x.x) (\lambda x.x))$

The definition also mentions terms of the form $(\lambda v.e)$, which could be, for example:

$$(\lambda x.x), (\lambda y.x), (\lambda x.xx), (\lambda x.(fx) y), (\lambda y.x)$$

You see, the letter v in $(\lambda v.e)$ stands for a variable, for example x , y or f .

If you look closely, you'll see that the two differ typographically: The "normal" variables x , y , f look like regular letters, while v is set in italics. The v is

That is, a variable that stands for another variable – a so-called *metavariable*.

In this chapter, the metavariable e represents any λ -term, while u , v , and w are always metavariables. However, this is purely a convention: we could just as easily have used any other names. You can choose letters.

In publications, λ -terms are often abbreviated, and we'll do the same here. This is probably done out of laziness, but it takes some getting used to. The first form of the abbreviation omits parentheses. Instead of the fully parenthesized term...

$$(\lambda f.(\lambda x.(fx)))$$

We will write this here:

$$\lambda f.\lambda x.fx$$

In principle, this term can be bracketed in different ways: $(\lambda f.((\lambda x.f) x))$, $(\lambda f.(\lambda x.(fx)))$ or $((\lambda f.\lambda x.f) x)$. However, we use the common convention that the body of an abstraction is extends as far to the right as possible. Therefore, the unparenthesized term clearly represents the a term completely enclosed in parentheses, which appears above it.

The terms of the form $\lambda v.e$ are restricted to one parameter. However, there are two further abbreviations in the notation of λ -terms:

- $\lambda x_1 \dots \lambda x_n.e$ stands for $\lambda x_1.(\lambda x_2.(\dots \lambda x_n.e) \dots)$.
- $e_0 \dots e_n$ stands for $(\dots (e_0 e_1) e_2) \dots e_n$.

$\lambda fxy.fx$ is therefore another way of writing the term $(\lambda f.(\lambda x.(\lambda y.((fx) y))))$.

Here are more examples of terms and their abbreviations:

$$\begin{aligned} & (fx) fx \\ & (fx) yfx \\ & (\lambda x.(xx)) \lambda x.xx \\ & (\lambda x.((xy) z)) \lambda x.xyz \\ & (\lambda x.(\lambda y.y(xy))) \lambda xy.xy \end{aligned}$$

Task 14.2

Write down the "complete" λ -terms for the following abbreviations:

$$\begin{aligned} & fxyz \\ & \lambda xyz. xyz \\ & ((\lambda xy. xy) ab) \end{aligned}$$
14.3 λ -INTUITION

It is no coincidence that the languages of instruction use exactly the same terms as the λ -calculus.

A lambda expression with one parameter corresponds to an abstraction in the λ -calculus, and the applications in the teaching languages correspond to the applications in the λ -calculus. The teaching languages were deliberately based on the λ -calculus.

The intuition for the λ -terms is similar to that in the languages being taught: An abstraction stands for a mathematical function, specifically one that can be calculated by a computer program. An application refers to the use of a function, and a variable refers to the parameter of an enclosing abstraction and represents the operand.
the application.

But it's important to note: This is just an intuition at this point. So far, we only have the language of the λ -calculus, but not what the λ -terms mean or how they behave.

What is remarkable about the λ -calculus is that it *only* contains functions, not even numbers. $\#t$, $\#f$, or composite data. That's why the language of calculus seems appropriate at first glance. Still spartan and unintuitive: It's not even possible to write down a function that adds two numbers so directly – after all, there are no numbers. However, as will become clear below. As will be highlighted in section 14.8, all these things can be modeled using functions.

One more thing: The functions each only have one parameter. However, this isn't a real limitation: We can cleverly modify functions with multiple parameters to create multi-stage functions with one parameter each, as already discussed in section 9.5. on page 270.

14.4 HOW DO VARIABLES WORK?

In λ -calculus, (almost) everything revolves around variables. They all function according to the same principle. as with the languages taught, the lexical connection. We discussed lexical connection in section 9.6 on page 274.

Just a quick reminder: The occurrence of a variable v as a λ -term always belongs to the innermost. The enclosing abstraction $\lambda v.e$, whose parameter is also v , is therefore the case for $\lambda x.y$ ($\lambda y.y$).

The first y is free, while the second y is bound by the second abstraction.

Furthermore: If there is no abstraction that binds a variable, the variable is called *free*. To precisely define binding and "freedom," we need two helper functions for dealing with variables.

Definition 14.2 (Free and bound variables)

The functions ``free`` and ``bound`` provide the sets of *free* and *bound* variables, respectively, for a λ -term.

$$\begin{aligned}
 \text{free}(e) &\stackrel{\text{def}}{=} \begin{cases} \lambda \{v\} & \text{if } e = v \\ \text{free}(e_0) \cup \text{free}(e_1) & \text{if } e = e_0 \ e_1 \\ \text{free}(e_0) \setminus \{v\} & \text{if } e = \lambda v.e_0 \end{cases} \\
 \text{bound}(e) &\stackrel{\text{def}}{=} \begin{cases} \lambda \ \lambda & \text{if } e = v \\ \text{bound}(e_0) \cup \text{bound}(e_1) & \text{if } e = e_0 \ e_1 \\ \text{bound}(e_0) \cup \{v\} & \text{if } e = \lambda v.e_0 \end{cases}
 \end{aligned}$$

A λ -term e is called *closed* if $\text{free}(e) = \lambda$.

Some examples:

$$\begin{aligned}
 \text{free}(\lambda x.y) &= \{y\} \\
 \text{bound}(\lambda x.y) &= \{x\} \\
 \text{free}(\lambda y.y) &= \lambda \\
 \text{bound}(\lambda y.y) &= \{y\} \\
 \text{free}(\lambda x.\lambda y.\lambda x.x \ (\lambda z.ay)) &= \{a\} \\
 \text{bound}(\lambda x.\lambda y.\lambda x.x \ (\lambda z.ay)) &= \{x, y, z\}
 \end{aligned}$$

The intuition is as follows: An abstraction in a λ -term *binds* a variable within its

scope. A free variable, on the other hand, is one that is not bound by an abstraction. Therefore, in an abstraction, ``free`` represents the difference between the free and the bound variables.

In contrast, ``bound`` collects all variables from abstractions. If you combine ``free`` and ``bound`` of a term as ``free(e) ∪ bound(e)``, you get all the variables of a λ -term.

Within a term, the same variable can appear both freely and bound:

$$\begin{aligned}\text{free}(\lambda x. y (\lambda y. y)) &= \{y\} \\ \text{bound}(\lambda x. y (\lambda y. y)) &= \{x, y\}\end{aligned}$$

The letter y appears both inside *and* outside a binding abstraction. The free and bound forms... "sein" therefore always refers to specific occurrences of a variable in a λ -term.

Task 14.3

Write down the results of the following applications of free and bound:

$\text{free}(xy)$
 $\text{free}(\lambda x. xy)$
 $\text{free}(\lambda x y. xy)$
 $\text{free}((\lambda x. y) x)$
 $\text{bound}(xy)$
 $\text{bound}(\lambda x. xy)$
 $\text{bound}(\lambda x y. xy)$
 $\text{bound}((\lambda x. y) x)$

λ

The λ -calculus relies on replacing variables with others without changing the membership of the variable occurrences and their corresponding abstractions. The mechanism for this is also called *substitution*.

Definition 14.3 (Substitution)

For $e, f \in \mathcal{L}_\lambda$, $e[v \mapsto f]$ means that in e the variable v is replaced or *substituted* by f . The function is defined as follows:

$$\begin{aligned}e[v \mapsto f] &\stackrel{\text{def}}{=} \begin{cases} f & \text{if } e = v \\ e & \text{if } e \text{ is a variable and } e \neq v \\ \lambda y. e0 & \text{if } e = \lambda y. e0 \\ \lambda y u. (e0[v \mapsto f]) & \text{if } e = \lambda y u. e0 \text{ and } u \neq y, u \neq \text{free}(f) \\ \lambda y u. (e0[u \mapsto f][v \mapsto f]) & \text{if } e = \lambda y u. e0 \\ & \text{and } u \neq y, u \neq \text{free}(f), u \neq \text{free}(e0) \neq \text{free}(f) \\ e0[v \mapsto f] & \text{if } e = e0 \end{cases}\end{aligned}$$

The definition of substitution appears complicated at first glance, but it follows the principle of lexical binding. The first rule states that the occurrence of a variable is replaced by a substitution of precisely that variable:

$$v[v \dot{\rightarrow} f] = f$$

The second rule states that the occurrence of *another* variable is not affected by the substitution:

$$e[v \dot{\rightarrow} f] = e \quad \text{e is a variable and } e \neq v$$

The third rule is somewhat surprising at first glance:

$$(\dot{\lambda}v.e0)[v \dot{\rightarrow} f] = \dot{\lambda}v.e0$$

A $\dot{\lambda}$ -expression whose parameter is the variable to be substituted remains unchanged. This is because the $\dot{\lambda}$ -expression already defines the membership of all occurrences of v in $e0$: an occurrence of v in $e0$ belongs either to this abstraction or to another abstraction with v as a parameter that is further inside $e0$ – v is bound to $(\dot{\lambda}v.e0)$ and $v \dot{\rightarrow} \text{bound}(\dot{\lambda}v.e0)$. Since the substitution must not change these memberships, it leaves the v alone.

The situation is different if the variable of abstraction is something else – the fourth rule:

$$(\dot{\lambda}u.e0)[v \dot{\rightarrow} f] = \dot{\lambda}u.(e0[v \dot{\rightarrow} f]) \quad \text{u } \dot{\neq} v, u \dot{\rightarrow} \text{free}(f)$$

In this case, the substitution is applied to the body of the abstraction. It is important that u does not occur freely within f – otherwise, when f is substituted, a free occurrence of u could be bound by the enclosing abstraction. This would also violate the membership rule of lexical binding.

The fifth rule describes what happens if u does indeed occur freely in f :

$$(\dot{\lambda}u.e0)[v \dot{\rightarrow} f] = \dot{\lambda}u.\dot{\lambda}u'.(e0[u \dot{\rightarrow} u'] \dot{\rightarrow} [v \dot{\rightarrow} f]) \quad \text{u } \dot{\neq} v, u \dot{\rightarrow} \text{free}(f) \dot{\rightarrow} \text{free}(e0) \dot{\rightarrow} \text{free}(f)$$

To prevent the free u in f from being "captured" by the abstraction, the u in the abstraction is replaced by a "fresh" u' .

u' replaced, which does not yet exist anywhere freely.

The rule for substitution on applications appears recursively in the operator and operand:

$$(e0 e1)[v \dot{\rightarrow} f] = (e0[v \dot{\rightarrow} f])(e1[v \dot{\rightarrow} f])$$

Here is a slightly more extensive example of substitution:

$$\begin{aligned}
 (\lambda x. \lambda y. x (\lambda z. z) z)[z \dot{y} xy] &= \lambda x. \lambda y. ((\lambda y. x (\lambda z. z) z)[x \dot{y} x \dot{y}][z \dot{y} xy]) \\
 &= \lambda x. \lambda y. ((\lambda y. ((x (\lambda z. z) z)[x \dot{y} x \dot{y}]))[z \dot{y} xy]) \\
 &= \lambda x. \lambda y. ((\lambda y. (x[x \dot{y} x \dot{y}] ((\lambda z. z)[x \dot{y} x \dot{y}]) z[x \dot{y} x \dot{y}]))[z \dot{y} xy]) \\
 &= \lambda x. \lambda y. ((\lambda y. (x \dot{y} (\lambda z. z) z)[z \dot{y} xy]) = \lambda x. \\
 &= \lambda x. \lambda y. \lambda y. ((x \dot{y} (\lambda z. z) z)[y \dot{y} y \dot{y}][z \dot{y} xy]) \\
 &= \lambda x. \lambda y. \lambda y. \lambda y. ((x \dot{y} [y \dot{y}] ((\lambda z. z)[y \dot{y} y \dot{y}]) z[y \dot{y} y \dot{y}])[z \dot{y} xy]) \\
 &= \lambda x. \lambda y. \lambda y. \lambda y. ((x \dot{y} (\lambda z. z) z)[z \dot{y} \\
 &= \lambda x. \lambda y. \lambda y. \lambda y. [z \dot{y} xy] ((\lambda z. z)[z \dot{y} xy]) z[z \dot{y} xy]) \\
 &= \lambda x. \lambda y. \lambda y. \lambda y. (\lambda z. z) (xy)
 \end{aligned}$$

It is clearly visible how the free variables x and y from the substitution $z \dot{y} xy$ remain free in the result, while the bound variables x and y from the original term are renamed to avoid an erroneous binding of their substituted namesakes.

14.5 REDUCTION IN λ -CALCULA

The λ -calculus is a so-called *reduction calculus*. This means it establishes rules for how one λ -term is transformed, or *reduced*, into another λ -term. ("Reduce" sounds as if the term becomes smaller. Sometimes it does become smaller, but sometimes it also becomes larger.) We have already seen an example of a reduction calculus, namely the stepper in DrRacket, which also transforms a "left" expression into a "right" expression. Here are the most important reduction rules in the λ -calculus, also named with Greek letters:

Definition 14.4 (Reduction rules)

The reduction rules in the λ -calculus are the β -reduction $\beta\beta$ and the η -reduction $\eta\eta$:

$$\begin{aligned}
 \lambda x. e \beta\beta \lambda y. (e[x \dot{y} y]) \text{ if } y \text{ free}(e) \\
 (\lambda v. e) f \eta\eta e[v \dot{y} f]
 \end{aligned}$$

Alpha reduction (often *also called alpha conversion*) renames one bound variable to another. Here are some examples:

$$\begin{aligned} & \lambda a. a \ \lambda y. \lambda c. c \\ & \lambda a. ab \ \lambda y. \lambda c. cb \\ & \lambda a. (\lambda a. aa) \ a \ \lambda y. \lambda c. (\lambda a. \ a^y \ a^y) \ c \end{aligned}$$

Beta reduction is the central rule of the λ -calculus and stands for functional application: an abstraction is applied by replacing the occurrences of its parameter with the operand of an application. Here are two examples:

$$\begin{aligned} & (\lambda x. x) \ y \ \lambda y. y \\ & (\lambda a. aa) \ (\lambda x. x) \ \lambda y. (\lambda x. x) \ (\lambda x. x) \end{aligned}$$

In the examples above, the reduction rule is applied to an entire λ -term. However, it is also permissible to apply a reduction rule to a subterm:

$$\begin{aligned} & \lambda b. \lambda a. a \ \lambda y. \lambda b. \lambda c. c \\ & \lambda z. (\lambda x. x) \ y \ \lambda y. \lambda z. y \end{aligned}$$

This is true for every reduction calculus; it's inherent to the concept of "reduction" itself. You're familiar with this principle from regular arithmetic:

$$y \times (x + x + x) = y \times (3 \times x)$$

The stepper demonstrates that reduction to sub-terms also works in the languages taught: These are highlighted in color. However, there is a difference: In arithmetic and in λ -calculus, reduction to *any* sub-term is permitted. Under certain circumstances, several different reductions are possible:

$$\begin{aligned} & (\lambda v. vv) \ (\underline{(\lambda u. u) \ w}) \ \lambda y. (\lambda v. vv) \ w \ (\lambda v. vv) \ (\underline{(\lambda u. u) \ w}) \\ & \underline{w} \ \lambda y. (\underline{(\lambda u. u) \ w}) \ (\underline{(\lambda u. u) \ w}) \end{aligned}$$

We have underlined which subterm is reduced: This subterm is called *the redex*. In the languages being taught, it is always precisely defined which subterm is the redex. We will learn the rules for this in section 14.7 on page 452 .

$$\frac{(\ddot{y}v.vv) \text{ w } \ddot{y}\ddot{y} \text{ ww}}{(\ddot{y}u.u) \text{ w } ((\ddot{y}u.u) \text{ w}) \ddot{y}\ddot{y} \text{ w } ((\ddot{y}u.u) \text{ w})} \frac{\ddot{y}\ddot{y} \text{ ww}}{\quad}$$

Can you reduce the redex in this example in another way, i.e., at

Can I choose at least one other step differently? If so, will the result be the same?

What happens if you reduce it further?

 $\ddot{y}$

If we need several steps to get from one term to another, we will use the following notation "with asterisks":

$$\begin{array}{c} e^* \\ \ddot{y}y e \\ e \ddot{y}\ddot{y}y e \end{array} \quad \bar{y}$$

This means that we proceed in several reduction steps (there can also be 0 steps, i.e., $e = e_y$). This construction – “0 or more steps” – is also called *Transitive-reflexive conclusion*. The “reflexive” means that zero steps are also possible, the “transitive” means that there can be multiple steps.

We will also use the notation $\tilde{y}\tilde{y}$ to express that there is an $\tilde{y}$ - or a $\tilde{y}$ -reduction. Accordingly, $\tilde{y}\tilde{y}$ is a sequence of $\tilde{y}$ - and $\tilde{y}$ -reductions that can alternate arbitrarily. We go a step further: $\tilde{y}\tilde{y}$ means that the reduction can also proceed "backwards". Accordingly, there are also $\tilde{y}\tilde{y}$ and $\tilde{y}\tilde{y}$. $\tilde{y}\tilde{y}$ is called *the symmetric closure* of $\tilde{y}\tilde{y}$ ("symmetric" because the reduction works the same in both directions).

$\tilde{y}\tilde{y}^*$ the *reflexive-transitive-symmetric closure*.

If we can transform one $\tilde{y}$ -term into another using $\tilde{y}\tilde{y}, \tilde{y}^*$, we call this

Both terms are equivalent in the $\dot{y}$ -calculus:

Definition 14.5 (Equivalence in the $\ddot{y}$ -calculus)

Two terms $e_1, e_2 \in L_{\Sigma}$ are called *β -equivalent* or simply *equivalent* if e_1

$$\ddot{y}, \ddot{y} \in \mathbb{R}^2$$

This is true. The notation for this is $e_1 \dot{\vee} e_2$.

14.6 NORMAL FORMS

In the last section, we saw using the example $(\lambda v.vv) ((\lambda u.u) w)$ that there are several possible reduction sequences starting from one and the same λ -term. If you continue each one with λ -reductions to the end, you'll find that the same term always results. That isn't quite right: the term always has the same structure in the end, but potentially different names for the variables. However, you can use λ -reduction to...

Rename the variables so that they are also the same.

To discuss this property of the λ -calculus, we need the concept of *normal-form* that describes terms that cannot be reduced further:

Definition 14.6 (Normal form)

Let e be a λ -term. A λ -term e is a *normal form* of e if $e \rightarrow^* e$ valid and no λ -term e' exists with $e \rightarrow e'$.

We can use normal forms to simplify the proof of equations in calculus: If someone claims that $e_1 \equiv e_2$, we don't know which sequence of λ and β reductions transformed one term into the other. It might be that we have to try many reduction sequences to find the correct one.

Fortunately, the proof for $e_1 \equiv e_2$ is simpler if we can reduce both to a normal form. Then we only need to check whether we can convert the normal forms into each other by λ -reductions. If so, then e_1 and e_2 are equivalent; otherwise, they are not. The phrases "the normal forms are λ -equivalent" or "the normal forms are equal modulo λ -reduction" are also used for this situation.

An important property on the way to the uniqueness of normal forms is the theorem of Church/Rosser:

Theorem 14.7 (Church/Rosser property) The λ -reduction rule has the *Church/Rosser property*: For any λ -terms e_1 and e_2 with $e_1 \rightarrow^* e_2$, there always exists a λ -term e $e_1 \rightarrow^* e$ and $e_2 \rightarrow^* e$.

Figure 14.1 graphically represents the statement of the Church-Rosser theorem. The proof of the theorem is unfortunately quite extensive and technical. It can be found in the relevant literature on λ -calculus [HS86].

The Church/Rosser property paves the way for the use of normal forms for finite-the of proofs in the λ -calculus:

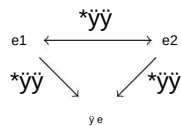


Figure 14.1: The Church/Rosser property

Theorem 14.8 (Uniqueness of the normal form) A λ -term e has at most one normal form modulo β -reduction.

Proof : Assume there are two distinct normal forms e_1 and e_2 of e . According to [Sentence 14.7](#), then requires another λ -term e' give with $e_1 \beta e' \beta e_1$ and $e_2 \beta e'$. Either are therefore not distinct, or at least one of them is not a normal form, contradicting the assumption. $\square$

[Theorem 14.8](#) confirms that the λ -calculus is a useful mechanism for describing the behavior of computer programs: For a λ -term, it is irrelevant in which order the reductions are applied: Every reduction sequence that leads to a normal form always leads to the same normal form.

Unfortunately, some λ -terms do not have a normal form. Here is an example:

$$(\lambda x.xx)(\lambda x.xx) \beta (\lambda x.xx) (\lambda x.xx)$$

Such terms without normal forms can be reduced indefinitely without the process ever reaching a conclusion. They thus correspond to programs that continue calculating endlessly. This is not a specific defect of the λ -calculus: any calculus powerful enough to model arbitrary computer programs has this property.

14.7 EVALUATION STRATEGIES

The application of the Church-Rosser theorem has a practical drawback: While it states that the equivalence of two terms can be proven by comparing their normal forms, it unfortunately says nothing about how these normal forms are found. If we delegate the evaluation to a computer, we should devise a strategy that leads to the normal form as efficiently as possible and avoids unnecessary evaluation steps.

Such an *evaluation strategy* is usually formulated in such a way that, starting from a λ -term, it uniquely determines the λ -redex for the next evaluation step. For the λ -calculus, there are several popular evaluation strategies, each with its own advantages and disadvantages regarding the effective finding of normal forms.

A popular evaluation strategy is left-wing reduction, also known as *normal-order reduction*, *tion* or *leftmost-outermost reduction*.

Definition 14.9 (Left wing reduction)

In left-wing reduction, the λ -redex that is located as far left as possible is always reduced. Specifically, this means:

1. If the λ -term has the form $(\lambda v.e) f$, then the entire term is the nearest redex – i.e., *outside*.
2. If the λ -term has the form ef , and e is *not* an abstraction, then look for the redex within e – that is, *on the left*.
3. If the λ -term has the form ef , and e is *not* an abstraction, and if there is no redex in e to finding is searching for the Redex within f .
4. If the λ -term has the form $\lambda v.e$, then search for the redex within e .

Here is an example:

$$\begin{array}{c} (\lambda x.\lambda z.(\lambda y.y) x) ((\lambda y.y) z) \lambda y.\lambda z.(\lambda y.y) ((\lambda y.y) z) \\ \lambda y.\lambda z.(\lambda y.y) z \lambda y. \\ \lambda z.z \end{array}$$

The reduction of the left wing has the following extremely pleasant characteristic:

Theorem 14.10 If e is a normal form, then the left-wing reduction of e leads to e modulo λ -reduction.

Therefore, if a normal form exists, the left-wing reduction will also find it. There are other evaluation strategies as well:

Definition 14.11 (Call-by-Name Reduction)

Call *-by-name reduction* is like left-wing reduction, but without the last rule for terms of the form $\lambda v.e$.

Call-by-name reduction, therefore, stops prematurely compared to left-wing reduction, as soon as an abstraction is produced. In the example above, this happens after the first step:

$$(\tilde{y}x.\tilde{y}z.(\tilde{y}y.y) x) ((\tilde{y}y.y) z) \tilde{y}\tilde{y} \tilde{y}z.(\tilde{y}y.y) ((\tilde{y}y.y) z)$$

It's great that the left-wing reduction always finds a normal form if one exists. Unfortunately, this doesn't always happen in the most efficient way. In the following example, the subterm $((\tilde{y}y.y) z)$ is first "doubled" and therefore has to be reduced twice.

$$\begin{array}{c} (\tilde{y}x.xx) ((\tilde{y}y.y) z) \tilde{y}\tilde{y} ((\tilde{y}y.y) z) ((\tilde{y}y.y) z) \tilde{y}\tilde{y} z ((\tilde{y}y.y) \\ z) \quad \text{—————} \\ \tilde{y}\tilde{y} zz \end{array}$$

Another evaluation strategy avoids such double work: *Left-inner reduction*, also called *applicative-order reduction* or *leftmost-innermost reduction*.

Definition 14.12 (Left inner reduction)

In left medial reduction, the $\tilde{y}$ -redex that is located as far left medially as possible is always reduced. Specifically, this means:

1. If the $\tilde{y}$ -term has the form ef , then look for the redex inside e – i.e., *on the left inside*.
2. If the $\tilde{y}$ -term has the form ef and if no redex can be found in e , search for the Redex within f .
3. If the $\tilde{y}$ -term has the form $(\tilde{y}v.e) f$ and no redex can be found in f , then the whole Term the next redex.
4. If the $\tilde{y}$ -term has the form $\tilde{y}v.e$, then search for the redex within e .

Left-in reduction is more effective than left-out reduction in the example above, because the argument of the outer application is evaluated first:

$$\begin{array}{c} (\tilde{y}x.xx) ((\tilde{y}y.y) z) \tilde{y}\tilde{y}i (\tilde{y}x.xx) z \\ \tilde{y}\tilde{y}i zz \end{array}$$

Unfortunately, left-in reduction does not always lead to a normal form, even if left-out reduction does. The following term, for example, has two redexes – the whole term and $(\tilde{y}z.zz) (\tilde{y}z.zz)$:

$$(\tilde{y}x.\tilde{y}y.y) ((\tilde{y}z.zz) (\tilde{y}z.zz))$$

The left-in strategy selects the inner subterm as the first redex:

$$(\lambda z.z) (\lambda z.z) \lambda y. (\lambda z.z) (\lambda z.z)$$

Thus, the left-inside reduction runs indefinitely in a circle, while the left-outside reduction immediately reduces the entire term and yields the normal form $\lambda y.y$.

A variant of left-in reduction, which is used in most programming languages, is *call-by-value reduction*. It forms the basis, in particular, for our teaching languages.

Definition 14.13 (Call-by-Value Reduction)

Call *-by-value reduction* is like left-inside reduction, but without the last rule for Terms of the form $\lambda v.e$.

Task 14.5

Run the following program (and others if you like) on the stepper and see for yourself, that he uses call-by-value reduction:

```
(define z "z")
((lambda (x) (lambda (z) ((lambda (y) y) x))) ((lambda (y) y) z))
```

λ

Programming languages with call-by-value reduction are called *strict languages*. There are also *non-strict* languages, such as Haskell [Mar10], which are based on so-called *lazy evaluation*. Their evaluation strategy is closely related to call-by-name evaluation in λ-calculus.

However, it avoids multiple unnecessary evaluations of expressions by assigning them the value the first time and reuse it afterwards.

14.8 The λ-calculus as a programming language

So far, it seems that the λ-calculus is of no use: The three types of λ-terms correspond to the abstractions, applications, and variables from the teaching languages,

But everything else is missing, for example, arithmetic with numbers, but also Boolean values and branching. We could add all of that to the λ-calculus, but that would make the calculus more complicated. this makes it more difficult to prove important properties of the calculus.

However, we can also try to reconstruct the missing constructs within the calculus. We know, for example, that we can represent functions with multiple parameters using Schönfinkel. We can replicate this strategy. We will pursue this strategy in this chapter and show that we can replicate the following. Constructs can be recreated by translation into the "pure" λ-calculus:

- Branching and Boolean values
- Pay
- **define** and recursion

These constructs make the λ-calculus as powerful as a fully-fledged programming language, without making the calculus itself more complicated.

14.8.1 BRANCHES AND BOOLISH VALUES

The binary branch (if bka) selects the sequential function depending on the boolean value b. k or the alternative a. We can express this idea in λ-terms. We define true

as a term that selects the first of two arguments and rejects the second; false is the opposite.

The branching applies the condition to the consequent and the alternative:

true	def	= λxy.x
false	def	= λxy.y
if	def	= λtka.tka

The following example shows how the if function works:

$$\begin{aligned}
 \text{if true } e1 \ e2 &= (\lambda tka.tka) \text{ true } e1 \ e2 \\
 &\lambda\lambda (\lambda ka.\text{true } ka) \ e1 \ e2 \\
 &\lambda\lambda_2 \text{ true } e1 \ e2 \\
 &= (\lambda ka.k) \ e1 \ e2 \\
 &\lambda\lambda (\lambda a.e1) \ e2 \\
 &\lambda\lambda e1
 \end{aligned}$$

Task 14.6

Write down the analogous proof for false!

λ

14.8.2 NATURAL NUMBERS

There are various approaches to reproducing numbers. One of them is called *Church numerals*.

The Church numeral $\tilde{y}n\tilde{y}$ of a natural number n is a function that has an n -fold application undertakes.

$$\tilde{y}n\tilde{y} \stackrel{\text{def}}{=} \tilde{y}f.\tilde{y}x.(f^n(x))$$

The notation f^n used there is defined inductively as follows:

$$f^n(e) \stackrel{\text{def}}{=} \begin{cases} \tilde{y}e & \text{if } n = 0 \\ \tilde{y}(f(\tilde{y}^{n-1}(e))) & \text{otherwise} \end{cases}$$

According to this definition, $\tilde{y}0\tilde{y}$ is $\tilde{y}f.\tilde{y}x.x$, $\tilde{y}1\tilde{y}$ is $\tilde{y}f.\tilde{y}x.fx$, $\tilde{y}2\tilde{y}$ is $\tilde{y}f.\tilde{y}x.f(fx)$ etc.

Here is the definition of the *successor function* succ , which adds one to a number. It makes an application for this:

$$\text{succ} \stackrel{\text{def}}{=} \tilde{y}n.\tilde{y}f.\tilde{y}x.nf(fx)$$

Here is an example of how to use succ :

$$\begin{aligned} \text{succ } \tilde{y}1\tilde{y} &= (\tilde{y}n.\tilde{y}f.\tilde{y}x.nf(fx)) (\tilde{y}f.\tilde{y}x.fx) \\ &\quad \tilde{y}\tilde{y} \tilde{y}x.(\tilde{y}f.\tilde{y}x.fx) f(fx) \tilde{y}\tilde{y} \tilde{y}x.(\tilde{y}x.fx) (fx) \\ &\quad \tilde{y}\tilde{y} \tilde{y}x.(\tilde{y}x.f(fx)) = \tilde{y}2\tilde{y} \end{aligned}$$

The following is the definition of the *predecessor function*. It subtracts one from a number:

$$\text{pred} \stackrel{\text{def}}{=} \tilde{y}x.\tilde{y}y.\tilde{y}z.x (\tilde{y}p.\tilde{y}q.q (py)) ((\tilde{y}x.\tilde{y}y.x) z) (\tilde{y}x.x)$$

Task 14.7

Show that $\text{pred } \tilde{y}3\tilde{y}$ reduces to $\tilde{y}2\tilde{y}$!

$\tilde{y}$

In conjunction with the boolean values, the following function zerop tests a number to see if it is 0:

$$\text{zerop} \stackrel{\text{def}}{=} \tilde{y}n.n (\tilde{y}x.\text{false}) \text{true}$$

The functionality of `zerop` can be most easily understood through an example:

$$\begin{aligned} \text{zerop } \lambda 0 \lambda y &= (\lambda n. n \ (\lambda x. \text{false}) \ \text{true}) \ \lambda 0 \lambda y \\ &\lambda y \ \lambda 0 \lambda y \ (\lambda x. \text{false}) \ \text{true} \\ &= (\lambda f. \lambda x. x) \ (\lambda x. \text{false}) \ \text{true} \\ &\lambda y \ (\lambda x. x) \ \text{true} \\ &\lambda y \ \text{true} \end{aligned}$$

14.8.3 RECURSION AND FIXED POINT THEOREM

To write functions on natural numbers, we also need recursion. If we in

When people wrote recursive functions in the teaching languages, it was always in connection with **define**, so that the function could even have a name.

We can transform the simple **define function** into a combination of abstraction and application. set. From a program of this form:

```
(define ne) ...
```

Can we run the following program without the **define statement** :

```
((lambda (n) ...) e)
```

However, this trick doesn't work for recursion, since according to the rules of lexical binding, the `n` in `e` isn't visible at all. We'll show how it works anyway using the example of...

Faculty. A definition like this would be nice:

$$\text{fac} \stackrel{\text{def}}{=} \lambda x. \text{if} \ (\text{zerop } x) \ \lambda 1 \lambda y \ (\lambda x \ (\text{fac} \ (\text{pred } x)))$$

* stands for a λ -term that multiplies Church numerals. We pretend we already have them.

defined – its formulation is part of exercise 14.9.

Unfortunately, that's not a proper definition for `fac`: `fac` appears on both the left and the right-hand side. If `fac` is removed from the right-hand side, the following term remains:

$$\lambda x. \text{if} \ (\text{zerop } x) \ \lambda 1 \lambda y \ (\lambda x \ (\text{?} \ (\text{pred } x)))$$

At least this expression correctly calculates the factorial of 0, namely 1. For all numbers greater than

The expression for 0 is still unknown. Because the term is only valid for 0, we call it `fac0`:

$$\text{fac0} \stackrel{\text{def}}{=} \lambda x. \text{if} \ (\text{zerop } x) \ \lambda 1 \lambda y \ (\lambda x \ (\text{?} \ (\text{pred } x)))$$

It would be nice to have a formula that can at least calculate the factorial of 1.

To do this, `fac0` is inserted into its own definition in place of the question mark. The result looks like this:

$$\lambda x. \text{if } (\text{zerop } x) \lambda y \lambda y' (y \times (\text{fac0 } (\text{pred } x)))$$

Since `Dafac0` contains no self-reference, its definition can be used; the result should be called `fac1` according to the function :

$$\text{fac1} \stackrel{\text{def}}{=} \lambda x. \text{if } (\text{zerop } x) \lambda y \lambda y' (y \times ((\lambda x. \text{if } (\text{zerop } x) \lambda y \lambda y' (y \times (? (\text{pred } x)))) (\text{pred } x)))$$

In the same way, a term can be constructed that calculates the factorials up to 2:

$$\text{fac2} \stackrel{\text{def}}{=} \lambda x. \text{if } (\text{zerop } x) \lambda y \lambda y' (y \times (\text{fac1 } (\text{pred } x)))$$

This pattern can be continued indefinitely. Unfortunately, it never results in a term that

It is possible to calculate the factorials of *all* natural numbers, since the terms always remain finitely large.

After all, the `facn` terms only differ in the call to `facn-1`, which is executed every time

It is different. We therefore abstract and define the following term `FAC`:

$$\text{FAC} \stackrel{\text{def}}{=} \lambda \text{fac}. \lambda x. \text{if } (\text{zerop } x) 1 (y \times (\text{fac } (\text{pred } x)))$$

Now we can describe the `facn` functions more easily using `FAC`:

$$\begin{aligned} \text{fac0} &\stackrel{\text{def}}{=} \lambda x. \text{if } (\text{zerop } x) \lambda y \lambda y' (y \times (? (\text{pred } x))) \\ \text{fac1} &\stackrel{\text{def}}{=} \text{FAC } \text{fac0} \\ \text{fac2} &\stackrel{\text{def}}{=} \text{FAC } \text{fac1} \\ &\dots \end{aligned}$$

`FAC` is therefore a factory for factorial functions and shares with all `faci` the property that its

The definition is not recursive. While this has made the notation more compact, a correct definition of `fac` would have to contain an infinite chain of applications of `FAC`. Since

If such a term cannot be written down, only wishful thinking will help. Let us assume that...

`fac` has already been found. Then the following equation applies:

$$\text{fac } y \text{ FAC } \text{fac}$$

The single additional application that `FAC` performs ends up on an already infinite stack, thus not increasing its size. Therefore, `fac` is a so-called *fixed point* of

FAC: If fac goes in, it comes out the same way. If there were a way to find a fixed point for a λ -term, the problem would be solved. The following sentence shows that this is indeed possible:

Theorem 14.14 (Fixed-Point Theorem) For every λ -term F there exists a λ -term X with $FX \lambda X$.

Proof: Choose $X \stackrel{\text{def}}{=} YF$, where

$$Y \stackrel{\text{def}}{=} \lambda f. (\lambda x. f (xx)) (\lambda x. f (xx)).$$

Then the following applies:

$$\begin{aligned} YF &= (\lambda f. (\lambda x. f (xx)) (\lambda x. f (xx))) F \\ &\lambda (\lambda x. F (xx)) (\lambda x. F (xx)) \\ &\lambda (\lambda x. F ((\lambda x. F (xx)) (\lambda x. F (xx)))) \\ &\lambda (\lambda x. F ((\lambda f. (\lambda x. f (xx)) (\lambda x. f (xx))) F)) \\ &= F (YF) \end{aligned}$$

The λ -term Y , which calculates fixed points, is called *the fixed-point combinator*. With its help, the Define faculty:

$$\text{fac} \stackrel{\text{def}}{=} Y \text{ FAC}$$

Figure 14.2 shows how the factorial of 3 is calculated using this definition.

There are still elements of programming languages that we will not cover in this chapter. The λ -calculus has been used to represent, for example, strings or signatures. All of this is possible in principle. This makes the λ -calculus a useful model for programming and for... The basis of large parts of programming language research.

In other areas of computer science, an alternative model for programming is common: the so-called *Turing machine*. While it functions quite differently, it is equally capable of representing all the functions of a "real" programming language. Accordingly, the λ -calculus and the Turing machine is based on the same fundamental concept of *computable functions*, which is found in the theoretical computer science plays an important role.

fac $\tilde{y}3\tilde{y} = Y \text{ FAC } \tilde{y}3\tilde{y}$

(Theorem 14.14) $\tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\text{FAC } (Y \text{ FAC}) \tilde{y}3\tilde{y})$

$\tilde{y}\tilde{y} (\tilde{y}x.\text{if } (\text{zerop } x) \tilde{y}1\tilde{y} (\tilde{y} \times ((Y \text{ FAC}) (\text{pred } x)))) \tilde{y}3\tilde{y} \tilde{y}\tilde{y} \text{ if } (\text{zerop } \tilde{y}3\tilde{y}) \tilde{y}1\tilde{y} (\tilde{y} \tilde{y}3\tilde{y} ((Y \text{ FAC}) (\text{pred } \tilde{y}3\tilde{y}))) \tilde{y}\tilde{y}\tilde{y} \text{ if false } \tilde{y}1\tilde{y} (\tilde{y} \tilde{y}3\tilde{y} ((Y \text{ FAC}) \tilde{y}2\tilde{y})) \tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} ((Y \text{ FAC}) \tilde{y}2\tilde{y}))$

(Theorem 14.14) $\tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\text{FAC } (Y \text{ FAC})$

$\tilde{y}2\tilde{y}) \tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} ((\tilde{y}x.\text{if } (\text{zerop } x) \tilde{y}1\tilde{y} (\tilde{y} \times ((Y \text{ FAC}) (\text{pred } x)))) \tilde{y}2\tilde{y}) \tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\text{if } (\text{zerop } \tilde{y}2\tilde{y}) \tilde{y}1\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} ((Y \text{ FAC}) (\text{pred } \tilde{y}2\tilde{y}))) \tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\text{if false } \tilde{y}1\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} ((Y \text{ FAC}) \tilde{y}1\tilde{y}))) \tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} ((Y \text{ FAC}) \tilde{y}1\tilde{y})))$

(Theorem 14.14) $\tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} (\text{FAC } (Y \text{ FAC})$

$\tilde{y}1\tilde{y})) \tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} ((\tilde{y}x.\text{if } (\text{zerop } x) \tilde{y}1\tilde{y} (\tilde{y} \times ((Y \text{ FAC}) (\text{pred } \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} (\text{if false } \tilde{y}1\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} ((Y \text{ FAC}) 0)))) \tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} ((Y \text{ FAC}) 0))))$

(Theorem 14.14) $\tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} (\text{FAC } (Y \text{ FAC}) 0)))$

$\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} ((\tilde{y}x.\text{if } (\text{zerop } x) \tilde{y}1\tilde{y} (\tilde{y} \times ((Y \text{ FAC}) (\text{pred } x)))) 0))) \tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} (\text{if } (\text{zerop } 0) \tilde{y}1\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} ((Y \text{ FAC}) (\text{pred } 0)))) \tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} (\text{if true } \tilde{y}1\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} ((Y \text{ FAC}) (\text{pred } 0)))) \tilde{y}\tilde{y}\tilde{y} \tilde{y} \tilde{y}3\tilde{y} (\tilde{y} \tilde{y}2\tilde{y} (\tilde{y} \tilde{y}1\tilde{y} \tilde{y}1\tilde{y})) \tilde{y}\tilde{y}\tilde{y} \tilde{y}6\tilde{y})$

Figure 14.2: Calculation of the factorial of 3 in the λ-calculus

EXERCISES

Task 14.8

Prove that **pred** calculates the predecessor of a positive Church numeral! Show:

$$\text{pred}(\text{succ } \tilde{y}n\tilde{y}) \tilde{y}\tilde{y}\tilde{y} \tilde{y}n\tilde{y}$$

Task 14.9

Proof that there are λ-terms for the following operations on Church numerals:

$$\begin{aligned}
 \text{add } m \tilde{y} n \tilde{y} &= \tilde{y} m + n \tilde{y} \\
 \text{mult } m \tilde{y} n \tilde{y} &= \tilde{y} m n \tilde{y} \\
 \text{exp } m \tilde{y} n \tilde{y} &= \tilde{y} m^n \tilde{y} \text{ for } m > 0 \\
 &= \tilde{y} m \tilde{y} n \tilde{y} = \begin{cases} \tilde{y} & \text{true if } m = n \\ \tilde{y} & \text{false otherwise} \end{cases}
 \end{aligned}$$

Use the following definitions:

$$\begin{aligned}
 \text{add} &\stackrel{\text{def.}}{=} \tilde{y} x. \tilde{y} y. \tilde{y} p. \tilde{y} q. x p \quad (y p q) \\
 \text{multi} &\stackrel{\text{def.}}{=} \tilde{y} x. \tilde{y} y. \tilde{y} z. x \quad (y z) \\
 \text{exp} &\stackrel{\text{def.}}{=} \tilde{y} x. \tilde{y} y. y x
 \end{aligned}$$

and provide your own definition for $=$. The correctness of add can be proven directly.

For mult and exp you can use the following auxiliary equations, which you should also prove if possible:

$$\begin{aligned}
 (\tilde{y} n \tilde{y} x) m \tilde{y} &\tilde{y} x n m y \\
 \tilde{y} n \tilde{y} m x \tilde{y} \tilde{y} n m \tilde{y} &\text{ for } m > 0
 \end{aligned}$$

Task 14.10

The Y-combinator can also be translated literally into the languages of instruction. This results in the following definition:

```

(define y
  (lambda (f)
    ((lambda (x) (f (xx)))
     (lambda (x) (f (xx))))))

```

Try this definition, for example with:

```

(define FAC
  (y
   (lambda (fac)
     (lambda (n)

```

```

(cond
  ((zero? n) 1)
  ((positive? n)
   (* n (fac (- n 1))))))
(FAC 3)

```

You will see that y doesn't work with this definition. Why is that? Use the definition of call-by-value reduction to explain!

Show that the following variant of y is a fixed-point combinator that works:

```

(define y
  (lambda (f)
    ((lambda (x) (f
      (lambda (y) ((xx) y))))
     (lambda (x)
      (f (lambda (y) ((xx) y)))))))

```

Task 14.11

(Source: Ralf Hinze) Show that F is also a fixed-point combinator with the following definition. is:

```

F   $\stackrel{\text{def}}{=} G[26]$ 
G   $\stackrel{\text{def}}{=} \text{yabcdefghijklmnopqstuvwxyzr.}r(\text{this is a fixed point combinator})$ 

```

Here, $G[26]$ represents the lambda term, which is formed by writing G 26 times consecutively, i.e.,
 $GG \dots G = (\dots ((GG)G) \dots G).$

15 EXCURSUS: THE SECD MACHINE

This chapter builds upon the λ -calculus from the previous chapter. The λ -calculus was developed as a theoretical model for computable functions long before the invention of the computer. While it forms the basis for our teaching languages, it doesn't explain how these actually run on a "real" computer. Modern microprocessors are programmed in a *machine language* that differs significantly from the λ -calculus. For our programs to run on such a processor, they must be translated into machine language using a *compiler*.

In this chapter, we describe an idealized machine that was never actually built, but whose machine language is based on the same principles as real machines – the *SECD machine*. We then write a compiler from the λ -calculus into the SECD machine language, and also a so-called *virtual machine* that allows us to run the compiled programs.

15.1 THE APPLIED λ -CALCULATION

Section 14.8 on page 455 already showed that Boolean values and numbers can also be represented in the λ -calculus using λ -terms. While this is good to know from a theoretical perspective, it's too cumbersome in practice to always work with Church numerals. Therefore, we extend the λ -calculus with some extras so that it can directly handle numbers and Booleans. Hopefully, Section 14.8 has convinced you that we could always do without the new elements by translating them into the terms from Section 14.8.

The resulting extended λ -calculus is called *the applied λ -calculus*:

Definition 15.1 (Language of applied λ -calculus $L\lambda A$)

Let V be a countable set of variables, as in the λ -calculus.

Let B be a set of *underlying assets*. The underlying assets include at least the Boolean ones. Values and the natural numbers, that is:

$$B \stackrel{\text{def}}{=} \{\#f, \#t, 0, 1, 2, \dots\}$$

Let γ be the "built-in operations" for i a lot of *i-digit primitives* – the names of each natural number i . Each $F \gamma \gamma$ is assigned F op is an i -digit function – its *operation* – a function.

For example, addition could be a primitive operation $+ \in \lambda$. The operation for that were:

$$+ \text{ op}(a, b) \stackrel{\text{def}}{=} a + b$$

The language of applied λ -calculus, the set of *applied λ -terms*, $L\lambda A$, is defined by the following grammar:

$\lambda L\lambda A ::= \lambda V \lambda \mid (\lambda L\lambda A)$

$\lambda L\lambda A ::= (\lambda V \lambda \lambda L\lambda A)$

$\mid \lambda B \lambda$

$\mid (\lambda \lambda^1 \lambda L\lambda A)$

$\mid (\lambda \lambda^2 \lambda L\lambda A \lambda L\lambda A)$

...

$\mid (\lambda \lambda^n \lambda L\lambda A \dots \lambda L\lambda A) \text{ (n times)}$

The grammar is written in abbreviated form: The last clauses state that for every set of primitives $(\lambda e_1 \dots \lambda^i \lambda$ with arity i , there exists a corresponding clause with i operands. Terms of the form $(\lambda^i \lambda e_i)$, there are called *primitive applications*. In the applied λ -calculus, therefore, terms such as for example $(+ (\lambda^5 \lambda 3) 17)$ possible if $+$ and λ in λ are included.

Definition 15.2 (Values in the applied λ -calculus)

The union of abstractions and basic values is called *values*.

In order for the primitive operations to actually have meaning, a special reduction rule must be introduced for them: **Definition 15.3 (λ -**

reduction)

$$(\lambda^i \lambda e_1 \dots e_i) \lambda \lambda F \text{ op}(e_1, \dots, e_k) e_1, \dots, e_i \lambda B$$

This rule states that a primitive application, if all operands are values, can be reduced by applying the appropriate operation. For example, the example term above is reduced as follows:

$$(+ (\lambda^5 \lambda 3) 17) \lambda \lambda (+ (\lambda^2 \lambda 17) \lambda \lambda 19)$$

Task 15.1

It is not possible to describe branching using a delta rule. Why? To answer.

To answer this question, first write a possible definition of the corresponding function F .

if hin. Be-

I now think that the delta reduction only takes effect if all arguments of the `if` statement are values. Could you

Can you use this if statement to define, for example, the factorial from section 14.8.3 on page 458 ? $\tilde{y}$

In the following sections, we will discuss the SECD engine and compiler for the applied application.

Define the $\tilde{y}$ -calculus without binary branches. We already know how to define them using the methods from Section 14.8.1 on page 456 will allow you to define binary branches.

This can be implemented directly as a supplement in a series of practice exercises. Here is the first one:

Task 15.2

1. Extend the extended $\tilde{y}$ -calculus by adding a grammar clause for binary branching.
2. Define a reduction rule for binary branches, analogous to delta reduction. (You can
They call it $\tilde{y}$ -reduction – $\tilde{y}$ is the Greek letter «iota».)

$\tilde{y}$

15.2 THE SECD MACHINE

Unfortunately, the $\tilde{y}$ -calculus does not run directly on standard processors. Instead, the-

These processors use what is called *machine language*. Machine language consists of individual instructions that are stored sequentially in memory and executed sequentially.

This is what distinguishes machine language from our teaching languages, which are based on the call-by-value calculus: there, the evaluation runs from the inside out.

To bridge the gap between these two ideas, Peter Landin developed the SECD machine [Lan64]. Although the SECD machine is over 50 years old, it can also be used on modern machines.

still the basis for the interaction between programming language and machine language. The language of the SECD machine is an idealized machine language that also consists of

The SECD machine consists of instructions that are executed sequentially. However, it omits many details of true machine languages.

Definition 15.4 (Language of the SECD machine)

The instructions of the SECD machine are called *instructions* and are defined by the following grammar. defined:

$\lambda\lambda\lambda\lambda\lambda\lambda\lambda\lambda$

| $\lambda V\lambda$

| ap

| primF i for all F $\lambda\lambda\lambda\lambda\lambda\lambda\lambda\lambda$

| $(\lambda V\lambda, \lambda C\lambda)$

A machine language program – also called *machine code* or *code* – is a sequence of Instructions:

$$C \stackrel{\text{def}}{=} I^*$$

The SECD machine, however, operates on *so-called machine states*. A machine state is a 4-tuple from the set $S \times E \times C \times D$ (hence the name of the machine). The letters are chosen this way because *S* is the so-called *stack*, *E* is the so-called *environment*, respectively. In English, *C* is the *environment*, *C* is the already familiar code, and *D* is the so-called *dump*. The formal definitions of these sets are as follows; where *W* is the set of values:

$$\begin{aligned} S &\stackrel{\text{def}}{=} W^* \\ E &\stackrel{\text{def}}{=} V \times W \\ D &\stackrel{\text{def}}{=} (S \times E \times C) \times W \\ W &\stackrel{\text{def}}{=} B \times W \times (V \times C \times E) \end{aligned}$$

The stack is a sequence of values. In the machine, these are the values of the most recently evaluated values. Terms, where the last evaluated term is at the front or "top". The environment This is a partial mapping of variables to values: it replaces the substitution in the reduction relation of the λ -calculus. Instead of values being substituted for variables, the environment simply remembers which values the variables are bound to. Only when the value of a variable When needed, the machine retrieves it from the surrounding area.

The dump is a list of previous states of the machine: Each state is a tuple of Stack, environment, and code, also called *frame*. A dump stores the context, the as we encountered in section 10.5 on page 306.

Finally, the set *W* corresponds to the value concept from Definition 15.2: It includes the basic values, as well as 3-tuples of $(V \times C \times E)$. Such a triple, called a *closure*, represents the value of an abstraction. A closure consists of the variable of an abstraction, the machine code of its body, and the environment necessary to apply the abstraction:

The environment is needed so that the free variables of the abstraction can be evaluated according to the lexical binding. This differs from the stepper, where variables are assigned during the application will be directly replaced and thus disappear.

The SECD machine transforms a machine state into a single evaluation step. a new machine condition.

For environments, we use the notation $e(v)$: This stands for the value w of the tuple (v, w) in e .

During the evaluation process, environments are often extended with new bindings from a variable to a value. The notation $e[v \dot{\rightarrow} w]$ is useful for this. $e[v \dot{\rightarrow} w]$ is constructed from An environment e is a new environment in which the variable v is bound to the value w . Here is the definition:

$$e[v \dot{\rightarrow} w] \stackrel{\text{def}}{=} (e \setminus \{(v, w\}) \dot{\rightarrow} (v, w)) \dot{\rightarrow} \{(v, w)\}$$

First, any existing old bond is removed, and then a new one is added.

Note: We already used the notation $e[v \dot{\rightarrow} w]$ in the last chapter, namely for the substitution in the $\dot{\lambda}$ -calculus in Definition 14.3 on page 446. In this context – in the SECD machine – does the notation mean something else? Don't be confused!

To understand the machine, it is helpful to first see how $\dot{\lambda}$ -terms are converted into SECD code. A term e from the applied $\dot{\lambda}$ -calculus is translated into a machine language program e using the function.

e	$\stackrel{\text{def}}{=}$	$\dot{\lambda} b$	if $e = b \dot{\rightarrow} B$
		v	if $e = v \dot{\rightarrow} V$
		$e_0 e_1 \text{ ap}$	if $e = (e_0 e_1)$
		$e_1 \dots e_k \text{ primF } i$	if $e = (F e_1 \dots e_i)$
		(v, e_0)	if $e = \dot{\lambda} v. e_0$

For example, the translation $e_0 e_1 \text{ ap}$ for a term $(e_0 e_1)$ means that e_0 and e_1 are separate . The instructions for both translations will be linked together, so that... The SECD machine also executes them sequentially later, i.e., first evaluates e_0 , then e_1 . How As we will see, the SECD machine will display the results of the evaluation of e_0 and e_1 on the Place the stack. The instruction ``ap``, which is added later, then ensures that the SECD machine takes the results of ``e0`` (which must be the function) and the result of ``e1`` from the stack.

It retrieves the data and applies one thing to the other. Accordingly, `ap` stands for "execute application". Furthermore, `primF k` stands for "execute primitive `F`".

Basic values and variables are translated directly into machine code. An abstraction is performed. translated into a tuple consisting of its variable and the machine code for its body. All these instructions leave their result on the stack.

Here is an example of the translation:

$$\begin{aligned} \lambda f. \lambda x. \lambda y. f (+ x (\lambda y. y 2)) &= (f, \lambda x. \lambda y. f (+ x (\lambda y. y 2))) = (f, (x, \lambda y. f (+ x (\lambda y. y 2)))) = \\ &= (f, (x, (y, f (+ x (\lambda y. y 2))))) = (f, (x, (y, f (+ x (\lambda y. y 2)) ap))) = (f, (x, (y, f (+ x (\lambda y. y 2)) ap))) = (f, \\ &= (x, (y, f x (\lambda y. y 2) prim+ ap))) = (f, (x, (y, f x (\lambda y. y 2) prim+ ap))) = (f, (x, (y, f x y 2 prim\ddot{y} prime+ \\ &= ap))) = (f, (x, (y, f x y 2 prim\ddot{y} prim+ ap))) = (f, (x, (y, \\ &= f x y 2 prim\ddot{y} prim+ ap))) \end{aligned}$$

The example clearly shows how the body of the innermost abstraction is translated into a sequence of instructions that corresponds in order to the call-by-value reduction strategy: first evaluate `f`, then `x`, then `y`, then apply the primitive `*`, then `+`, and finally perform the application.

Task 15.3

Translate the following λ -terms into the intermediate representation of the SECD machine:

- $(\lambda xy. (+ xy)) (\lambda y. 5) 23$
- $(\lambda x. (\lambda y. x)) (\lambda xy. (&\& xy)) ((\lambda xy. (> xy)) 23) 42$
- $\text{true } 3. (\lambda xy. yxx) (\lambda z. z) (\lambda yz. zy) (yy) (yz)$

Here, `!` represents the boolean `not` and `&&` represents the boolean `and`.

$\ddot{y}$

We now define the SECD machine itself. This always takes one step at a time and

It transforms states `S`, `E`, `C`, and `D` into new states by executing the first instruction in `c`. We write this transformation with the symbol $\ddot{y}$, i.e.,

- Rule 15.1 (the *literal rule*) pushes a base value directly onto the stack.
- Rule 15.2 (the *variable rule*) determines the value of a variable from the environment and shifts it this onto the stack.
- Rule 15.3 is the *primitive rule*. In a primitive application, this many basic values must be used.

The primitive rule determines the arity of the primitive, which lies at the top of the stack.

The result of the primitive application is pushed to the top of the stack.
- Rule 15.4 is the *abstraction rule*: The tuple $(v, c\bar{y})$ arose from an abstraction during translation. The rule adds v and c . $\frac{}{\text{---}}$ with e to a closure that is pushed onto the stack becomes.
- Rule 15.5 is the *application rule*: In an application, a value must be placed at the top of the stack. as well as a closure. (As a reminder: An application can only be evaluated if...)

(when an abstraction is present. Abstractions are evaluated into closures.) In such a case, the application "saves" the current state to the dump, and the evaluation proceeds with an empty stack, the environment from the closure – extended by a binding. for the variable – and the code from the closure.

- Rule 15.6 is the *return rule*: It applies when the end of the code is reached.

This means that the evaluation of an application has just finished. However, there is still data on the dump, a secure state that is now being "brought back".

Here is an example of the SECD machine's operation for the term $((\tilde{y}x.\tilde{y}y.(+ xy)) 1) 2$:

$\tilde{y},$	$\tilde{y},$	$(x.(y, xy \text{ prime}+)) 1 \text{ ap } 2 \text{ ap}, \tilde{y})$	
$\tilde{y} ((x.(y, xy \text{ prim}+), \tilde{y}), \tilde{y} (1 (x,$	$\tilde{y},$	$1 \text{ ap } 2 \text{ ap},$	$\tilde{y})$
$(y, xy \text{ prim}+), \tilde{y}), \tilde{y} (\tilde{y}, \tilde{y} ((y, xy$	$\tilde{y},$	$\text{ap } 2 \text{ ap}, \tilde{y})$	
$\text{prim}+,$	$\{(x, 1)\},$	$(y, xy \text{ prime}+), (\tilde{y}, \tilde{y}, 2 \text{ ap}))$	
$\{(x, 1)\}), \tilde{y} ((y, xy \text{ prim}+, \{(x, 1)\}),$	$\{(x, 1)\},$	$\tilde{y},$	$(\tilde{y}, \tilde{y}, 2 \text{ ap}))$
$\tilde{y}, \tilde{y} (2 (y, xy \text{ prime}+, \{(x, 1)\}), \tilde{y}, \tilde{y} (\tilde{y}, \tilde{y}$		$2 \text{ ap},$	$\tilde{y})$
$(1, \tilde{y} (2 1, \tilde{y} (3, \tilde{y} (3,$		$\text{ap},$	$\tilde{y})$
	$\{(x, 1).(y, 2)\}, xy \text{ prim}+, \{(x, 1).(y,$		$(\tilde{y}, \tilde{y}, \tilde{y}))$
	$2)\}, y \text{ prim}+, \{(x, 1).(y, 2)\},$		$(\tilde{y}, \tilde{y}, \tilde{y}))$
	$\text{prim}+, \{(x, 1).(y, 2)\}, \tilde{y},$		$(\tilde{y}, \tilde{y}, \tilde{y}))$
			$(\tilde{y}, \tilde{y}, \tilde{y}))$
	$\tilde{y},$	$\tilde{y},$	$\tilde{y})$

Task 15.4

Consider the following SECD program:

$((f(x.(y, fx \text{ ap } y \text{ ap}))) (a.(b, ab \text{ prim}+)) \text{ ap } 23 \text{ ap } 42 \text{ ap}$

1. Translate the SECD program "backwards" into the corresponding L $\tilde{y}$ A term.
2. Evaluate the SECD program and write down the individual evaluation steps! $\tilde{y}$

The state transition relation $\tilde{y}$ is the basis for *the evaluation function* of the SECD machine, which calculates the meaning of a $\tilde{y}$ -term. This seems quite simple:

$\text{evalSECD} : \text{L}\tilde{y}\text{A } \tilde{y} B$

$\text{evalSECD}(e) = x \text{ if } (\tilde{y}, \tilde{y}, e, \tilde{y}) \tilde{y}\tilde{y} (x, e, \tilde{y}, \tilde{y})$

However, this definition has two drawbacks:

- The evaluation of $\tilde{y}$ -terms does not always terminate (as for example for the «endless term» $(\tilde{y}x.(xx)) (\tilde{y}x.(xx))$), so the state transition relation does not always terminate at a state of the form $(\tilde{y}, \tilde{y}, e, \tilde{y})$.
- The x in this definition is not always a base value – it can also be a closure.

The first drawback is that the evaluation function is only a "partial function". The second issue, if x is a closure, is that the result isn't very useful: To determine the exact meaning of the closure, it would have to be applied – but the program has already finished running. Therefore, it's not practical to distinguish between different closures. For this reason, a set Z of *answers* is defined for the purposes of the evaluation function, containing a designated special value for closures:

$$Z \stackrel{\text{def}}{=} B \cup \{\text{function}\}$$

This allows us to define the evaluation function as follows:

$$\begin{aligned} \text{evalSECD } \tilde{y} \tilde{L} \tilde{A} \times Z \tilde{y} b \\ \text{evalSECD } (e) = \begin{cases} \text{if } (\tilde{y}, \tilde{y}, e, \tilde{y}) \tilde{y} \tilde{y} (b, e, \tilde{y}, \tilde{y}) \\ \tilde{y} \text{ function if } (\tilde{y}, \tilde{y}, e, \tilde{y}) \tilde{y} \tilde{y} ((v, c, e\tilde{y}), e, \tilde{y}, \tilde{y}) \tilde{y} \end{cases} \end{aligned}$$

Of course, we also want to see this function running. To do this, we will program the mathematical definitions in the rest of the chapter.

Task 15.5

Continue with the implementation of binary branches:

1. Extend the grammar for SECD instructions to include an instruction for binary two-
The code must contain the code for the two branches of the if expression.
2. Extend the function with a clause for if expressions: It should first include the code
Provide the condition, followed by the new instruction.
3. Extend the definition of $\tilde{y}$ to include the new instruction.

$\tilde{y}$

15.3 QUOTES AND SYMBOLS

`secd/compute.rkt``code`

To program the definitions as elegantly as possible, we use an additional language level: Write Your Program! - Advanced. This level must be activated via the DrRacket menu under Language > Select Language for the programs in this chapter to work.

The key change compared to previous language levels is how REPL expresses values. For numbers, strings, and Boolean values, everything remains the same:

```
5
```

```
ŷ 5
```

```
"Mike is stupid"
```

```
ŷ "Mike is stupid"
```

```
#t
```

```
ŷ #t
```

However, the situation is different with lists:

```
(list 1 2 3 4 5 6)
```

```
ŷ (1 2 3 4 5 6)
```

The REPL prints a list by first printing an opening parenthesis, then the list items (separated by spaces), and then a closing parenthesis. This also works for an empty list:

```
empty
```

```
ŷ ()
```

In the new language level, the apostrophe can form literals for lists:

```
'(1 2 3 4 5 6)
```

```
ŷ (1 2 3 4 5 6)
```

```
'(1 #t "Mike" (2 3) "stupid" 4 #f 17) ŷ (1 #t "Mike" (2 3)
```

```
"stupid" 4 #f 17)
```

```
'()
```

```
ŷ ()
```

In the new language level, literals and printed external representations for lists use the same notation. They differ only in that the literal is preceded by an apostrophe. The apostrophe also works with numbers, strings, and Boolean values.

```
'5
```

```
ŷ 5
```

```
""Mike is stupid"
```

```
ŷ "Mike is stupid"
```

```
'#t
```

```
ŷ #t
```

The apostrophe at the beginning of an expression marks it as a literal. The value of the literal is printed exactly as it appears in the program (apart from spaces and line breaks). The apostrophe is called a "quote" in English, and that's why this notation is used.

Also known as a quotation mark for literals. From now on, we will refer to this apostrophe as "the quota". With numbers, strings, and Boolean literals, it is clear even without the quotation that they are...

These are literals. Therefore, the quotation is purely optional for them; they are called *self-quoting*.

However, with lists, misunderstandings with other compound forms are possible.
which also begin with an opening parenthesis.¹

With the introduction of quotes, a new type of value is added: *symbols*. Symbols are values similar to strings and consist of text. However, they differ in several ways.

by writing them as literals with quotes instead of quotation marks and in the REPL without Quotation marks will be printed:

```
'mike
```

```
↳ Mike
```

We can distinguish symbols from other values using the predicate " `symbol?` " :

```
(symbol? 'mike)
```

```
↳ #t
```

```
(symbol? "Mike")
```

```
↳ #f
```

Symbols are also the reason why lists are not self-quoting. Otherwise, it would be unclear whether `(run-over-dillo d1)` results in a literal for a list (with the symbols).

`(run-over-dillo` and `d1` as elements) or the call to the function `run-over-dillo`. That's why a quote is necessary.

Can we compare symbols using the built-in `equal` function? (See Figure 15.1.)

```
(equal? 'mike 'herb)
```

```
↳ #f
```

```
(equal? 'mike 'mike)
```

```
↳ #t
```

¹ In fact, the new notation for external representations is the standard representation in Racket. Earlier language levels used the alternative spelling to avoid confusion between list literals and to avoid compound forms.

The `equal?` function compares any values: Boolean characters, characters, and strings. For lists, it compares the elements, and for records, it compares the components.

```
(equal? 23 23) ⇒ #t

(equal? #t #f) ⇒ #f
(equal?
 "Axl" "Slash") ⇒ #f (equal? "Axl" "Axl")

⇒ #t
(equal? (list "Axl" "Slash") (list "Axl" "Slash"))
⇒ #t
(equal? (list "Axl" "Slash") (list "Axl" "Slash" "Duff")) ⇒ #f

(equal? (make-wallclock-time 12 24) (make-wallclock-time 12 24)) ⇒ #t

(equal? (make-wallclock-time 12 24) (make-wallclock-time 12 25)) ⇒ #f

(equal? (make-wallclock-time 12 24) (list "Axl" "Slash")) ⇒ #f
```

Figure 15.1: `equal?`

Symbols cannot consist of arbitrary text; spaces, for example, are forbidden. In fact, the names of the permitted symbols correspond exactly to the names of variables:

```
'Karl-Otto
⇒ Karl Otto

'lambda
⇒ lambda

'+
⇒ +

'*
⇒ *
```

This correspondence will play a crucial role later in this chapter. Symbols can, of course, also appear in lists and therefore in list literals:

```
('Karl-Otto VAT fragrance brand)
⇒ (Karl-Otto VAT fragrance brand)
```

Using symbols, values can be constructed that, when printed in the REPL, look like expressions:

```
'(+ 1 2)
ȳ (+ 1 2)
'(lambda (n) (+ n 1)) ȳ (lambda
(n) (+ n 1))
```

The value of `'(+ 1 2)` is a list with three elements: the symbol `+`, the number `1` and the number `2`. The value of `'(lambda (n) (+ n 1))` is also a list with three elements: the symbol `lambda`, a list with a single element, namely the symbol `n`, and another list with three elements: the symbol `+`, the symbol `n` and the number `1`.

Quote has another confusing characteristic:

```
"()
ȳ '()
```

This literal does not denote the empty list (then only `()` would be printed, without a quote):

```
(cons? "()")
ȳ #t
(first "()")
ȳ quote
(rest "()")
ȳ ()
```

The value of the expression `"()` is therefore a list with two elements: the first is the **quote** symbol and the second is the empty list. `t` itself is merely syntactic sugar for `(quote t)`:

```
(equal? (quote ()) '())
ȳ #t
(equal? (quote (quote ())) "()")
ȳ #t
```

Quote works for many values, but not all. A value for which Quote can construct a literal is called *representable*. A representable value is one of the following:

- a number
 - a boolean value
 - a string
 - a symbol • a
- list of representable values

To demonstrate the usefulness of symbols and quotes, we'll program something like a calculator. It can calculate the value of expressions like `(+2365)`. We'll use the following data definition:

An expression is one of the following: - a number; - a list of the form `(+ expression expression)`; - a list of the form `(* expression expression)`

As indicated by the phrase "one of the following," this is mixed data. To define a suitable signature, we need signatures for all cases. For the first case, `number` suffices. For the other two, it's not quite so simple because there's no `define-record` that automatically defines the signature for us. Therefore, we have to define the signature ourselves, and we do this using predicates for the two cases, which we write ourselves.

As a reminder: A predicate is a function that accepts any value, tests for a specific property, and returns either `#t` or `#f` accordingly. We need something like this for the two cases that begin with "a list of the form". First, addition:

Is a value an addition expression? (`:` `addition?` (`any -> boolean`))

```
(check-expect (addition? '5) #f) (check-expect
(addition? '(+ 1 2)) #t) (check-expect (addition? '(* 1 2)) #f)
```

```
(define addition? (lambda
(x)
  (and (cons? x)
       (equal? '+ (first x)))))
```

For the predicate `"addition?"`, we only programmed the bare essentials to distinguish additions from the other cases:

Task 15.6

The data definition above states "a list of the form `(+ expression expression)`".

Are there any inputs that return `#t` as a result when `addition?` is used, even though they are not lists of exactly this form? If so, give examples and change the definition of `addition?` to exclude these examples.

In a signature of the form

(predicate p)

p must be a predicate, i.e., a function with the following signature:

(any -> boolean)

This signature fits all values where the predicate returns #t .

Figure 15.2: predicate

From the predicate `addition?` we can create a signature `addition` using a signature with `predicate`. The `predicate` construct is new and described in Figure 15.2. Here is the definition of `addition`:

```
(define addition
  (signature (predicate addition?)))
```

We do the same for multiplications:

```
Is a value a multiplication expression? (: multiplication? (any ->
boolean))
```

```
(check-expect (multiplication? '5) #f) (check-expect
(multiplication? '(* 1 2)) #t) (check-expect (multiplication? '(+ 1 2))
#f)
```

```
(define multiplication?
  (lambda (x)
    (and (cons? x)
         (equal? * (first x))))) (define
multiplication (signature (predicate
multiplication?)))
```

Now we have signatures for the three cases and can therefore define a signature for expressions. kidneys:

```
(define expression
  (signature (mixed number addition multiplication)))
```

Now we can program the "calculator". Here is a brief description, signature, and test cases:

; Calculate the value of an expression (:

```
compute (expression -> number))
```

```
(check-expect (compute '23) 23) (check-  
expect (compute '(+ 23 42)) 65) (check-expect (compute  
'(+ 23 (* 6 7))) 65)
```

The functional definition strictly follows the design instructions. Here is the template:

```
(define compute  
  (lambda (exp)  
    (cond  
      ((number? exp) ...)  
      ((addition? exp) ...)  
      ((multiplication? exp) ...))))
```

In the first branch, `exp` is already a number – the function just needs to return it:

```
(define compute  
  (lambda (exp)  
    (cond  
      ((number? exp) exp)  
      ((addition? exp) ...)  
      ((multiplication? exp) ...))))
```

In the `addition` case, `exp` is a list whose second and third elements are each another expression. It is therefore composite data, but not defined with a record. We extract the components using `first` and `rest`.

```
(define compute  
  (lambda (exp)  
    (cond  
      ((number? exp) exp)  
      ((addition? exp) ...  
       (first (rest exp)) ... ... (first (rest (rest  
exp))) ...)  
      ((multiplication? exp) ...))))
```

Furthermore, we are dealing with self-references in each case, so recursive calls must also be added to the template:

```
(define compute
  (lambda (exp)
    (cond
      ((number? exp) exp)
      ((addition? exp)
       ... (compute (first (rest exp))) ... (compute (first
        (rest (rest exp)))) ... ((multiplication? exp) ...))))))
```

The two recursive calls each return the value of the two operands of the addition; the function then only needs to add them together. Multiplication works the same way:

```
(define compute
  (lambda (exp)
    (cond
      ((number? exp) exp)
      ((addition? exp) (+
        (compute (first (rest exp))) (compute (first
          (rest (rest exp)))))) ((multiplication? exp) (* (compute
            (first (rest exp))) (compute
              (first (rest (rest exp))))))))))
```

Complete!

Task 15.7

Change the data definitions so that addition and multiplication each accept any number of arguments, not just two. Modify `compute` accordingly!

λ

15.4 DATA DEFINITIONS FOR THE λ -CALCULATION

secd/secd.rkt

code

The SECD machine is a model for implementing the λ -calculus. Such an implementation can be built easily – this section shows how. The general roadmap arises from the structure of the SECD machine itself: After the obligatory data definitions, we translate terms into machine code. Then comes the state transition function and finally the evaluation function.

The first task, as always, is data analysis: It begins with the terms of the applied λ -calculus. A suitable representation using lists and symbols allows the terms in the "advanced" language level to look exactly like corresponding program terms: (+ 1 2)

	stands for (+ 1 2)
(lambda (x) x)	stands for $\lambda x.x$
((lambda (x) (xx)) (lambda (x) (xx)))	stands for $(\lambda x.(xx)) (\lambda x.(xx))$
etc.	

The data definition for this is directly based on Definition 15.1:

A lambda term is one of the following: - a symbol (for a variable)
 - a two-element list (for a regular application) - a list
 of the form (lambda (x) e) (for an abstraction)

; - a base value
 ; - a list with a primitive as the first element (for a primitive application)
 ;

Here is the corresponding signature definition:

```
(define term
  (signature
    (mixed symbol
      application
      abstraction
      primitive-application base)))
```

We still need to define the signatures for ``application``, ``abstraction``, ``primitive-application``, and ``base``, just like with the expressions in the previous section. Since these are not record types, we have to do this ourselves by defining a predicate for each. Here is the predicate for ``abstraction``:

```
; Predicate for abstractions
(: abstraction? (any -> boolean)) (define abstraction?

  (lambda (term) (and
    (cons? term)
    (equal? 'lambda (first term)))))
```

We turn this predicate into a signature using "predicate". Here is the definition:

```
(define abstraction (signature (predicate abstraction?)))
```

We define the other signatures accordingly. With the "application" signature, we need to be a little careful to avoid accidentally including Primitive applications among the "normal" applications.

```
; Predicate for regular applications (: application? (any
-> boolean)) (define application? (lambda (term)
(and (cons? term)
```

```
(not (equal? 'lambda (first term))) (not (primitive?
(first term))))))
```

```
(define application (signature (predicate application?)))
```

Is the function used there primitive? It has only the most essential primitive operations:

```
A primitive is one of the symbols +, -, *, /, = ; Predicate for primitive (:
primitive? (any -> boolean)) (define
primitive? (lambda (term)
```

```
(or (equal? '+ term) (equal?
'- term) (equal? '* term)
(equal? '/' term) (equal?
' = term))))
```

The definition of primitive-application is the same as application, except that only primitive applications are desired here:

```
; Predicate for primitive applications (: primitive-
application? (any -> boolean)) (define primitive-application?
(lambda (term) (and (cons? term)
```

```
(primitive? (first term))))
```

```
(define primitive-application (signature
  (predicate primitive-application?)))
```

The definition of ``base`` is still missing. For the sake of simplicity, we will restrict ourselves to Boolean bases.

Values and figures:

A base value is a Boolean value or a number.

For this too, we first write a predicate and thus define a signature:

```
; Predicate for underlying assets
(: base? (any -> boolean)) (define base?

  (lambda (term)
    (or (boolean? term) (number? term))))

(define base (signature (predicate base?)))
```

Task 15.8

Extend the definition of ``term`` to include binary branching in the form of ``if`` expressions. First, write a predicate ``binary-conditional?`` and then define a signature ``binary-condition`` using it. Extend ``term`` to include ``binary-conditional``. y

15.5 DATA DEFINITION FOR MACHINE LANGUAGE

Before the SECD machine can process a term, it must first be translated into machine code. Definition 15.4 directly generates data and signature definitions for instructions and machine code:

An instruction is one of the following: - a base value

```
; - a variable
; - an application instruction ; - an instruction for a
primitive application
; - an abstraction
(define instruction
  (signature
```

```
(mixed base
  symbol
  ap
  prim
  abs))
```

A machine code program is a list of instructions. (define machine-code (signature (list-of instruction)))

The base values and variables are the same as with the terms. We create separate data definitions for `ap`, `prim`, and `abs`. As with empty lists, the application instruction also has no properties.

```
; Application instruction (define-record
ap make-ap ap?)
```

The instruction for the primitive application `primF i` has as properties the name of the operator `F` and its number of arguments `i`, also called *arity*. Arity is called "arity" in English. Accordingly, the data and record definitions look like this:

```
An instruction for a primitive application has the following properties: - operator; - arity
(define-record prim
make-prim prim?
(prim-operator
symbol) (prim-arity natural))
```

The abstraction instruction of the form $(\lambda V \ddot{y}, \ddot{y} C \ddot{y})$ clearly has as properties the parameter and the code for the body, which we translate directly into data and record definitions:

An abstraction instruction has the following properties:

```
; - Parameter (a variable)
; - Code for the hull
(define-record abst
make-abst abst?
(abst-variable symbol) (abst-
code machine-code))
```

Task 15.9

Extend the definition of `instruction` to include an instruction for binary branching. Write a data definition and a record definition that fit the extension of the grammar for SECD instructions. Extend `instruction` with these. $\dot{y}$

15.6 A COMPILER FOR THE SECD MACHINE

Since terms and machine code now have data definitions, we can program a translation from the applied calculus into the machine language of the SECD machine. Such a translation is called a *compiler*. Here is a short description, signature, and skeleton of the compiler:

```
; Translate term to machine code (: term->machine-
code (term -> machine-code)) (define term->machine-code
```

```
(lambda (term)
  ...))
```

Here are two test cases:

```
(check-expect (term->machine-code '(+ 1 2)) (list 1 2 (make-
  prim '+ 2)))
(check-expect (term->machine-code '((lambda
  (x) (xx)) (lambda (x) (xx))))
  (list (make-abst 'x (list 'x 'x (make-ap))) (make-abst 'x (list 'x 'x
    (make-ap))) (make-ap)))
```

Since `term` contains mixed data, a branch is necessary – as always – to...

Form the body of the function:

```
(define term->machine-code
  (lambda (term)
    (cond
      ((symbol? term) ...)
      ((application? term) ...) ((abstraction?
        term) ...) ((base? term) ...)

      ((primitive-application? term) ...)))
```

In each case, the implementation corresponds exactly to the translation function in section 15.2 on page 469. As a reminder, here are the cases for variables and base values:

$$e \stackrel{\text{def}}{=} \begin{cases} \tilde{y} \text{ b if } e = \text{b } \tilde{y} \text{ B} \\ \tilde{y} \text{ v if } e = \text{v } \tilde{y} \text{ V } \tilde{y} \end{cases}$$

The code for this is directly equivalent – we just need to remember to always produce a list, even if only a single instruction comes out:

```
(define term->machine-code
  (lambda (term)
    (cond
      ((symbol? term) (list term))
      ((base? term) (list term))
      ...)))
```

In regular applications, the translation function looks like this:

$$e \stackrel{\text{def}}{=} e_0 \text{ e1 ap if } e = (e_0 \text{ e1})$$

The operator and operand are translated, and the whole thing is combined with an ap instruction to form a list:

```
(define term->machine-code
  (lambda (term) (cond
    ...
    ((application? term)
     (append (term->machine-code (first term)) (append (term-
       >machine-code (first (rest term))) (list (make-ap))))))
    ...)))
```

In primitive applications, the translation function looks like this:

$$e \stackrel{\text{def}}{=} e_1 \dots e_k \text{ primF } i \quad \text{if } e = (\text{F } e_1 \dots e_i)$$

First, the operands are translated into machine code, the results are concatenated, and finally a priming instruction is added at the end:

```
(define term->machine-code
  (lambda (term)
    (cond
      ...
      ((primitive-application? term) (append
        (append-
          lists (map term-
            >machine-code (rest term))) (list (make-prim (first
              term) (length (rest term))))))
      ...)))
```

This code snippet uses the helper function `append-lists`, which turns a list of lists into a single list by concatenating the elements. We implement it using `fold`:

```
; append the elements of a list of lists together (: append-lists ((list-of (list-of
  %a)) -> (list-of %a)))
```

```
(define append-lists (lambda
  (list) (fold '() append
    list)))
```

Back to the translation: An abstraction is translated directly into an `abs` instruction, which contains the machine code for the body. We must therefore also translate the body...

Zen:

$$e \stackrel{\text{def}}{=} (v, e0) \text{ if } e = \dot{y}v.e0$$

```
(define term->machine-code
  (lambda (term)
    (cond
      ...
      ((abstraction? term)
        (list
          (make-abst (first (first (rest term)))
            (term->machine-code
              (first (rest (rest term))))))))))
```

The compiler is finished!

Task 15.10

Extend the compiler `term->machine code` to include binary branches. Write a few tests!

Ÿ

15.7 Execute SECD code

Unfortunately, a SECD machine is not available for purchase, so we simulate it with another program, a so-called *virtual machine*.

First, data definitions are needed for the four components of the machine – S, E, C, and D. We defined the set mathematically as follows:

$$S \stackrel{\text{def}}{=} W\ddot{y}$$

Accordingly, we define the stack as a list of values:

A stack is a list of values.

(define stack (signature (list-of value)))

The definition of values , corresponding to the mathematical definition of W, will come a little later.

Mathematically speaking, neighborhoods from the set E are sets of tuples:

$$E \ddot{y} V \times W$$

In the code, environments are lists of *bindings* that correspond to the tuples $V \times W$:

An environment is a list of constraints.

There is only one binding for each variable. (define environment (signature (list-of binding)))

A bond consists of:

; - Variable

; - Value

(define-record binding

make-binding binding? (binding-
variable symbol) (binding-value
value))

The empty environment is needed more often; therefore, we define it in advance:

```
; the empty environment
(define the-empty-environment empty)
```

(But `empty` is shorter, you might object. Yes, but then we can't tell from `empty` whether it's an empty list of instructions, an environment, or some other list entirely.)

There are two operations for an environment `e`: extending it by a binding `e[v ↦ w]` and looking up a binding `e(v)`. First, the extension: the implementation corresponds exactly to the mathematical definition. Here is a short description and signature:

```
; extend an environment by adding a binding (: extend-
environment (environment symbol value -> environment))
```

Here are a few test cases:

```
(check-expect (extend-environment the-empty-environment 'axl 59)
               (list (make-binding 'axl 59)))
(check-expect (extend-environment (extend-
                                   environment the-empty-environment
                                             'axl 60)
                                   'slash 57)
               (list (make-binding 'slash 57) (make-binding 'axl 60)))
(check-expect (extend-environment (extend-
                                   environment the-empty-environment 'axl 59)
                                   'axl 60)
               (list (make-binding 'axl 60)))
```

The definition removes any existing binding for `v` and adds a new one:

```
(define extend-environment
  (lambda (environment variable value)
    (cons (make-binding variable value)
          (remove-environment-binding environment variable))))
```

The helper function `remove-environment-binding` is responsible for removing the old binding. It follows the construction instructions. Here is the template:

```
; remove the binding for a variable from an environment (: remove-environment-binding
(environment symbol -> environment))
```

```
(define remove-environment-binding (lambda
  (environment variable)
  (cond
    ((empty? environment) ...) ((cons?
  environment)
  ...
  (first environment)
  (remove-environment-binding (rest environment) variable) ...))))
```

Since `(first environment)` is a binding, i.e., composite data, we can include calls for the selectors in the template:

```
(define remove-environment-binding (lambda
  (environment variable)
  (cond
    ((empty? environment) ...) ((cons?
  environment)
  ...
  (first environment) (binding-
  variable (first environment)) (binding-value (first
  environment)) (remove-environment-binding (rest
  environment) variable) ...))))
```

When defining the binding, we are interested in whether it belongs to the variable we are looking for. Accordingly, we complete the template:

```
(define remove-environment-binding (lambda
  (environment variable)
  (cond
    ((empty? environment) empty) ((cons?
  environment)
  (if (equal? variable (binding-variable (first environment)))
    (rest environment)
    (cons (first environment) (remove-
  environment-binding (rest environment)
  variable))))))
```

The second operation, checking for a binding in the environment, also follows the construction guidelines. Here is a brief description, signature, and test cases:

; find the binding for a variable in an environment (: lookup-environment (environment symbol -> value))

```
(check-expect
```

```
  (lookup-environment (list (make-binding 'slash 57) (make-binding 'axl 60))
```

```
    'axl)
```

```
  60)
```

```
(check-expect
```

```
  (lookup-environment (list (make-binding 'slash 57) (make-binding 'axl 60))
```

```
    'slash)
```

```
  57)
```

Here is the template:

```
(define lookup-environment
```

```
  (lambda (environment variable)
```

```
    (cond
```

```
      ((empty? environment) ...) ((cons?
environment)
```

```
  ...
```

```
  (first environment)
```

```
  (binding-variable (first environment)) (binding-value (first
environment)) (lookup-environment (rest environment)
```

```
variable)
```

```
...))))))
```

As with ``remove-environment-binding``, this function compares the binding variable with the variable being searched for. Here is the finished function:

```
(define lookup-environment
```

```
  (lambda (environment variable)
```

```
    (cond
```

```
      ((empty? environment) (violation "unbound variable")) ((cons? environment)
```

```
(if (equal? variable (binding-variable (first environment))) (binding-value (first environment))
    (lookup-environment (rest environment) variable))))))
```

This completes the operations on environments. Next up are dumps: D is defined as a sequence of frames:

$$D \stackrel{\text{def}}{=} (S \times E \times C)^*$$

A dump is a list of frames (define dump (signature
(list-of frame)))

From the frame triplet, we create a data definition and a record definition:

A frame consists of:

; - Stack

; - Vicinity

; - Code

```
(define-record frame
  make-frame frame?
  (frame-stack stack) (frame-
    environment environment)
  (frame-code machine-code))
```

Finally, a representation for the set W of values is still needed: A value is either a base value or a closure. Base values were already defined in Section 15.4 on page 484 ; closures, i.e., tuples from the set $V \times C \times E$, are still missing.

; A SECD value is a base value or a closure (define value (signature (mixed
base closure)))

A closure consists of:

; - Variable

; - Code

; - environment

```
(define-record closure)
  make-closure closure?
  (closure-variable symbol) (closure-
    code machine-code)
  (closure-environment environment))
```

Using these definitions, it is possible to specify a data definition and a record definition for the states of the SECD machine, i.e., the tuples from $S \times E \times C \times D$:

; A SECD condition consists of:

; - Stack

; - Vicinity

; - Code

; - Dump

(define-record secd

make-secd secd?

(secd-stack stack) (secd-

environment environment)

(secd-code machine-code)

(secd-dump dump)

Now we can finally write the state transition function. It accepts a SECD state and returns one as well. Here is a short description and signature:

; Calculate state transition (: secd-step

(secd -> secd))

We also need test cases. We'll create one test case per rule for $\tilde{y}$. Here's the first rule, for base values:

$$(s, e, bc, d) \tilde{y} (bs, e, c, d) _ _$$

To test this, we create a SECD state whose code consists only of the base value. This should then be on the stack:

(check-expect

(secd-step

(make-secd empty the-empty-environment (list 5) empty))

(make-secd (list 5) the-empty-environment empty empty))

Here is the rule for variables:

$$(s, e, vc, d) \tilde{y} (e(v)s, e, c, d) _ _$$

Therefore, if the code begins with a variable, the SECD engine must look it up in the environment. So we insert a variable called ``axl`` into the code and include an environment with a binding for ``axl``. The value of the binding must then be placed on the stack.

```
(check-expect
 (secd-step
  (make-secd empty (list (make-binding 'axl 60)) (list 'axl) empty))
 (make-secd (list 60) (list (make-binding 'axl 60)) empty empty))
```

Here is the rule for Primitiva:

$$(b_k \dots b_1 s, e, \text{primF } k \ c, d) \dot{\rightarrow} (bs, e, c, d) \text{ — — } \\ k \text{ where } F \dot{\rightarrow} \dot{\rightarrow}^k \text{ and } FB(b_1, \dots, b_k) = b$$

For the test, we'll use addition. It requires the two addends on the stack:

```
(check-expect
 (secd-step
  (make-secd (list 23 42) the-empty-environment (list (make-prim '+
    2)) empty)) (make-secd (list 65)
    the-empty-
    environment empty empty))
```

Next up is the abstraction rule:

$$(s, e, (v, c\dot{\rightarrow})c, d) \dot{\rightarrow} ((v, c\dot{\rightarrow}, e)s, e, c, d) \text{ — — }$$

For the test, we build an abstraction with the parameter `axl` and the free variable `slash`. The conversion-
The `gebung` contains a binding for `slash` – this should then be wrapped in the closure:

```
(check-expect
 (secd-step
  (make-secd empty (list (make-binding 'slash 57))
    (list (make-abst 'axl
      (list 'axl 'slash (make-prim '+ 2))))
    empty))
 (make-secd (list (make-closure 'axl
  (list 'axl 'slash (make-prim '+ 2)) (list (make-binding
    'slash 57))))
  (list (make-binding 'slash 57)) empty empty))
```

Finally, there is the application rule:

$$(w(v, c\bar{y}, e\bar{y})s, e, ap\ c, d)\bar{y}(\bar{y}, e\bar{y}[v\ \bar{y}\ w], c\bar{y}, (s, e\bar{c})d) \quad _ _$$

To test it, we place the closure from the previous test case onto the stack, along with one argument. Afterwards, the stack should be empty, the environment should have been inherited from the closure, the code should continue with the closure's code, and the dump should contain a frame.

```
(check-expect (secd-
  step (make-
    secd (list 60
      (make-closure 'axl
        (list 'axl 'slash (make-prim '+ 2)) (list (make-binding
          'slash 57))))
    the-empty-environment (list (make-ap)) empty)) (make-secd empty (list
  (make-binding 'axl 60)
    (make-binding 'slash 57)) (list 'axl 'slash (make-prim '+ 2)) (list (make-frame empty
      the-empty-environment empty))))))
```

According to the rules of the SECD machine, the body of ``secd-step`` must be a branch between the different cases in the code component of ``state``. These follow the construction guidelines for lists and mixed data. It is already clear from the rules that all rules require access to the components of ``state``. Local variables are created for these right at the beginning:

```
(define secd-step
  (lambda (state)
    (define stack (secd-stack state)) (define environment
      (secd-environment state))
    (define code (secd-code state))
    (define dump (secd-dump state)) (cond

      ((cons? code)
        (cond
          ((base? (first code)) ...)
          ((symbol? (first code)) ...))
```

```
((prim? (first code)) ...) ((abs? (first
code)) ...) ((ap? (first code)) ...)))
((empty? code) ...)))
```

We will now translate the rules into code one after the other. Here again is the rule for base values:

$$(s, e, bc_d) \dot{\rightarrow} (bs, e, c, d)$$

```
(define secd-step
  (lambda (state)
    ...
    (cond
      ((base? (first code))
       (make-secd (cons (first code) stack)
                   environment
                   (rest code)
                   dump)
       ...)
    ...))
```

Here again is the rule for variables:

$$(s, e, vc_d) \dot{\rightarrow} (e(v)s, e, c, d)$$

```
(define secd-step
  (lambda (state)
    ...
    (cond
      ((symbol? (first code)) (make-
        secd (cons
              (lookup-environment environment (first code)) stack)
              environment
              (rest code)
              dump)
        ...)
    ...))
```

The rule for primitive applications is somewhat more complex:

$$(b_k \dots b_1 s, e, \text{primF } k \ c, d) \dot{y} (bs, e, c, d) \quad \text{---} \quad \text{---} \\ \dot{y} \dot{y} \quad \text{kk where } F \quad \text{and } FB(b_1, \dots, b_k) = b$$

For the implementation, helper functions are needed that fetch the arguments from the stack and reverse their order, remove the arguments from the stack, and finally calculate the actual $\dot{y}$ -transition:

```
(define secd-step
  (lambda (state)
    ...
    (cond
      ...
      ((prim? (first code)) (make-
        secd (cons
              (apply-primitive (prim-
                operator (first code)) (take-reverse
                  (prim-arity (first code))
                    stack))
              (drop (prim-arity (first code)) stack))
              environment
              (rest code)
              dump)
        ...))
    ...))
```

The function ``drop`` is precisely the function required in exercise 7.5 on page 217 :

```
; drop the first elements of a list (: drop (natural (list-of %a) ->
(list-of %a)))
```

The ``take-reverse`` function is the counterpart to ``drop``, which returns the first n elements of a list in reverse order. We accomplish this using a tail-recursive helper function. From section 10.6 on page 308, we know that tail-recursive construction of lists always reverses the order:

```
; calculate the first elements of a list in reverse order (: take-reverse (natural (list-of %a) -> (list-of
%a)))
```

```
(check-expect (take-reverse 2 '(1 2 3 4 5)) '(2 1)) (check-expect (take-reverse 0
'(1 2 3 4 5)) '()) (check-expect (take-reverse 5 '(1 2 3 4 5)) '(5 4 3 2 1))
```

```
(define take-reverse
  (lambda (n list0)
    ;; (: loop (natural (list-of a) (list-of a) -> (list-of a))) (define accumulate

    (lambda (n list acc)
      (cond
        ((zero? n) acc)
        ((positive? n)
         (accumulate (- n 1) (rest list)
                     (cons (first list) acc))))))
    (accumulate n list0 '()))
```

Task 15.11

Program a different version of `take-reverse` by first writing a tail-recursive function `take` that returns the first n elements of the list for a given number n . Write

Then there 's `take-reverse` , a combination of `take` and `reverse`. What are the advantages and disadvantages?

Both versions of `take-reverse`?

Ÿ

From a primitive and a list of arguments, `apply-primitive` calculates the result of the primitive application. `primitive` is a case distinction, so the function body is a corresponding branch. For the signature of `apply-primitive`, we still need a signature that matches the predicate `primitive` (see page 483).

```
(define primitive (signature (predicate primitive?)))
```

```
; Calculate delta transition (: apply-primitive
(primitive (list-of value) -> value))
```

```
(check-expect (apply-primitive '+ '(1 2)) 3) (check-expect (apply-
primitive '- '(2 1)) 1)
```

```

(define apply-primitive
  (lambda (primitive args)
    (cond
      ((equal? primitive '+) (+ (first
                                args) (first (rest args))))
      ((equal? primitive '-') (- (first
                                args) (first (rest args)))) ((equal? primitive '=)

      (= (first args) (first (rest args)))) ((equal? primitive '*')
      (* (first args) (first (rest args))))
      ((equal? primitive '/') (/ (first args) (first (rest args)))))))

```

The rule for abstractions turns an abstraction into a closure:

$$(\underline{s}, e, (v, c\ddot{y}).c, \underline{d}) \ddot{y} ((v, c\ddot{y}).e) s.e, c, \underline{d})$$

The code does the same thing:

```

(define secd-step
  (lambda (state)
    ...
    (cond
      ...
      ((abs? (first code))
       (make-secd (cons
                   (make-closure (abst-variable (first code))
                                (abst-code (first code))
                                environment)
                   (stack)
                   environment
                   (rest code)
                   dump)))
      ...)))

```

Here is the rule for the application:

$$(w(v, c\ddot{y}, e\ddot{y}), s, e, ap\ c, d) \ddot{y} (\ddot{y}, e\ddot{y} [v \ddot{y} w], c\ddot{y}, (s, e, c)d)$$

For $e\ddot{y} [v \ddot{y} w]$ we use the [extend-environment](#) function, which we wrote above-have:

```
(define secd-step
  (lambda (state)
    ...
    (cond
      ...
      ((ap? (first code))
       (define closure (first (rest stack))) (make-secd empty
        (extend-environment
          (closure-environment closure) (closure-
            variable closure)
          (first stack)
          (closure-code closure) (cons
            (make frame (rest (rest stack))
              environment
              (rest code)
              dump)))
        ...))
    ...))
```

Finally, there's the code for returning a value from a function. Here's the rule:

$$(w, e, \ddot{y}, (s__, e\ddot{y}, c\ddot{y})d) \ddot{y} (ws\ddot{y}__e\ddot{y}, c\ddot{y}, d)$$

Here is the code for it:

```
(define secd-step
  (lambda (state)
    ...
    (cond
```

```

...
(empty? code)
(define frame (first dump))
(make-secd
  (cons (first stack)
        (frame-stack frame)
        (frame-environment frame)
        (frame-code frame) (rest
                              dump))))
...))

```

In order for the SECD machine to be put into operation, a term e must first be translated into an initial state $(\ddot{y}, \ddot{y}, e, \ddot{y})$. The following helper function accomplishes this:

```

; Make SECD initial state from term (: inject-secd
(term -> secd)) (define inject-secd (lambda
(term)

```

```

  (make-secd empty the-
              empty-environment (term-
                                  >machine-code/t term)
              empty)))

```

Now we can try out the machine:

```

(secd-step (inject-secd '(+ 1 2)))  $\ddot{y}$  #<record:secd
(1) () (2 #<record:prim + 2>) ()> (secd-step (secd-step (inject-secd '(+ 1
2))))  $\ddot{y}$  #<record:secd (2 1) () (#<record:prim + 2>) ()> (secd-step
(secd-step (secd-step (inject-secd '(+ 1 2)))))  $\ddot{y}$  #<record:secd (3) () ()>

```

The evaluation function `evalSECD` is still missing, which requires an auxiliary function to calculate the reflexive-transitive closure of the state transition $\ddot{y}\ddot{y}$:

```

; calculate state transitions until the end (: secd-step* (secd
-> secd)) (define secd-step*

(lambda (state)

```

```

(if (and (empty? (secd-code state)) (empty? (secd-
      dump state)))
    state
    (secd-step* (secd-step state))))

```

The evaluation function is directly based on the mathematical definition:

Calculate the evaluation function for the SECD machine

```
(: eval-secd (term -> (mixed value (enum 'function))))
```

```

(check-expect (eval-secd '(+ 1 2)) 3) (check-expect (eval-
secd '(((lambda (x) (lambda (y) (+ xy))) 1) 2))
      3)

```

```

(define eval-secd
  (lambda (term)
    (define value (first
      (secd-stack
        (secd-step*
          (inject-secd term))))))
    (if (base? value)
        value
        'function)))

```

This is how the SECD machine runs:

```
(eval-secd '(((lambda (x) (lambda (y) (+ xy))) 1) 2)) y 3
```

Complete!

Task 15.12

Extend the definition of `secd-step` to include the instruction for binary branches.

Ÿ

15.8 THE END RECURSIVE SECD MACHINE

The SECD engine has a flaw: In end-cursive applications, it should ideally not consume any additional space, as in language learning programs, since no context is needed.

The following example evaluation for the term $(\tilde{y}x.xx)(\tilde{y}x.xx)$ shows, however, that the dump as the evaluation progresses, it becomes increasingly larger:

$(\tilde{y},$	$\tilde{y},$	$(x, xx\ ap)$
		$(x, xx\ ap)$
		$ap,$
$\tilde{y}\ ((x, xx\ ap, \tilde{y}), \tilde{y}, \tilde{y}\ ((x, xx\ ap, \tilde{y})\ (x, xx$		$\tilde{y})$
$ap, \tilde{y}), \tilde{y}, \tilde{y}\ ((x, xx\ ap, \tilde{y}), \tilde{y}\ ((x, xx$		$(x, xx\ ap)\ ap, \tilde{y})$
$ap, \tilde{y})$		$ap,$
$(x, xx\ ap, \tilde{y}), \tilde{y}\ ((x, xx$	$\{(x, (x, xx\ ap, \tilde{y}))\}, xx\ ap,$	$\tilde{y})$
$ap, \tilde{y}), \tilde{y}\ ((x, xx$	$xx\ ap, \tilde{y}), x\ ap,$	$(\tilde{y}, \tilde{y}, \tilde{y}))$
$ap, \tilde{y}), \tilde{y}\ ((x, xx$		$(\tilde{y}, \tilde{y}, \tilde{y}))$
	$\{(x, (x, xx\ ap, \tilde{y}))\}, xx\ ap,$	$(\tilde{y}, \tilde{y}, \tilde{y}))$
$\tilde{y}\ ((x, xx\ ap, \tilde{y}),$	$\{(x, (x, xx\ ap, \tilde{y}))\}, x\ ap,$	$(\tilde{y}, \{(x, (x, xx\ ap, \tilde{y}))\}, \tilde{y})$
		$(\tilde{y}, \tilde{y}, \tilde{y}))$
$\tilde{y}\ ((x, xx\ ap, \tilde{y})\ (x, xx\ ap, \tilde{y}), \{(x, (x, xx\ ap, \tilde{y}))\}, ap,$		$(\tilde{y}, \{(x, (x, xx\ ap, \tilde{y}))\}, \tilde{y})$
		$(\tilde{y}, \tilde{y}, \tilde{y}))$
$\tilde{y}\ (\tilde{y},$	$\{(x, (x, xx\ ap, \tilde{y}))\}, xx\ ap,$	$(\tilde{y}, \{(x, (x, xx\ ap, \tilde{y}))\}, \tilde{y})$
		$(\tilde{y}, \{(x, (x, xx\ ap, \tilde{y}))\}, \tilde{y})$
		$(\tilde{y}, \tilde{y}, \tilde{y}))$
$\tilde{y}\ ((x, xx\ ap, \tilde{y}),$	$\{(x, (x, xx\ ap, \tilde{y}))\}, x\ ap,$	$(\tilde{y}, \{(x, (x, xx\ ap, \tilde{y}))\}, \tilde{y})$
		$(\tilde{y}, \{(x, (x, xx\ ap, \tilde{y}))\}, \tilde{y})$
		$(\tilde{y}, \tilde{y}, \tilde{y}))$
...		

Fortunately, we can fix this shortcoming: The SECD machine must be tail-recursive and To handle "normal" applications differently, we introduce a new instruction, na-mens tailap, which executes an application like ap, but signals tail recursion:

```

 ỹỹỹ ...
  | tailap

```

Next, the translation function for terms into machine code needs to be changed: Applications that have context around them should be translated with ap, those without. Context with tailap. Since the application alone doesn't reveal the context, but only the context... For the term "around", the translation function is split into two parts: for one term The evaluation function is always used when there is context. Another A function is always called when there is no surrounding context.

Context always arises through applications. When evaluating a term (e0 e1), the following must be considered: After e0, e1 must still be evaluated, and after evaluating e1 , the application must still be processed.

This process is carried out. Both e_0 and e_1 are contextualized. The same applies to the arguments. of primitive applications.

On the other hand, abstractions initially cut off the context for their core: Whether the body has context or not is only determined during application. Accordingly, Applications and abstractions switch between the two functions and back and forth. her:

	$\tilde{y} \ b$	if $e = b \ \tilde{y} \ B$
	v	if $e = v \ \tilde{y} \ V$
$e \ \underline{\text{def}}$	$_ \ e_0 \ e_1 \ \text{ap}$	if $e = (e_0 \ e_1)$
	$e_1 \ \dots \ e_k \ \text{primF } k$	if $e = (F \ e_1 \ \dots \ e_k)$
	$_ (v, e_0 \ \tilde{y})$	if $e = \tilde{y}v.e_0$
	$\tilde{y} \ b$	if $e = b \ \tilde{y} \ B$
	v	if $e = v \ \tilde{y} \ V$
$e \ \tilde{y} \ \underline{\text{def}}$	$_ \ e_0 \ e_1 \ \text{tailap}$	if $e = (e_0 \ e_1)$
	$e_1 \ \dots \ e_k \ \text{primF } k$	if $e = (F \ e_1 \ \dots \ e_k)$
	$_ (v, e_0 \ \tilde{y})$	if $e = \tilde{y}v.e_0$

Now we need a state transition rule that can process tailap. This is derived from...

Directly from the rules for `ap` and the return of a value: `tailap` works just like `ap` directly. followed by the return rule. Here are the two rules again as a reminder:

$$\begin{aligned}
 & (w(v, \underline{c\tilde{y}}, e\tilde{y})s, e, \text{ap } \underline{c}, \underline{d}) \ \tilde{y} \ (\tilde{y}, e\tilde{y} [v \ \tilde{y} \ w], \underline{c\tilde{y}}, (s, e, \underline{c})d) \ _ \ _ \\
 & (w, e, \tilde{y}, (s \ \tilde{y}, e\tilde{y}, \underline{c\tilde{y}})d) \ \tilde{y} \ (ws\tilde{y}, e\tilde{y} \ _, \underline{c\tilde{y}}, \underline{d})
 \end{aligned}$$

Since the first rule creates a new dump frame and the second rule "destroys" a dump frame, This work is usually unnecessary for tailap:

$$(w(v, \underline{c\tilde{y}}, e\tilde{y})s, e, \text{tailap } c, d) \ \tilde{y} \ (s, e\tilde{y} [v \ \tilde{y} \ w], \underline{c\tilde{y}} \ _, _, \underline{d})$$

This means the example still runs endlessly, but at least without using up more and more space:

(ȳ,	ȳ,	(x, xx tailap)	
		(x, xx tailap)	
		ap,	ȳ)
ȳ ((x, xx tailap, ȳ),	ȳ,	(x, xx tailap)	
		ap,	ȳ)
ȳ ((x, xx tailap, ȳ) (x, xx tailap, ȳ), ȳ, ȳ (ȳ, ȳ (ȳ ((x, xx		ap,	ȳ)
tailap,	{(x,(x, xx tailap, ȳ))}, xx tailap, {(x,(x, xx tailap,		(ȳ, ȳ, ȳ))
ȳ), ȳ ((x, xx tailap, ȳ) (x,	ȳ))), x tailap,		(ȳ, ȳ, ȳ))
xx tailap, ȳ), {(x,(x, xx tailap, ȳ))}, tailap, ȳ (ȳ, ȳ ((x, xx tailap, ȳ), ȳ ((x, xx tailap, ȳ) (x, xx			(ȳ, ȳ, ȳ))
tailap,	{(x,(x, xx tailap, ȳ))}, xx tailap, {(x,(x, xx tailap,		(ȳ, ȳ, ȳ))
ȳ), {(x,(x, xx tailap, ȳ))},	ȳ))), x tailap,		(ȳ, ȳ, ȳ))
tailap, ȳ (ȳ, ȳ ((x, xx tailap, ȳ), ȳ ((x, xx tailap, ȳ) (x, xx tailap, ȳ), {(x,(x, xx tailap, ȳ))}, tailap,			(ȳ, ȳ, ȳ))
ȳ (ȳ,	{(x,(x, xx tailap, ȳ))}, xx tailap, {(x,(x, xx tailap,		(ȳ, ȳ, ȳ))
ȳ ((x, xx tailap, ȳ),	ȳ))), x tailap,		(ȳ, ȳ, ȳ))
			(ȳ, ȳ, ȳ))
	{(x,(x, xx tailap, ȳ))}, xx tailap, {(x,(x, xx tailap,		(ȳ, ȳ, ȳ))
	ȳ))), x tailap,		(ȳ, ȳ, ȳ))

The implementation of the tail-recursive SECD machine is the subject of exercise 15.13.

EXERCISES

Task 15.13

Extend the implementation of the SECD machine to include correct handling of tail recursion! First, extend the data definition for machine code. Then, implement the translation of $\tilde{y}$ terms for the tail-recursive SECD machine. Finally, extend the state transition function to include a case for the tailap instruction.

Task 15.14

Extend the SECD machine to include primitives of other arity levels, for example [abs](#) or [odd?](#)

Task 15.15

Extend the applied $\tilde{y}$ -calculus to include abstractions and applications with more than one Parameters. Extend the SECD machine and its implementation accordingly.

Task 15.16

Show, using the SECD machine extended with tail recursion, that `tailap` always comes at the end, meaning it actually has no context.

Task 15.17

The SECD machine, extended with tail recursion, has a new instruction called `tailap`. However, this is not strictly necessary, because tail-recursive `ap` instructions can also be identified by their placement at the end of the code.

Reformulate the state transition rules of the SECD machine with tail recursion so that the functionality, in particular the correct handling of tail-recursive applications, works with the result of the normal compiler, which does not generate a `tailap` instruction.

Task 15.18

Instead of representing environments through lists of bindings, it is also possible to use functions, so that `lookup-environment` looks like this:

```
(define lookup-environment
  (lambda (environment variable)
    (environment variable)))
```

Add a suitable definition for `extend-environment`.

Task 15.19

At first glance, it seems somewhat cumbersome to wrap the entire environment in the closure every time an abstraction is evaluated. What would change if this step were omitted, so that closures only contained variables and machine code for the body? Formulate the corresponding rules for the SECD machine and modify the implementation accordingly. Does the SECD machine still function correctly after the change?

EPILOGUE

That's it. For now. We hope this book has helped you learn to program. Of course, there's much more to know and learn about programming, but

didn't fit into this book. Here are some suggestions on how it could continue, together with...

Some book recommendations on the subject:

If you've programmed before, then you'll certainly find what you're looking for in this book.

Programming with *variable assignments* is missing, as are input and output, and graphical interfaces.

User interfaces and other interactions with the "outside world." In programming jargon, these are all called *effects*, and they are part of the daily work of programming.

in addition. Unfortunately, effects make programming significantly more difficult and complex; we must

Therefore, use them sparingly and carefully. You can learn how to handle effects directly in the

Practice the racket system; there is plenty of information in the help center and also literature [\[FHB13\]](#). A

good starting point for programming with mutable variables is the

Rust programming language, there is also a good book about it [\[KN18\]](#).

Many programming languages use *types*. Types are similar to the signatures of this book, a notation for the permissible inputs and outputs of functions. While signatures are checked at runtime in our language, types are checked even before the code is executed.

The program is executed. Types are a powerful tool for data analysis and...

Developing correct programs is important. However, certain types can sometimes take some getting used to.

because we cannot test a program that is not type-correct, which occasionally causes the

Programming effort is increased, and it makes it more difficult to find the causes of errors. Among the typed languages, *Scala* [\[CB14\]](#) and *Haskell* [\[Hut16\]](#) are particularly interesting and instructive.

In this book, we have only written small programs, each of which fits into a single file.

It fits. When a simple program becomes a *larger system*, we need to think about it.

how can we organize things so that we don't constantly have to keep everything in mind that's going on?

This is what happens to the system when we further develop it. These considerations fall under the umbrella term of *software architecture*, and there's still much more to learn there as well. One particularly interesting topic at the moment is...

Collection of techniques under the heading *Domain-Driven Design* [\[Eva04\]](#).

Furthermore, there are of course many other programming languages with sometimes different ideas than the one mentioned.

The languages taught in this book include *object-oriented programming* and *logic programming*. The most

readily available are the other programming languages of the Racket system, including the "main language," which is also called Racket. You can find information about this in the *Racket Guide* in the Help Center.

Racket is also a good starting point for object-oriented programming; see the Racket Guide under *Classes and Objects* in the Help Center. With Racket, you can...

You can also install the logic programming language *MiniKanren* , and there's a great book about it [\[FBKH18\]](#). Anyone generally interested in programming languages and their concepts will find relevant books by Christian Wagenknecht [\[Wag16\]](#) and Norman Ramsey [\[Ram22\]](#) helpful. The former even incorporates all the concepts into Racket.

Have fun!

BIBLIOGRAPHY

Adams, Stephen: *Efficient sets—a balancing act*. Journal of Functional Programming, 3(4):553–561, October 1993.

Bird, Richard and **Jeremy Gibbons:** *Algorithm Design with Haskell*. Cambridge University Press, June 2020.

Chiusano, Paul and **Rúnar Bjarnason:** *Functional Programming in Scala*. Manning, 2014.

Claessen, Koen and **John Hughes:** *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs*. In: *Proceedings of the International Conference on Functional Programming (ICFP)*, pages 268–279, 2000.

Church, Alonzo: *The Calculi of Lambda Conversion*. Princeton University Press, Princeton, New York, 1941.

Evans, Eric: *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison Wesley, 2004.

Friedman, Daniel P., William E. Byrd, Oleg Kiselyov and **Jason Hemann:** *The Reasoned Schemer*. 2nd edition. MIT Press, 2018.

Felleisen, Matthias, Robert Bruce Findler, Matthew Flatt and **Shriram Krishnamurthi:** *How to Design Programs*. MIT Press, 2nd edition, 2014.

Felleisen, Matthias, David Van Horn and **Conrad Barski:** *Realm of Racket*. No Starch Press, 2013.

Goldberg, David: *What every computer scientist should know about floating-point arithmetic*. ACM Computing Surveys, 23(1):5–48, March 1991.

Hofstadter, Douglas R.: *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books, New York, 1979.

Hindley, JR and **JP Seldin:** *Introduction to Combinators and λ -Calculus*, Volume 1 of the *Mathematical Sciences Student Texts series*. Cambridge University Press, London, 1986.

Hutton, Graham: *Programming in Haskell*. Cambridge University Press, 2016.

Klabnik, Steve and Carol Nichols: *The Rust Programming Language*. No Starch Press, 2018.

Landin, Peter: *The mechanical evaluation of expressions*. The Computer Journal, 6:308–320, 1964.

Landin, Peter: *The next 700 programming languages*. Communications of the ACM, 9(3):157–166, March 1966.

Haskell 2010 Language Report, 2010. <https://www.haskell.org/onlinereport/haskell2010/>.

McCarthy, John: *Recursive functions of symbolic expressions and their computation by machine, Part I*. Communications of the ACM, 3(4):184–195, April 1960.

Peyton Jones, Simon and Jean-Marc Eber: *How to write a financial contract*. In: **Gibb-ons, Jeremy and Oege de Moor** (editors): *The Fun of Programming*. Palgrave Macmillan, 2003.

Ramsey, Norman: *Programming Languages: Build, Prove, and Compare*. Cambridge University Press, 2022.

Wagenknecht, Christian: *Programming Paradigms: An Introduction Based on by Racket*. Springer Verlag, 2016.

VARIABLE INDEX

<, 52	cheap-power, 41, 44 binding,
<=, 52	489 blank-road-
=, 52	window, 333 boolean, 53
>, 52	
>=, 52	boolean?, 128
	both, 157, 169
abs, 288, 346	
abst, 485	car, 337
abstraction, 483	car-length, 337 car-
abstraction?, 482 addition,	on-position?, 345 car-width, 337
479	
addition?, 478	check-error, 82
everything, 20	check-expect, 36 check-
an-unknown-parent, 358	property, 404 circle, 318
ancestor?, 360	circle-area,
and, 52	274
animal-weight, 122	closure, 493
animate, 335	color, 318
ap, 485	compute, 481
append, 192	concatenate, 190, 192
append-element, 291	cond, 70
append-lists, 488	confluence, 143
application, 483	confluence-location, 365 confluence-
application?, 483 apply-	main-stem, 365 confluence-tributary,
primitive, 499	365
	cons, 175
balanced-search-tree-insert, 388 balanced-search-	cons-list, 174, 175 cons-
tree?, 421	list-of, 180
band-sorted?, 414 base,	cons?, 175
484	contract, 158
base?, 484	contract-payment, 161 contract-
between, 211, 213	remainder, 166
big-bang, 350	copies, 209

- count-from, 212
- Creek, 142
- curry, 272
- cute?, 59, 75
- date, 155
- date<=?, 167
- dead-eyes, 329
- define-record, 88, 96, 121, 182 depth, 366 dillo, 102, 120, 326 dillo body, 327 dillo image, 329 dillo on road, 331 dillos-on-road, 342 dizzy (make-band-member Dizzy Reed1963)), 410 drawn-games, 253, 254 duff (make-band-member Duff McKagan1964)), 410 dump, 493 ellipse, 318 else, 69 empty-image, 430 empty-list, 174 enum, 57 environment, 489 equal?, 476 eschach, 143 euros, 153 eval-secd, 503 even?, 217 evens, 310 everyone?, 425 exchange-for-cute, 78 expect, 428 expression, 479 extend-environment, 490 extract-games, 256 false, 370 feed-animal, 125 feed-dillo, 105, 109 feed-parrot, 123 filter, 259 first, 174, 175 flows-from?, 147 fold, 270 for-all, 402, 403 frame, 493 from, 284 game, 246 game draw?, 252 gap height, 332 guest points, 249, 251 guns-n-roses, 410 heat->temperature, 72, 401 heat-water, 69, 82 heat-water-0, 60 heat-water-1, 62 heat-water-2, 73 highway75, 186 home points, 247–249 home-won-games, 254 home-won?, 254 hour, 95 if, 78 image-height, 321 image-width, 321

initial-world, 342
 inject-secd, 502
 insert, 410
 instruction, 484
 integer, 35
 integer-from-to, 55 invert,
 289, 290 invert-
 helper, 294

 lambda, 27
 later, 155
 length, 186, 192
 line, 319
 list, 184
 list->balanced-search-tree, 420 list-apply,
 263, 269 list-fold-left, 311
 list-length, 184, 185 list-
 min, 196 list-min-
 nonempty, 197
 list-of, 181

 list-of-numbers, 175, 183 list-
 product, 179, 265 list-sum,
 177, 265, 297, 298 list-sum-
 helper, 295, 298 list-team-goals,
 262 list-total-goals, 260 live-
 dillos, 190, 257 live-dillos-
 under-car-count, 347

 London-Hudson, 358
 Lookup Environment, 492, 507

 machine code, 484
 make-balanced-node, 394, 423 make-
 both, 170
 make-compute-points, 250

 make-multiple, 169
 make-sized-node, 384 map,
 265
 marking count, 333
 marking height, 332
 marking-segment-pixels, 334 markings,
 332 max-profit,
 306 member?, 368
 meters->pixels,
 331 meters-per-tick, 336
 minute, 95 minutes-since-
 midnight,
 96, 97 minutes-since-midnight->wallclock-
 time,
 98
 mixed, 119
 mode, 317
 model, 92
 mouse-event, 354
 multiple, 154, 168
 multiplication, 479
 multiplication?, 479

 natural, 35
 natural?, 128
 neckar-1, 143
 next-world, 354
 no-slope, 133
 node-of, 363
 not, 53
 nothing, 157
 nth, 210
 number, 35
 number?, 128

 odd?, 217

- on-key, 350
- one-euro, 152
- or, 52
- overlay, 322
- overlayxy, 322
- pad event, 355
- parent, 358, 362
- parent-ancestor?, 361
- parrot, 117, 120
- person, 361
- pet, 57, 75
- pixels->meters, 331 place-
car-on-road, 340 place-dillos-on-
road, 341 place-image, 323 place-
image-align, 323 place-
image-on-road, 339 place-score,
341 plays-game?, 258 plays-game?/
team, 270 plays-
nürnberg?, 258
- polygon, 319 position, 330
- positive?, 206
- posn, 319
- power, 207, 301
- power2, 216
- predecessor, 206 prim,
143, 485 prime-
powers-stream, 286 primitive, 499
- primitive-application ,
483 primitive-application?, 483 primitive?,
483 proper-sized-search-tree?, 419
- pulled-point, 320
- rational, 35
- rational?, 128
- react-to-key, 352 real,
35 real?,
128
- rectangle, 317
- remove-environment-binding, 491
- rest, 174, 175
- reverse, 295
- river, 142
- river?, 365
- road-width, 333
- road window, 335 road
window at ticks, 336
- road-window-height, 333 run-over-
dillo, 105, 106, 326
- run-over-dillos, 188
- run-over-dillos-on-road, 349
- say-number, 54, 55
- sneak, 143 search-
tree-member?, 378
- search tree of, 375 secd,
494 secd-
step, 496, 500 secd-step*,
502 side, 330
- signature, 57
- sized-label-of, 383 sized-
node-label, 386 sized-node-of,
385
- sized-search-tree-member?, 387 sized-search-
tree-of, 386 sized-tree-of, 385

sized-tree-size, 385
slash, 358
slope, 134
sort band, 412
sqrt, 171
square, 437
stack, 489
standard computer, 93 stream-
take, 284
string, 35
string->number, 23 string-
append, 22 string-length,
23 string-list, 180
string=?, 52, 59 string?,
128 electricity tariff
bill amount, 43

successor, 205
sugar-content, 127 sugar-
traffic-light, 129, 132 sugar-weight->traffic-
light, 130 sugars, 126 symbol, 475

take-reverse, 498 team
goals, 261 team
goals/team, 271
temperature->heat, 72 term,
482 term-
>machine code, 486
text, 319
the-empty-environment, 490

ticks->meters, 336
tile, 27 to-
draw, 350 total-
goals, 260 total-memory,
90, 91 traffic-light, 126 tree,
363

tree-insert, 374 tree-
member?, 371 tree-of,
363
triangle, 318
true, 370

uncurry, 274
unknown-parent, 357

value, 493
visible markings, 333

wallclock-time, 94, 95 watt-for-
little, 42, 44 wheel, 337 wheels-
on-one-side,
337 world, 342

world->image, 343 world-
at-ticks, 344 world-car-
position, 343

x-place, 324

y-place, 324

zero-bond, 151
zero?, 206

INDEX

M^* , 229	$\tilde{y}$ -reduction, 448
$\tilde{y}$, 221	Alternative, 79
$\tilde{y}$, 220	applied $\tilde{y}$ -calculus, 465
$\underline{\text{def}}$, 77	View, 327
$\tilde{y}$, 229	Answer, 473
$\tilde{y}$, 219	Apostrophe, 474
$\tilde{y} \tilde{y}$, 457	applicative-order reduction, 454
$\tilde{y}$, 446	Application, 30, 441
$\tilde{y}\tilde{y}$,	Application rule, 471
219 $\tilde{y}\tilde{y}$, 448	Argument, 30, 31
$\tilde{y}\tilde{y}$, 448 \,	Associativity, 427
221	List, 57, 75
$\tilde{y}$, 220	Expression, 16
$\tilde{y}$, 220	Evaluation function, 472
$\tilde{y}$, 220	Evaluation strategy, 452
$\tilde{y}$, 220	
$\times$, 221	balance
$\tilde{y}$, 220	of search trees, 381
	Base, 205
Cover, 37, 56 derived	Basic element, 228
form, 80 completed,	Underlying value, 465
445	Tree, 357
Diploma	size-annotated, 383
reflexive-transitive-symmetric, 450	Condition, 51
symmetric, 450	computable functions, 460 $\tilde{y}$ -
transitive-reflexive, 450	reduction, 448
Absolute amount, 79,346	Binding, 275, 489
Abstraction, 27, 30, 31, 43, 441	dynamic, 277
Abstraction rule, 471	lexical, 277
countable set, 441	static, 277
Accumulator, 293	Binary tree, 357, 362
Algebra, 68, 429	binary branching, 77
Algorithm, 382	Sheet 363

- Boolean case distinction, 77 Boolean expression, 51 Boolean value, 51 , 456 bound , 445
- Bruch, 35
- Call-by-value reduction, 455 , 467
- cat, 232
- Church Number 457
- Church/Rosser property, 451
- Closure, 468
- Code, 468
- Compiler, 486
- Computer, 87
- Cons list, 174
- curryficate, 272
- Data, 31
- Data analysis, 74
- Data definition, 57 , 73
- Definition, 19
 - local, 195
- Definition window, 15 ÿ-reduction, 466
- De Morgan's Law, 242
- Difference, 221
- Distributivity, 437
- Domain logic, 326
- DrRacket, 11 , 15
- Dump, 468
- Average, 221 dynamic bond, 277
- proper subset, 220
- Editor, 15
- Effect, 509
- Property, 403
- Input, 16 built-in predicates, 128
- Element, 219
- Ellipsis, 41
- else branch, 70
- finite sequence, 229
- finite set, 220
- Tail recursion, 308
- degenerate search tree, 381
- Environment, 468
- exclusive or, 53
- Exponent, 205
- fac , 458
- Faculty, 216 , 302
- Case distinction, 51 , 54 , 74 , false, 118
- 51 false, 51
- false, 456
- falsifiable, 404
- Field, 88
- Fibonacci sequence, 306
- Fibonacci function, 243
- Fixed point, 459
- Fixed-point combinator, 288 , 460
- Fixed-point theorem, 460
- Episode 229
- form
 - derived, 80
- Frame, 468
- free, 445
- free variable, 445
- Function, 31
 - higher order, 245 , 250
- Functional application, 30
- Function call, 30

- integers, 35
- integers, 220
- Gaussian summation formula, 221,
- 241 bound variable, 445
- bound occurrence, 275 mixed
- data, 117, 118
- Scaffold, 33,
- 41 global variable, 275
- Grammar, 233
- size-annotated tree, 383

- Semigroup, 429
- Higher order function, 245, 250
- Homomorphism, 431
- Height
 - of a tree, 365

- idempotent, 288
- if, 456
- image.rkt, 317
- Index, 209
- Induction, 225
 - noetherian, 243
 - structural, 237
 - complete, 225
 - well-founded, 243
- Induction start, 225, 231
- Induction axiom, 240
- Induction circuit, 225, 231
- Inductive anchoring, 228
- inductive definition, 228
- inductive set, 228
- inductive closure, 228, 240
- inexact number, 407
- Information, 31
- inclusive or, 53

- Insertionsort, 410
- Instruction, 467
- Interaction window, 15
- Invariant, 297
- Inverse, 409
- Isomorphism, 274
- Iteration, 308

- Cartesian product, 221
- Klammer, 16
- Class 118
- Nodes, 362
- Combinator, 46, 141, 144, 150, 427
- Comment, 32
- complex numbers, 35
- Component, 88
- Consistent, 79
- Construction instructions, 39
- Constructor, 89
- Context, 308, 468 ;
- context-free grammar, 233
- Brief description, 34, 40

- L $\ddot{y}$ A, 466
- L $\ddot{y}$, 441
- $\ddot{y}$ -calculus,
 - 439 $\ddot{y}$ -term,
 - 441 lazy evaluation,
 - 455 empty
- sequence, 229
- empty list, 173
- empty set, 220 leftmost-innermost
- reduction, 454 leftmost-outermost reduction, 453
- Languages
 - taught, 15 lexical binding, 277, 447,
 - 469 linear growth, 295

- Left wing reduction, 453
- Left inner reduction, 454
- List, 173
 - empty, 173
- Literal, 22
- Literal rule, 471
- Logarithm, 380 local
- variable, 195 , 275 local
- variable, 195
- Mantra, 44
- Marking, 362
- Machine code, 468
- Machine language, 465 , 467
- Quantity,
 - 219 finally, 220
- Metavariable, 443
- Model, 326
- modulo, 451
- Monoid, 430
- N, 219
- Successors, 240 , 457
- natural numbers, 35
- natural number, 34 , 205 , 219 , 457
- Negation, 53
- neutral element, 180 , 429 non-strict
- language, 455 Noetherian
- induction, 243 Noetherian order,
 - 243 normal-order reduction, 453
- Normal form, 451
- Operand, 16
- Operator, 16
- Noetherian
 - order, 243
 - well-founded, 243
- Parrot, 117
- Parameters, 30 , 31 ,
 - 441 parametric polymorphism, 185
- Peano axioms, 240
- Period, 18
- Path, 365
- Polymorphism, 185
- positive natural numbers, 206
- Potency, 205
- pred, 457
- primitive application, 466
- Primitive rule, 471
- Primitive, 465
- property-based testing, 404
- Predicate, 119 , 478
- Predicates
 - built-in, 128
- quadratic growth, 292
- quote , 475
- R , 220
- Racket, 12
- rational numbers,
 - 35 rational numbers, 35
- Record Definition, 88 , 96
- Record signature, 89
- Record type, 88
- Records, 88
- Red-Black-Trees, 397
- Redex, 449
- Reduction calculus, 448
- Redundancy, 415 , 418
- real numbers, 35 , 220

- reflexive-transitive-symmetrical closure,
450
- Recursion
 - structural, 235
- recursive call, 147 relational
- problem, 413
- REPL, 15
- Representation, 31, 474
- representable value, 477
- Rosenbaum, 398
- rotation
 - of a tree, 382 of a
 - search tree, 391
- Rumpf, 30, 31, 33, 41, 441
- Return rule, 472
- Template, 76
- Scheme, 12
- Bow, 308
- Schönfinkeln, 270, 272, 444
- Self-reference, 141, 143, 228
- self-quoting, 475
- Selector, 90
- Semantic brackets, 236
- Visibility, 275
- Signature, 34, 35
- Signature definition, 57
- Signature declaration, 35, 40
- Signature Constructor, 181
- Signature parameters, 181
- Signature variable, 182
- Signature violation, 38
- smart constructor, 170
- Software Architecture, 509
- Variety, 34
- Language level, 184
- Beginners, 15
- advanced, 473
- Stack, 468
- static bond, 277
- Number of digits, 485
- Stepper, 28
- strict language, 455
- String, 22
- structural induction, 237 structural
- recursion, 235
- Substitution, 446
- succ, 457
- Search tree, 375
 - degenerate, 381
- Total, 16
- Symbol, 475
- symmetrical termination, 450
- syntactic sugar, 80
- tail call*, 308
- Teachpack, 23, 317
- Subtree, 362
- subset, 220 real,
220
- Test, 36
- Test case 36
- Tests, 40
- Text, 22
- Ticks, 336
- depth
 - of a tree, 365
- Animal,
118 transitive-reflexive closure, 450
- true, 51
- true, 456
- Tuple, 221

Turing machine, 460

Type 509

surroundings, 468

Unit test, 402

Variable, 19

free, 445

bound, 445

global, 275

local, 275

Variable rule, 471

Association, 220

Branching, 51, 54, 70, 456

View, 327

virtual machine, 489

predecessor, 457

Occurrences, 275, 446

true, 51

Truth value, 51

well-founded induction, 243

root, 364

Y, 460

Z, 219

string, 22

zerop, 457

Sugar, syntactic, 80

composite data, 87

Base-2 logarithm, 380

Zweig, 54, 70

Interim result, 292, 295